

LLM 파인튜닝 마스터, sLLM 만들기

변형호



변형호



<https://github.com/NotoriousH2>



Hyungho Byun

[학력]

- 경기과학고등학교 졸업
- KAIST 전산학부 졸업 (수리과학과 부전공)
- 서울대학교 컴퓨터공학부 박사 졸업 (2023. 2)

[경력]

- 노토랩 대표
- 삼성SDS, SK Telecom, 신한투자증권, GS그룹, HL그룹, 한국엔컴퍼니 등 LLM 강의
- POSCO, 국가보안기술연구소 기술 자문
- KT AI 연구소 기술 세미나
- 삼성전자 LLM 트렌드 세미나 진행
- 두산백과 정보과학/인공지능 분야 집필진
- HD현대 산업전문인력 AI역량강화 사업 강의

Contents

● Introduction

- 과정 소개
- LLM 개론과 이론적 배경

● OpenAI API

- OpenAI API 소개
- GPT 모델을 이용한 LLM 어플리케이션 만들기

● 프롬프트 엔지니어링

- 프롬프트 엔지니어링의 구성 요소
- 최신 프롬프트 엔지니어링 기법

● LangChain

- LangChain 기초
- LangChain Expression Language

Contents (2)

● RAG

- RAG (Retrieval-Augmented Generation) 소개
- 최신 RAG 성능 향상 기법

● LLM Agent

- LLM Tool Calling과 Agent
- LLM Agentic Work
- LangGraph 멀티 에이전트

● Open-Weight LLMs

- 주요 오픈 LLM 소개 및 비교
- HuggingFace를 이용한 LLM 모델 로드

Contents (3)

● Fine Tuning

- LLM 파인 튜닝 기법과 PEFT
- 파인 튜닝 데이터 구성 방법

● Alignment

- LLM 모델 Alignment를 위한 DPO/PPO 데이터 구성
- LLM Alignment 실습

● Serving

- 모델 사용 효율을 높이는 양자화/서빙 라이브러리
- vLLM을 이용한 LLM 추론 효율 높이기
- LangSmith를 이용한 성능 평가와 모니터링

● Projects

Introduction

2025: ChatGPT 3년차

● HumanEval Benchmark

```
def parse_nested_parens(paren_string: str) -> List[int]:  
    """ 이 함수의 입력은 공백으로 구분된 여러 그룹의 중첩 괄호 문자열입니다.  
    각 그룹 별로 괄호의 최대 깊이를 출력하세요.  
    예: (() ) ((( ))) -> [2, 3]  
    """
```

● GPT-3.5 Turbo: 66%

● GPT-4 Turbo: 73%

● GPT-4o-mini: 88%

Ch.1 Introduction

최근 모델: HUMANEVAL 정도는 정복

○ <https://artificialanalysis.ai/leaderboards/models>

MODEL ↑↓	CREATOR ↑↓	CONTEXT WINDOW ↑↓	ARTIFICIAL ANALYSIS INTELLIGENCE INDEX ↑↓	HUMANEVAL (CODING) ↑↓	MATH-500 (QUANTITATIVE REASONING) ↑↓	AIME 2024 (COMPETITION MATH) ↑↓	MEDIAN ↑↓ <i>Tokens/s</i>	MEDIAN ↑↓ <i>First Chunk (s)</i>
o4-mini (high)	 OpenAI	200k	70	99%	99%	94%	137.2	35.46
Gemini 2.5 Pro		1m	69	99%	98%	87%	147.1	36.61
o3	 OpenAI	128k	67	99%	99%	90%	221.8	14.23
Grok 3 mini Reasoning (high)		1m	67	98%	99%	93%	92.6	0.26
o3-mini (high)	 OpenAI	200k	66		99%	86%	162.6	45.77
o3-mini	 OpenAI	200k	63	97%	97%	77%	160.8	14.50
Qwen3 235B A22B (Reasoning)		128k	62		93%	84%	38.0	0.56
o1 Llama 3.1 Nemotron Ultra 253B Reasoning Flash (Reasoning)	 OpenAI	200k	62	97%	97%	72%	141.6	22.76
		128k	61		95%	75%		
		1m	60		98%	84%	334.6	8.53
DeepSeek R1		128k	60	98%	97%	68%	24.9	3.67
Qwen3 32B (Reasoning)		128k	59		96%	81%	48.9	0.58

LLM 성능: 어디까지?

- 기존 평가 데이터를 뛰어넘는 실력: “더 어려운 문제가 필요하다”
 - 인간과의 프로그래밍 경쟁, 수학 올림피아드 문제 등의 성능 평가 지표 추가
- 특히 코딩의 경우, 개발자의 업무를 굉장히 많은 부분 대체
 - Cursor, OpenAI Codex, Claude Code 등의 급속 성장
- 단순 언어 모델이 아닌, 업무를 제대로 이해하고 실행하는 수준으로 발전

Ch.1 Introduction

핵심 요인 1: Reasoning과 Test-Time Computing

- 모델에게 생각할 시간(과 텍스트 공간)을 주면, 어려운 문제를 잘 풀 수 있다
 - OpenAI가 열고, DeepSeek이 크게 확산시키다

고등학생이 물에 들어가지 않고, 과학적으로 호수의 부피를 구하는 가장 쉬운 방법은 무엇일까?
방법을 3개 정도 생각해 보세요.
답변은 한국어로 작성하세요.

< 2 / 2 >



⊗ Thinking... ^

호수의 부피를 구하려면 물에 들어가지 않고도 몇 가지 과학적인 방법을 사용할 수 있겠다. 고등학생 수준에서 생각해볼 수 있는 방법을 세 가지 정도 떠올려 보자.

첫 번째로는 지도나 위성 이미지를 이용하는 방법이 있을 것 같다. 호수의 형태를 지도에서 확인하고, 호수의 면적을 구한 다음, 깊이를 측정해서 부피를 계산하는 거야. 면적은 구글 어스 같은 프로그램으로 호수의 둘레를 따라 면적을 계산

Ch.1 Introduction

최근 모델: Long Thinking Makes Perfect

● First Chunk Median: Thinking에 걸리는 시간

● Thinking이 항상 필요하지는 않으므로,

Budget 설정 (Gemini 2.5 Flash)

Hybrid 모드 (Qwen 3 Reasoning)

등의 발전 방향 제시

● Overthinking의 단점도 최근 발견

- <https://arxiv.org/pdf/2505.17813>

MODEL ↑↓	CREATOR ↑↓	CONTEXT WINDOW ↑↓	ARTIFICIAL ANALYSIS INTELLIGENCE INDEX ↑↓	MEDIAN Tokens/s ↑↓	MEDIAN First Chunk (s) ↑↓
o4-mini (high)	 OpenAI	200k	70	137.2	35.46
Gemini 2.5 Pro		1m	69	147.1	36.61
o3	 OpenAI	128k	67	221.8	14.23
Grok 3 mini Reasoning (high)		1m	67	92.6	0.26
o3-mini (high)	 OpenAI	200k	66	162.6	45.77
o3-mini	 OpenAI	200k	63	160.8	14.50
Qwen3 235B A22B (Reasoning)	 Alibaba	128k	62	38.0	0.56
o1	 OpenAI	200k	62	141.6	22.76
Llama 3.1 Nemotron Ultra 253B Reasoning	 NVIDIA	128k	61		
Gemini 2.5 Flash (Reasoning)		1m	60	334.6	8.53
DeepSeek R1	 deepseek	128k	60	24.9	3.67
Qwen3 32B (Reasoning)	 Alibaba	128k	59	48.9	0.58

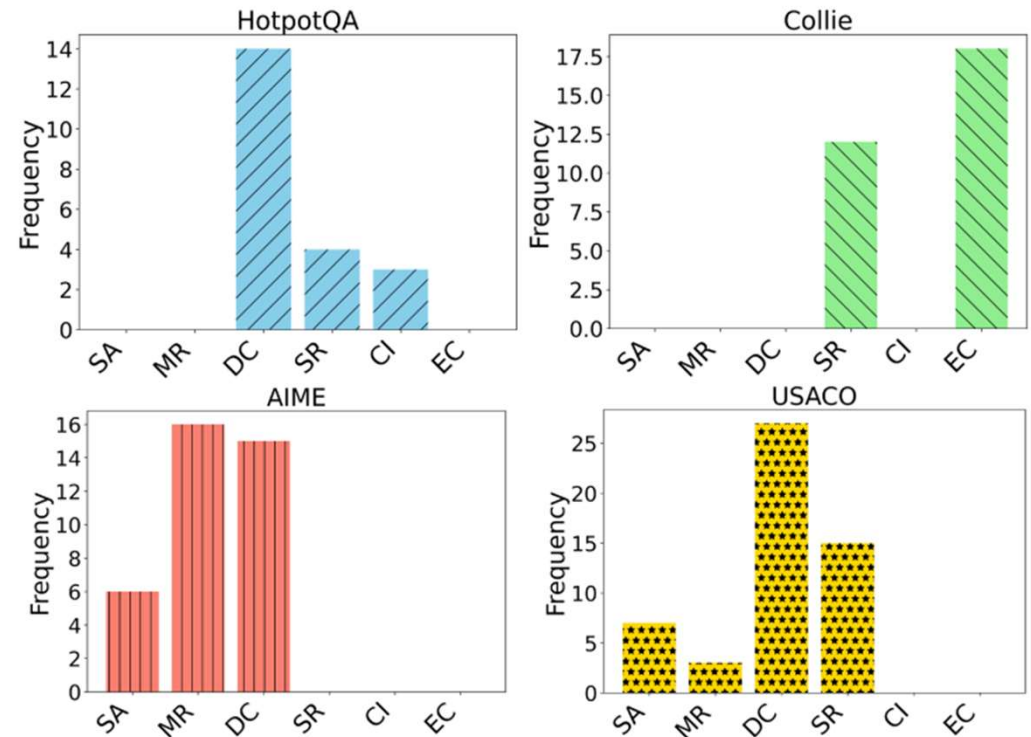
Ch.1 Introduction

Thinking 모델은 어떻게 문제를 푸는가?

● OpenAI-o1의 출력을 6개의 패턴으로 분류

- Systematic Analysis : 전체적 구조 파악
- Method Reuse : 알려진 문제 해결법 적용
- Divide and Conquer : 하위 문제 분할
- Self-Refinement : 자체 평가와 수정
- Context Identification : 추가 문맥 활용
- Emphasizing Constraints : 제약 조건 강조

● 가장 많이 사용된 방법 : D.C and S.R



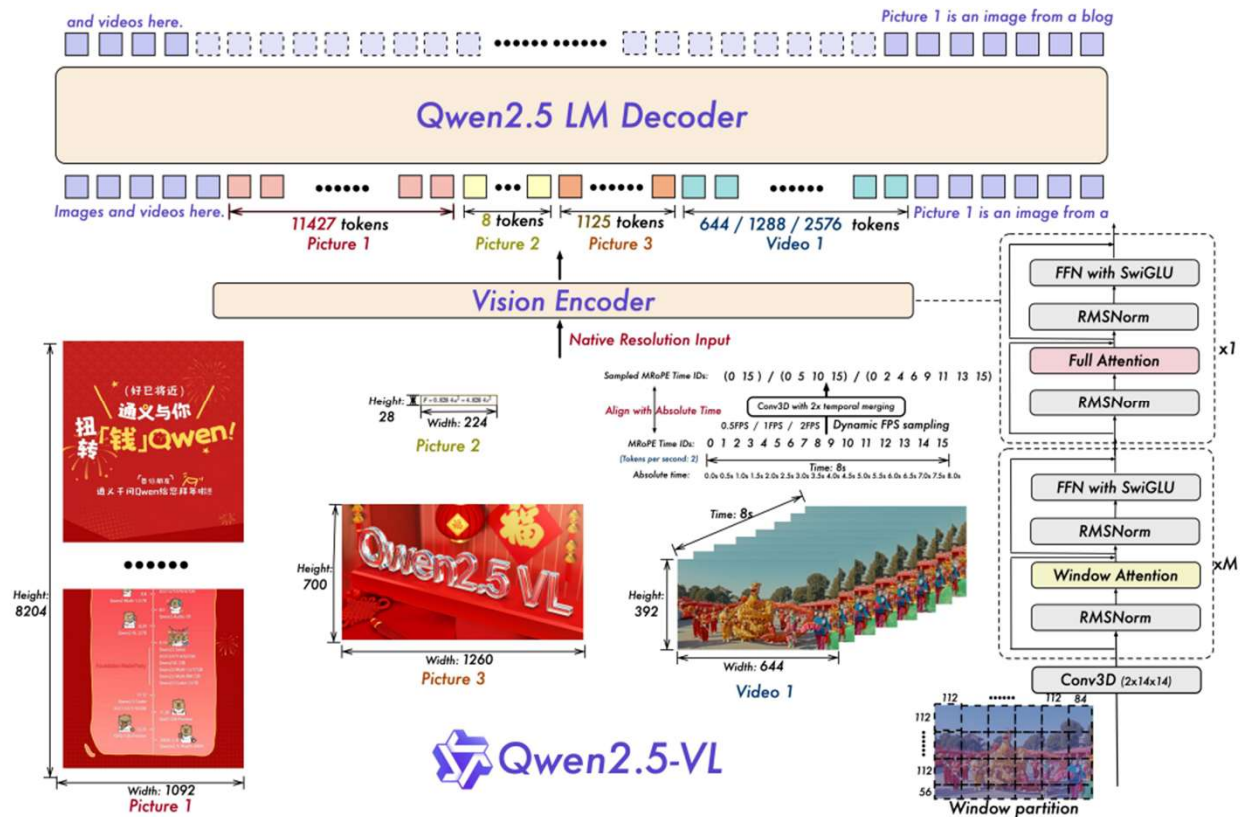
Source: A COMPARATIVE STUDY ON REASONING PATTERNS OF OPENAI'S O1 MODEL <https://arxiv.org/abs/2410.13639>

Ch.1 Introduction

LLM의 발전 방향: 멀티모달 LLM

- LLM 모델에 Vision 능력을 결합하는 방향
(GPT, Gemini, Qwen-VL, Gemma 시리즈)

- 텍스트 이외의 데이터도 생성하는 방향
(GPT-4o의 인기)



Ch.1 Introduction

LLM의 발전 방향: Tool Calling과 Agent

● Agent: LLM에 Tool Calling 능력 탑재하기

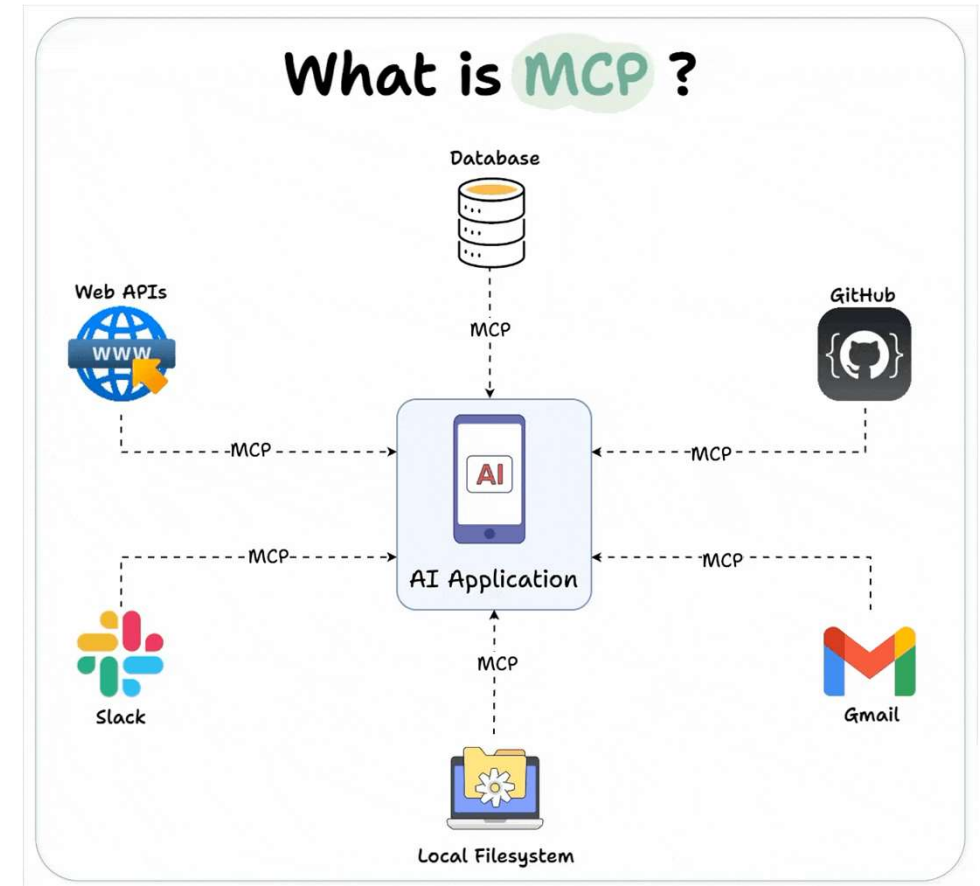
- LLM이 외부 함수나 API를 호출하도록 프롬프트를 통해 전달

● Model Context Protocol: Tool Calling의 표준화?

- Claude, Cursor 등과 연계되어 25년 3월부터 다양한 어플리케이션이 개발되는 중

● Example) Anthropic의 Computer Use Agent

- https://www.youtube.com/watch?v=ODaHJzOyVCQ&ab_channel=Anthropic



Source: <https://medium.com/@halilxibrahim>

Ch.1 Introduction

LLM의 발전 방향: sLLM

● sLLM & Domain-Specific LLMs

- 가지치기(Pruning)과 증류(Distillation) 기반의 방법으로 작은 사이즈의 LLM 성능도 빠르게 성장하는 중
- Gemma 3 1B, Llama 3.2 1B/3B, Qwen 3 0.6B/1.5B, ...
- 엣지 디바이스나 CPU 환경도 고려

- 단독으로는 성능이 매우 부족하지만, 파인 튜닝을 통해 간단한 작업을 시키거나 도메인 특화 모델을 구성 가능

Model	AIME 2024	
	pass@1	cons@64
GPT-4o-0513	9.3	13.4
Claude-3.5-Sonnet-1022	16.0	26.7
OpenAI-o1-mini	63.6	80.0
QwQ-32B-Preview	50.0	60.0
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3
DeepSeek-R1-Distill-Llama-8B	50.4	80.0
DeepSeek-R1-Distill-Llama-70B	70.0	86.7

LLM 어플리케이션 만들기

- 모델의 성능은 크기와 매우 큰 연관이 있다.

- 학습 데이터 양/ Context Window/ 지시 수행 능력 모두에서 큰 차이
- 그러나, LLM 어플리케이션에서는 고성능이 필요하지 않은 작업들도 존재한다.
- 일반적으로 10B 이상의 모델은 어느 정도의 능력을 보유하고 있었으나, 점점 더 작은 크기의 모델들도 활용 가능성이 높아지는 중

- LLM 어플리케이션을 어떻게 구성하냐에 따라 해당 부분을 보완할 수 있다

- 프롬프트 엔지니어링, RAG, 파인 튜닝, Agent 구조

LLM 어플리케이션 활용하기

- LLM은 (텍스트로 구성하기만 하면) 비정형 데이터를 이해하고 처리한다.
 - 데이터 전처리 ← 산업 전반적인 분야에서 효과적 활용
 - LLM에게 넣기 위한 전처리 ← 기업들의 AX(AI Transformation)을 힘들게 하는 대표 요인
- 현업에서는 LLM 단독보다는 실제 데이터를 기반으로 수행
 - Ex) 약관 기반의 Q/A , 데이터 기반의 리포트 작성, 긴 텍스트의 요약
 - 참고할 자료가 주어진다 (~= LLM이 이미 알고 있는 지식이 덜 중요하다)
 - 도메인에 특화되어 있다
 - 어플리케이션만의 평가 방법이 필요할 수 있다

LLM을 위한 AI

변형

LLM 기반 기술 이해하기

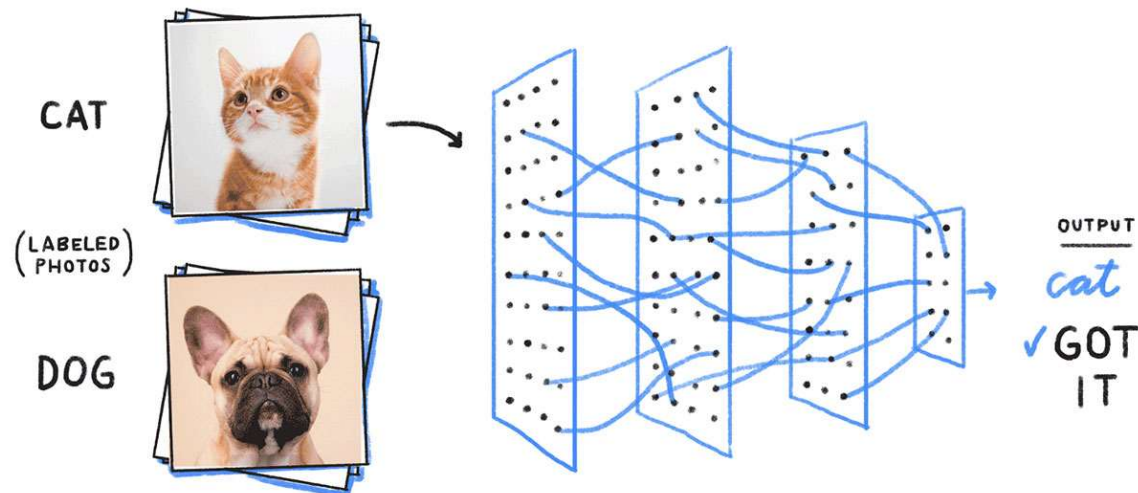
- LLM은 언어 데이터 패턴을 학습하여 언어를 생성하는 기계 학습 모델
- 현대 LLM을 구성하는 핵심 모델 트랜스포머(Transformer)의 등장 배경 이해하기
- 트랜스포머의 파생 모델 GPT의 시작부터, GPT-4o까지의 발전 과정 알아보기

Ch.2 AI for LLM

기계 학습 – Before LLM

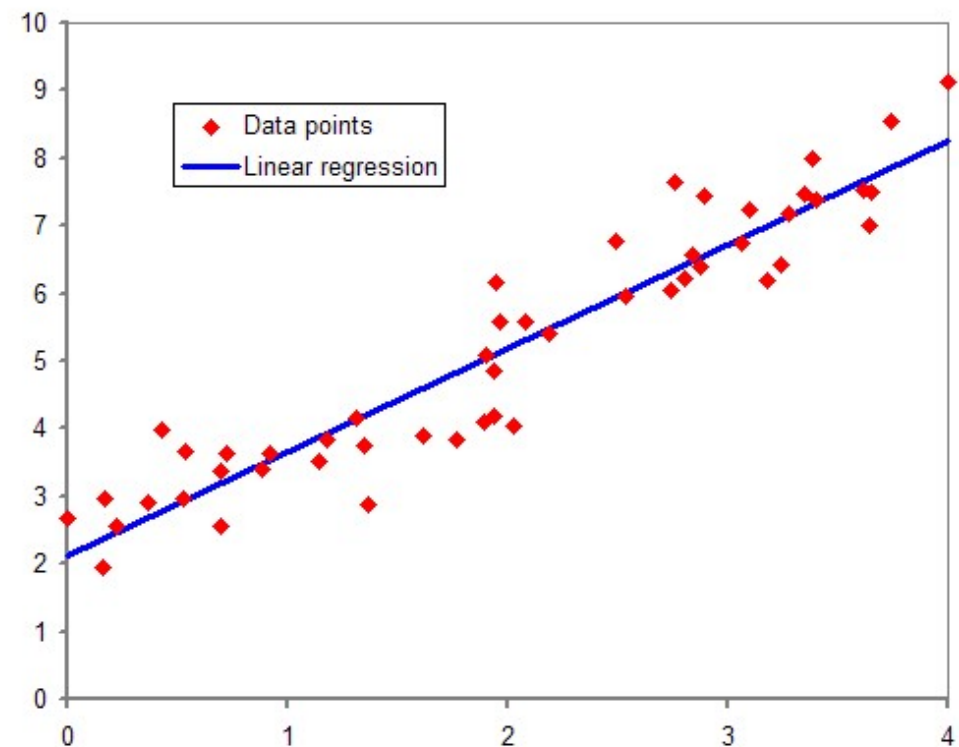
●데이터를 이용해서 데이터의 패턴을 학습하기

- 개/고양이 분류 문제 : 사진을 보고 1(개), 0(고양이)를 출력
- 주가 예측 문제: 한 달 간의 주가 정보와 공시를 보고 오늘 종가 예측하기



LLM 기본 원리: 데이터의 패턴 이해

- 대량의 데이터를 학습하면, 패턴을 이해한다.
- 언어의 패턴을 이해하면, 입력에 대해 유사한 데이터를 생성할 수 있다
 - 다량의 풍부한 데이터를 학습하면 더욱 자연스러워진다
- Data Points: 데이터 (텍스트, 이미지, 오디오, ...)
- Linear Regression: 파라미터가 2개인 모델
 - LLM의 파라미터는 수억~수천억 개



자연어 학습의 어려움

- 수치형 데이터가 아님

- 모델의 입력으로 넣을 수 없음
- 데이터를 모델에 넣으려면 숫자로 변환해야 함

- 길이가 정해져 있지 않음

- Feature의 개수를 조절하기 어려움

- 순서가 중요함

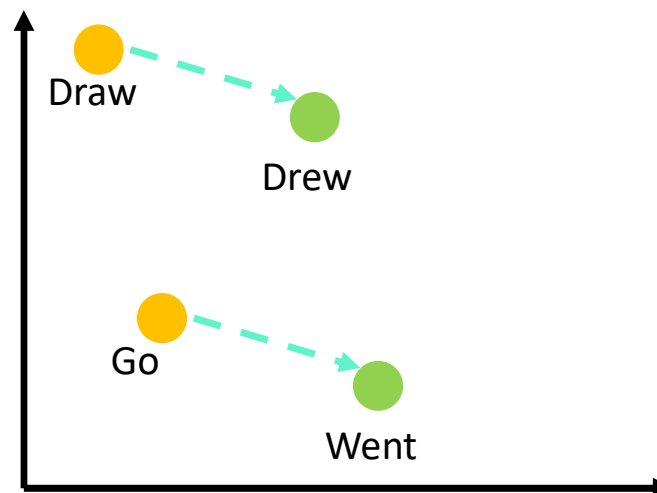
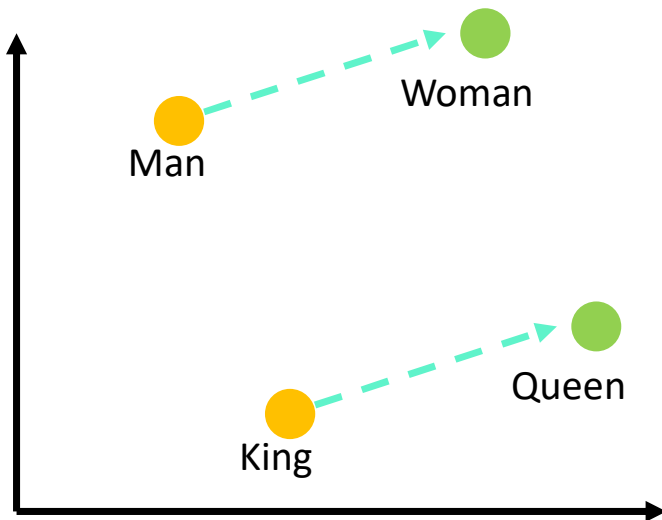
- A likes B v.s B likes A

임베딩: 추상적 데이터 컴퓨터에게 이해시키기

● Word Embedding : 단어의 의미를 Dense 벡터로 변환 ($\leftrightarrow$ 인코딩: (0,1,0))

- 컴퓨터가 연산할 수 있는 형태로 전달

- 대표적 임베딩 방법: Word2Vec
- Ex) <https://word2vec.kr/search/>



Seoul - Korea = X
 Rome - Italy $\sim$ X
 Germany + X = Berlin?

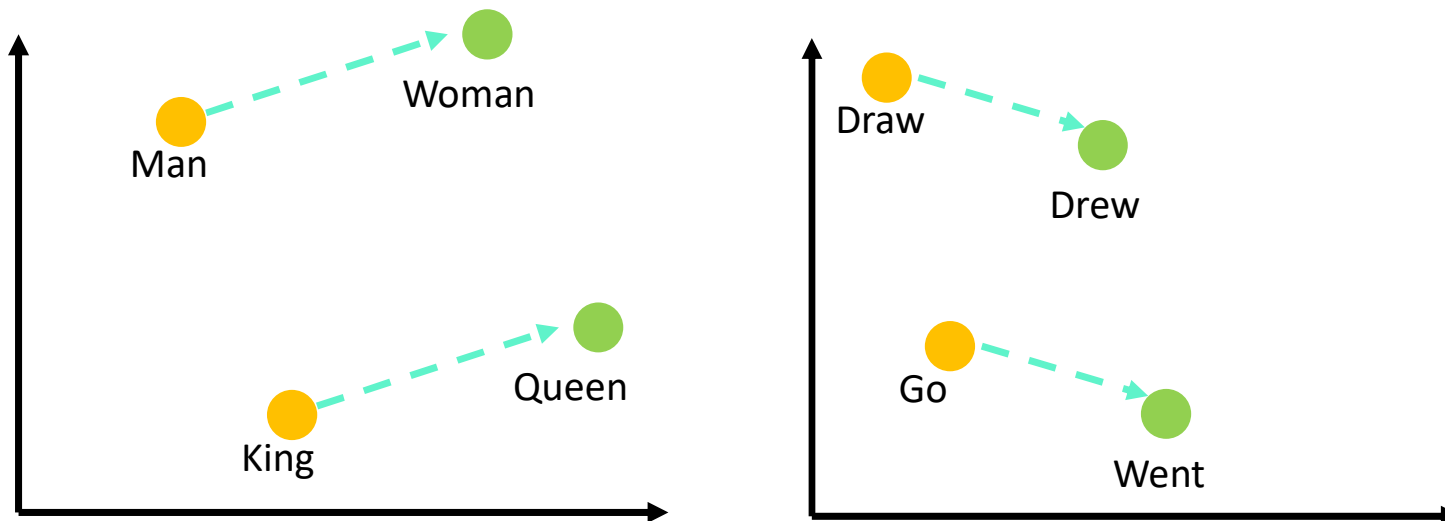
Word Embedding의 한계점

●동음이의어 처리의 어려움

- Ex) King Sejong / Diamond King / Martin Luther King / King Bed / ...

●오타 및 새로운 단어 처리 불가

- 모르는 단어는 <UNK>와 같은 카테고리로 처리



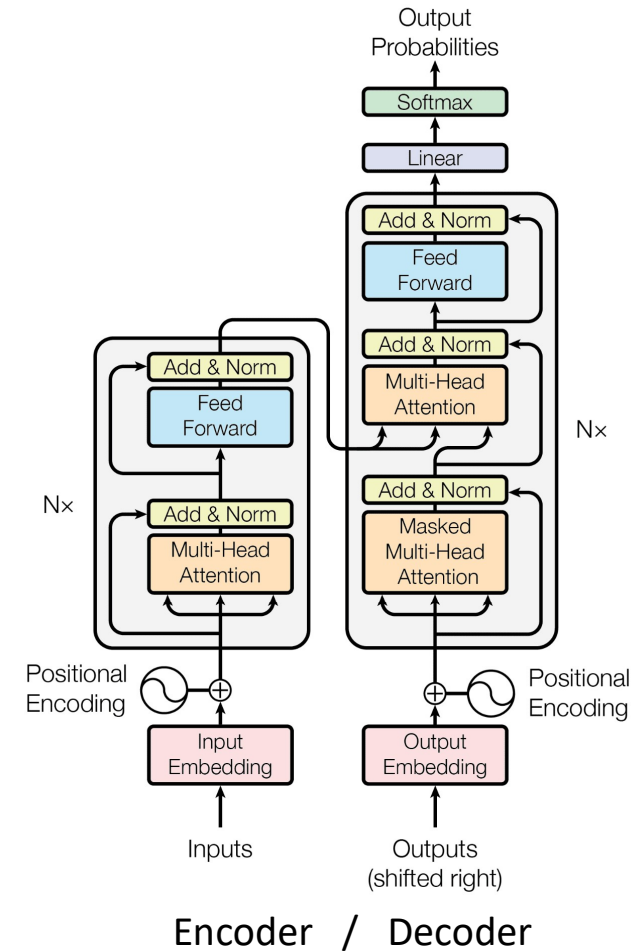
맥락(Context)

- “I go to skool” 이라는 문장에서
go의 의미는 ‘가다’에 해당할 것이고,
skool은 ‘school’의 오타로 생각할 수 있다
- 어떻게 알 수 있는가?
 - Skool은 school과 철자 구성이 비슷하다
 - 주변 단어가 “I, to, School” 이기 때문에 ‘가다’를 유추할 수 있다
- 텍스트의 의미는 단어의 세부 구성과 주변 맥락을 파악해야 알 수 있다
 - Token (토큰)과 Context (맥락) 기반의 자연어 처리 방법 도입
 - 토큰 레벨의 임베딩 + 맥락 이해 모델

트랜스포머: LLM 시대의 일등 공신

● 2017년 구글에서 논문 공개

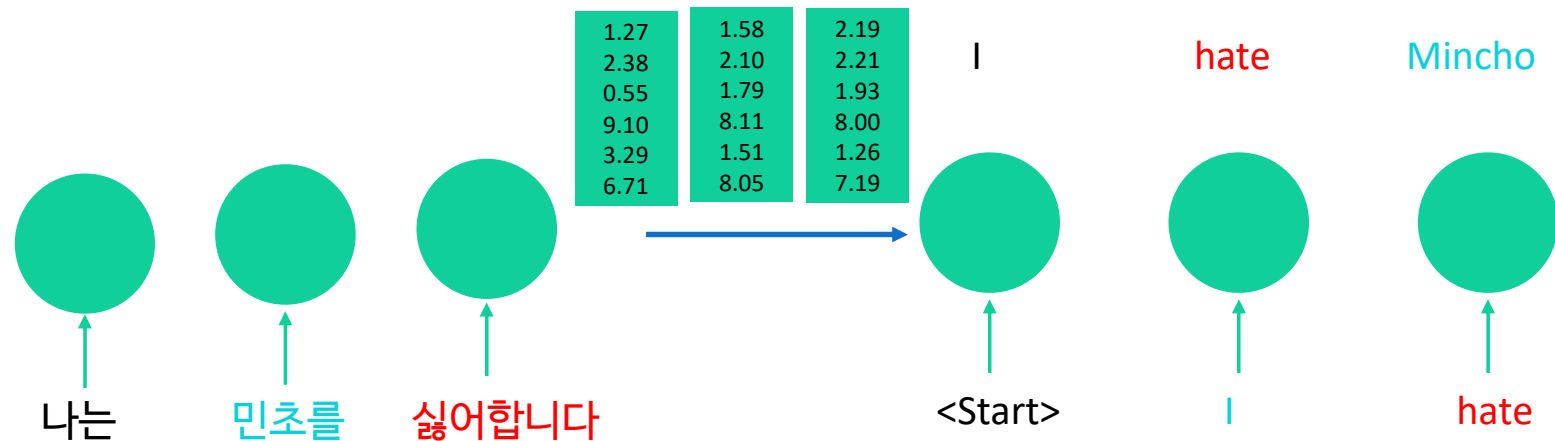
- 문장 - 문장 변환 문제에서 높은 성능을 보임
 - 대표적인 문제: 번역 (영어 → 불어)
- 인코더 - 디코더 구조 채택
 - 인코더 : 입력 문장의 의미를 담은 벡터 생성
 - 디코더 : 인코더 결과를 받아 목표 문장 **순차적으로** 생성



Source: <https://arxiv.org/abs/1706.03762>

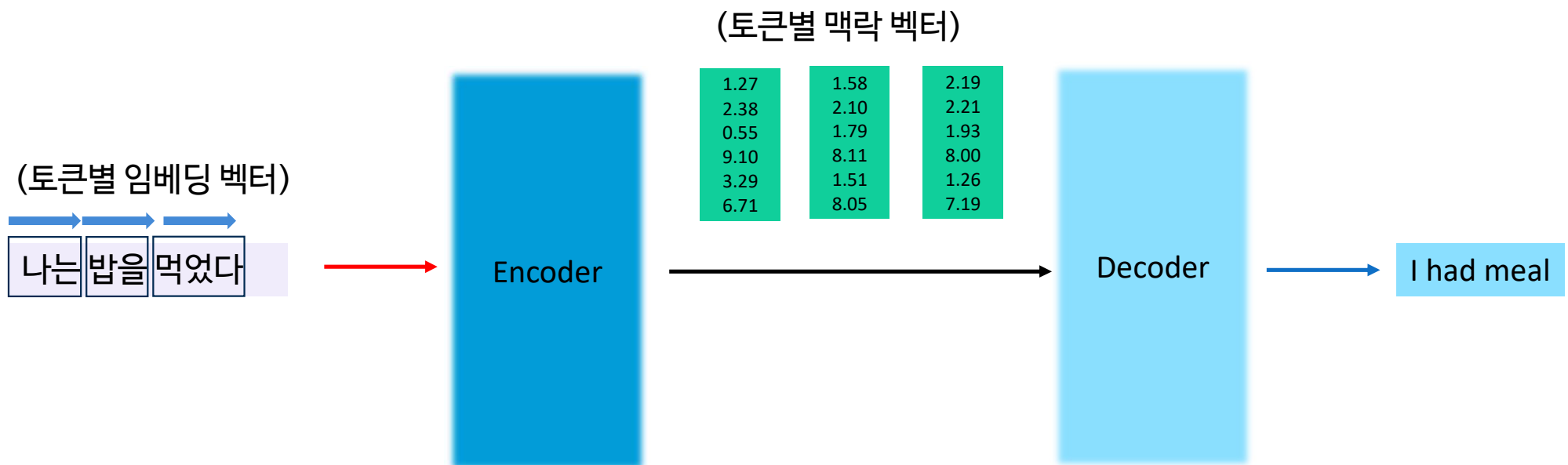
‘Attention Is All You Need’ (2017)

- “어텐션만으로 훌륭하게 정보를 추출할 수 있다”
 - (Attention + RNN) - RNN
 - 순서 없이 단어가 각자 입력되는 방식 (병렬 처리 가능)
 - 나는 민초를 싫어합니다 → (나는, 민초를, 싫어합니다)



Transformer : 순환 없는 인코더-디코더 구조

- 인코더: 토큰 개수만큼의 벡터가 **병렬적으로** 생성
- 디코더: 목표 문장이 **순차적으로** 생성



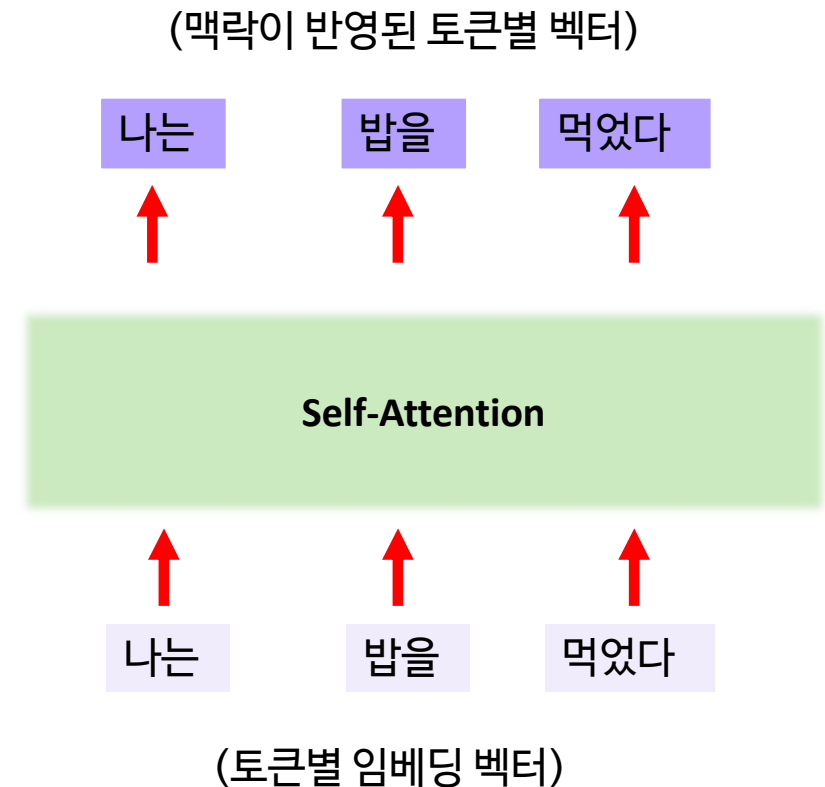
Self-Attention : 다른 단어와의 관계로 자신 표현하기

● 다른 표현: ‘이 문장에서 이 단어는 어떤 의미인가?’

- “나는” = “나는 밥을 먹었다” 에서의 “나는”
- “밥을” = “나는 밥을 먹었다” 에서의 “밥을”
- “먹었다” = “나는 밥을 먹었다” 에서의 “먹었다”

● 각 토큰을 Query, Key, Value 등의 값으로 변환하여 N*N 번 계산

- LLM 계산량/캐시 증가의 주요 단계로
- 최근 모델에서는 해당 부분의 계산량을 줄임



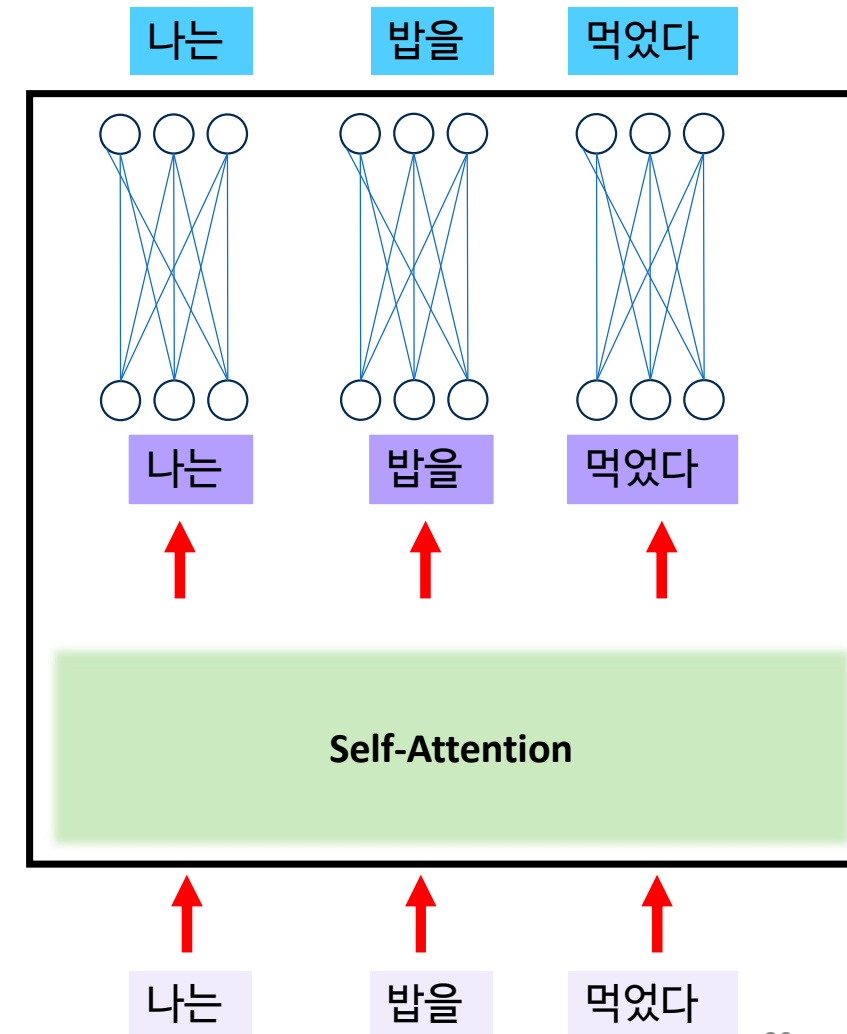
Encoder

● FFNN : Feed-Forward Neural Network

- 작은 딥 러닝 네트워크
- 비선형성을 추가하여 깊은 관계를 학습

● Encoder: Self-Attention + FFNN

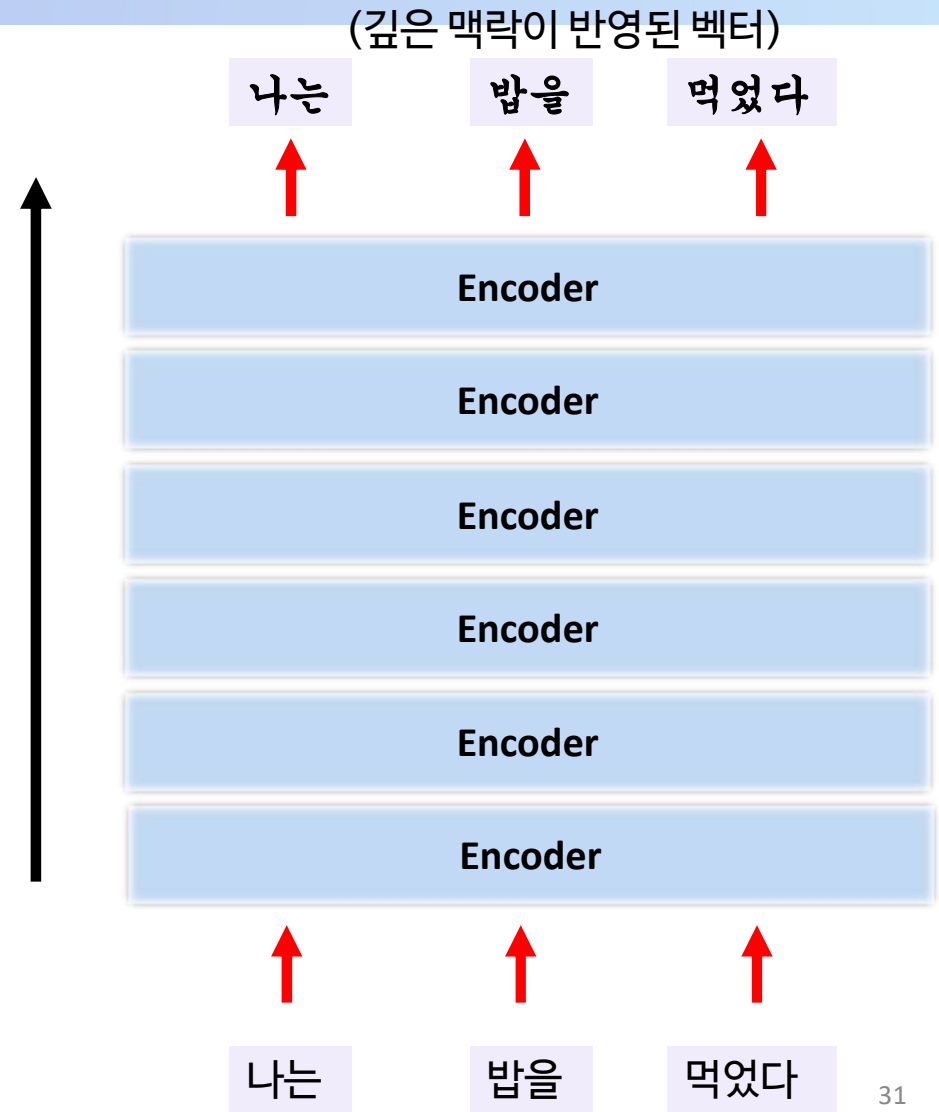
- 단어와 문맥을 이해하여 각 단어를 병렬적으로 전달
- RNN과 다르게 Long Context의 정보를 잃지 않는 장점



Ch.2 AI for LLM

Encoder 6연속 쌓기 : Encoding

- 직전 인코더의 출력을 다음 인코더의 입력으로 전달
 - 파라미터의 수를 늘리고, 복잡한 의미를 이해
- 최종 출력: 입력 문장의 토큰별 의미 벡터



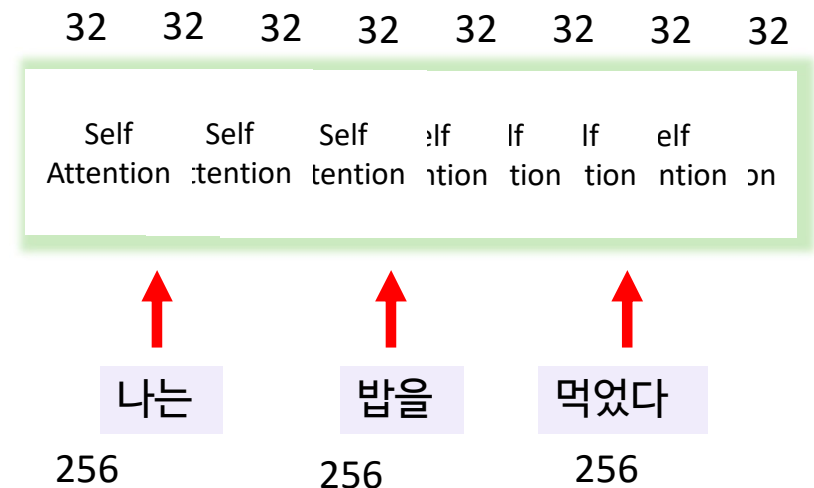
Multi-Head Attention

● 맥락을 계산하는 Self Attention 층은 여러 개의 연산으로 구성

- N가지의 맥락을 고려하고, 이를 다시 합쳐 차원을 복원
- Ex) 256 차원 임베딩 $\rightarrow 32 * 8 \rightarrow 256$

● “N명의 전문가가 읽고 판단”

- 학습 후에 보니 실제로 각각 다른 역할 수행
 - 문법, 문맥 유사성, 언어적 특징, ...



Ch.2 AI for LLM

Positional Encoding

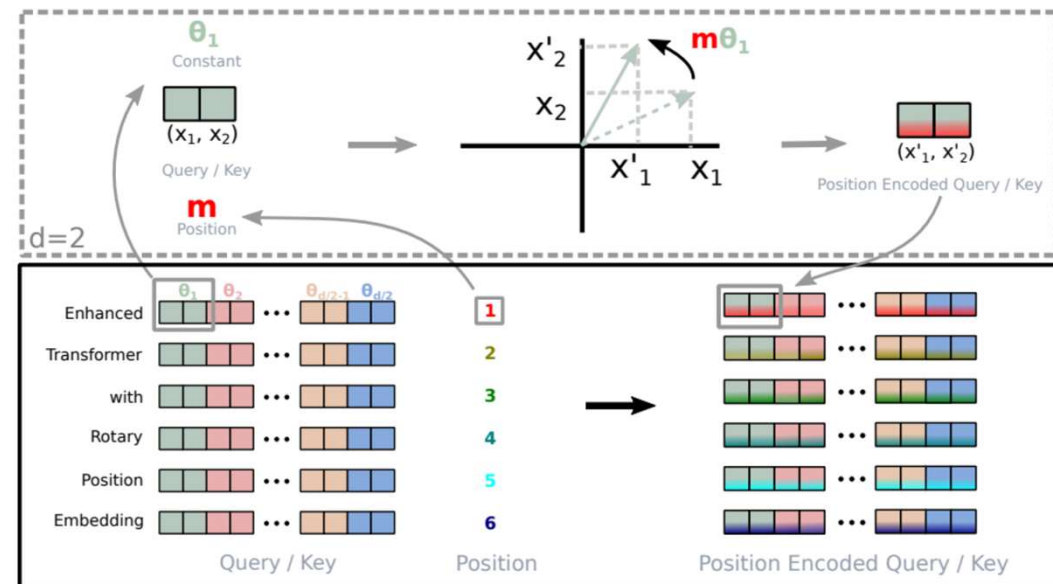
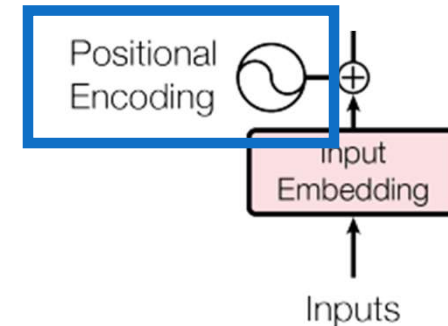
● 토큰 임베딩만으로는 Self Attention에서 순서를 반영하기 어려움

- Ex) 나는 학교에 가고, 강아지는 산책을 간다.
나는 산책을 가고, 강아지는 학교에 간다.

● 위치 인코딩 벡터를 통해 토큰의 위치 정보 반영

- 단어 임베딩 + 위치 인코딩의 결과 활용
- Ex) '위' + '1번 단어 인코딩'
'위' + '3번 단어 인코딩'

● 최근에는 RoPE라는 회전 기반 방법 사용



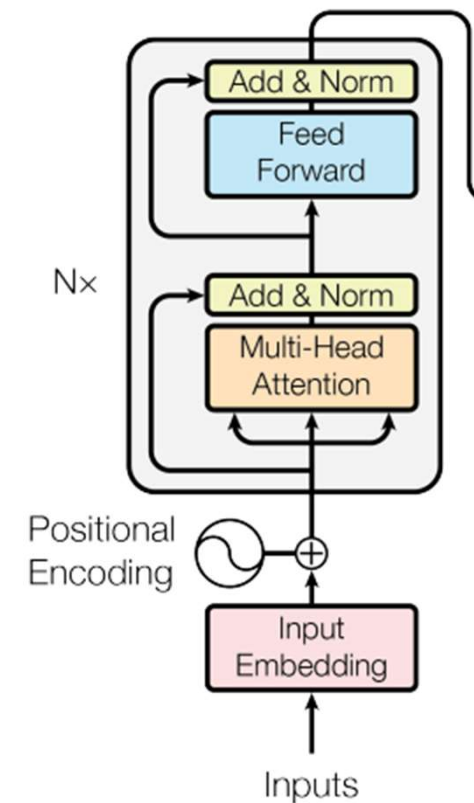
Source: RoFormer: Enhanced Transformer with Rotary Position Embedding
<https://arxiv.org/abs/2104.09864>

Encoder 구조

● Input : 입력 문장 단어(토큰) 목록

- Ex) 나는 밥을 먹었다 → 나는, 밥을, 먹었다

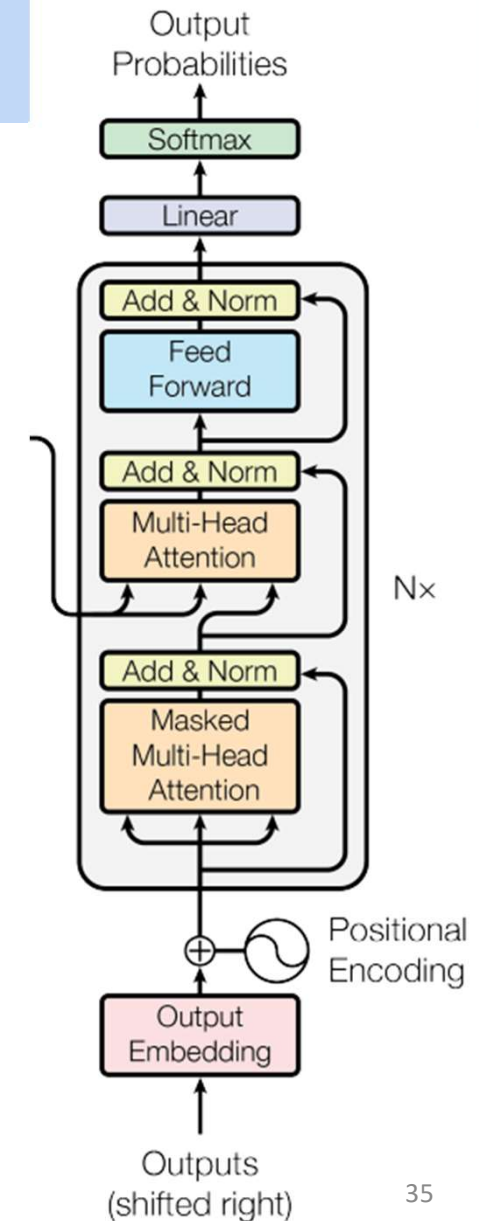
● Output: 입력 문장의 단어(토큰)별 의미 벡터



Ch.2 AI for LLM

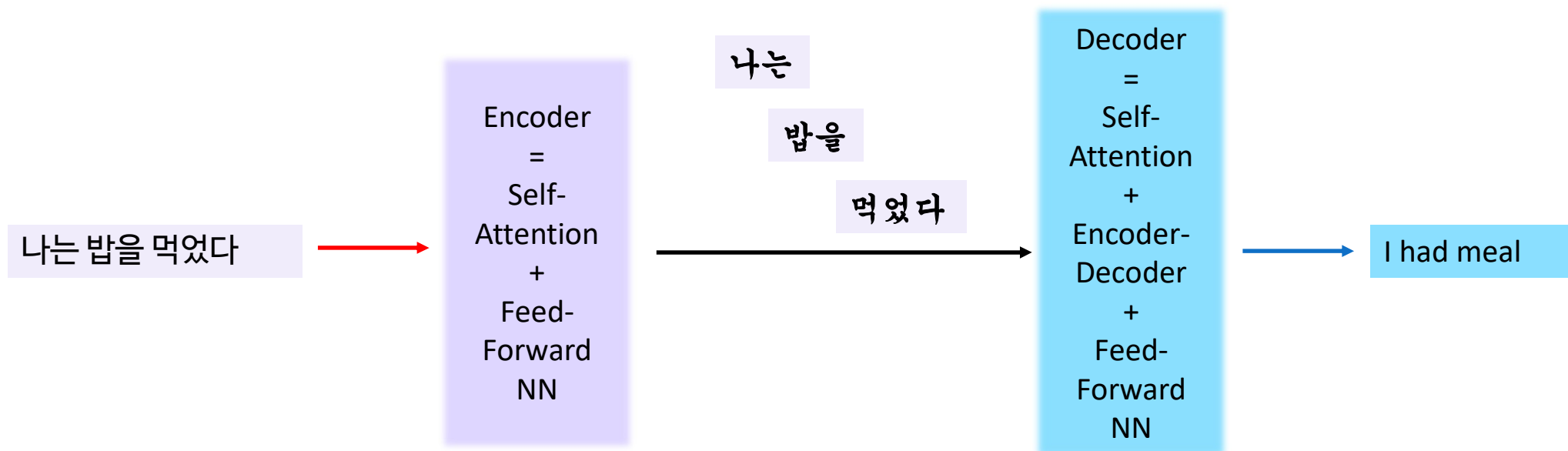
Decoder 구조

- 목적: <BOS> I had meal 출력
- Input : 인코더 결과 + 출력 문장 앞부분
 - Ex) [나는, 밥을, 먹었다] + <BOS> I had
- Output: 그 다음 단어(토큰)의 확률 분포
 - Ex) meal: 70%, to: 18%, been: 5%, ...
 - 샘플링으로 다음 단어 선택
 - 생성된 단어는 입력에 추가되어 다음 단어 생성에 활용
 - 자기 회귀적(Autoregressive) 출력



Decoder 구조

- 인코더 구조와 유사하나, 인코더 입력의 영향을 받는
Encoder-Decoder Attention Layer 존재



Ch.2 AI for LLM

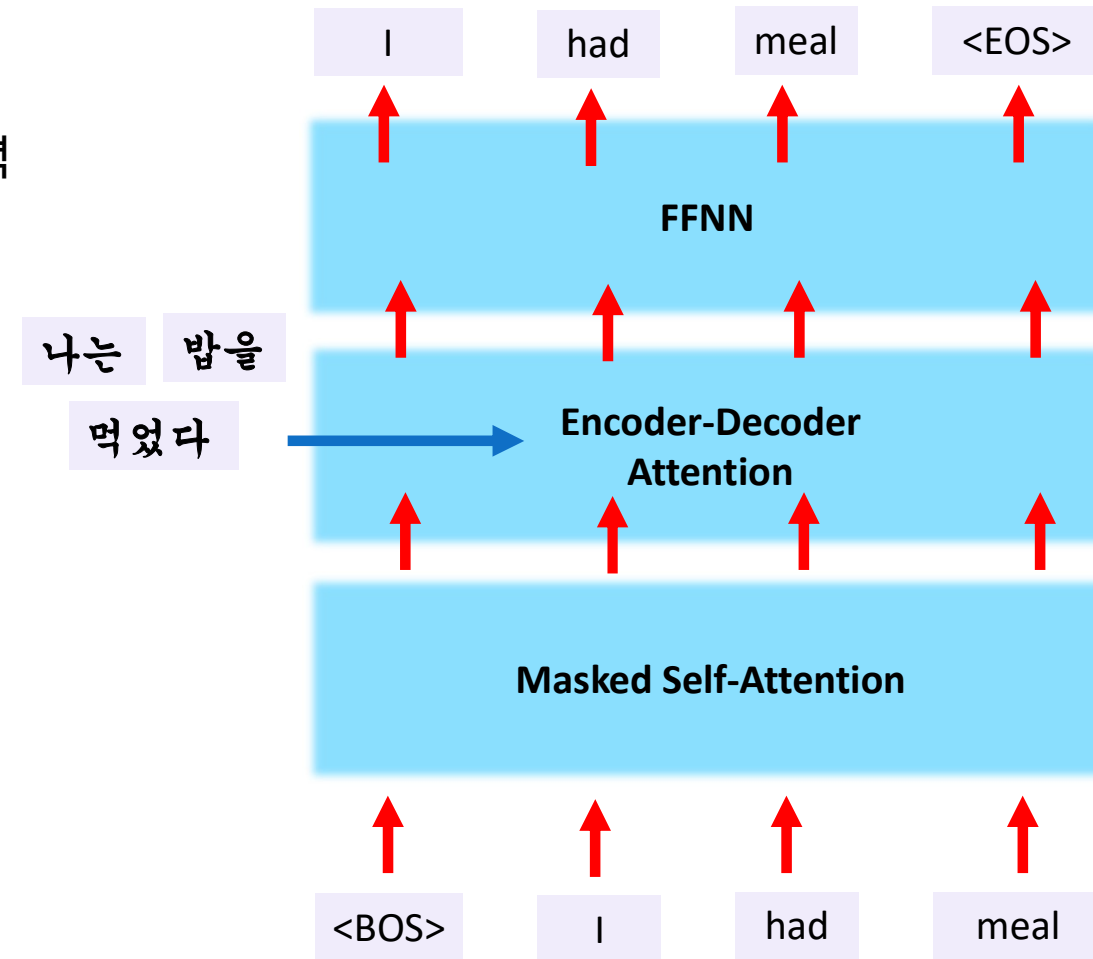
Decoding : 순차적 출력

● 디코딩 과정은 직전 출력의 영향을 받음 → 순차적 출력

- <BOS> → I
- <BOS> I → had
- <BOS> I had → meal
- <BOS> I had meal → <EOS>

● Encoder-Decoder Attention

- 원본 문장의 벡터를 받아서 출력 방향을 결정



Ch.2 AI for LLM

Decoding: Masking

- 단, Decoding 과정에서는 단어의 직전 출력들에 대해서만

Self Attention을 계산함

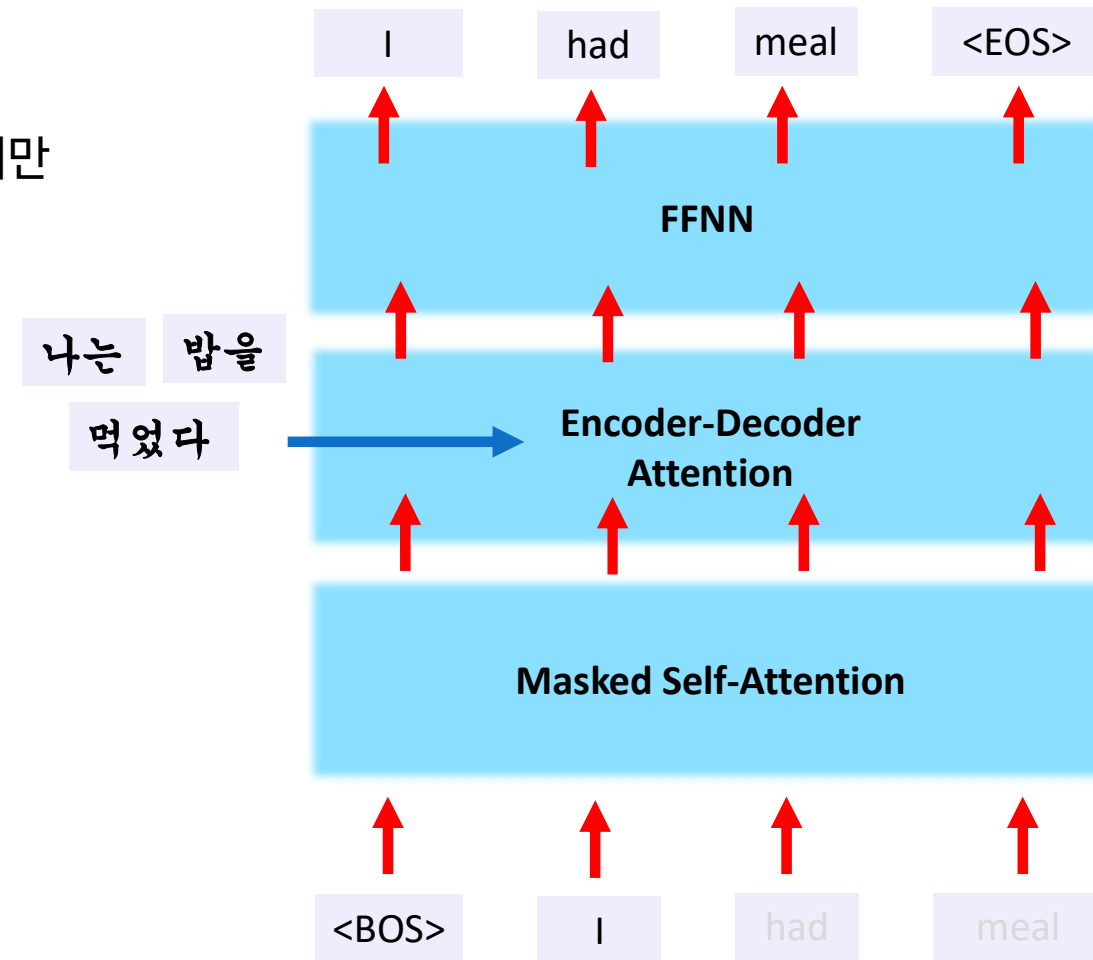
- Ex) I 에 대해 계산할 때

추론 과정에서 had, meal의 영향을 받지 않았으므로

<BOS>, I 만 고려하여 업데이트

- 인코더 Self Attention과의 결정적 차이

- 양방향 맥락 vs 단방향 맥락



트랜스포머 성공의 영향

● 트랜스포머는 번역과 같은 문장-문장 변환 문제에서 매우 뛰어난 성능을 보임

- 인코더 : 입력 문장의 의미를 효과적으로 파악
- 디코더 : 출력 문장을 자연스럽게 생성

● 이후에, 2개의 자연어 처리 모델로 분화

- BERT (2018) : 인코더 구조를 활용하여 문장 의미 파악
 - 분류 문제 등을 푸는 NLU(Understanding) 모델
 - 이후 텍스트 전체의 의미를 계산하는 임베딩 모델로 발전
- GPT (2018) : 디코더 구조를 활용하여 텍스트 생성
 - 다음 단어를 예측하여 문장을 작성하는 NLG(Generation) 모델, 그 이후는...

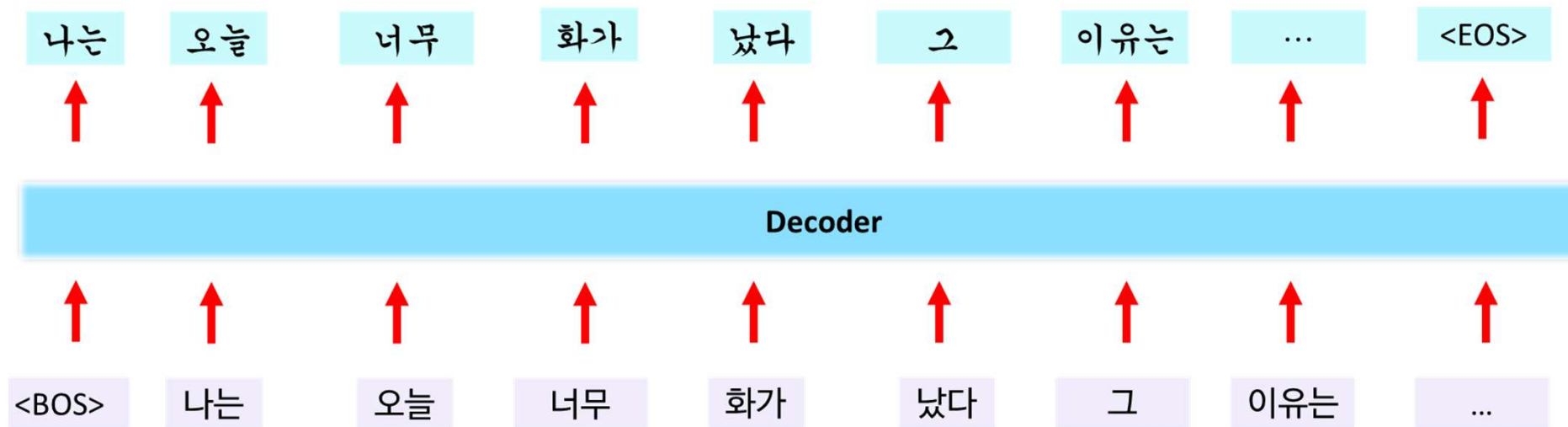
트랜스포머 기반의 “사전 학습” 모델 : BERT와 GPT

- (Recap) 개/고양이 분류 문제는 “이미지 분류”라는 단일 Task만을 목적으로 학습
 - 리뷰 분류 문제도 비슷한 방식
- BERT와 GPT는 특정 Task가 아닌, 언어의 패턴 자체를 학습하고자 함
 - 언어의 패턴을 학습하자,
 - 약간의 추가 학습만 거쳐도 단일 Task 모델처럼 뛰어난 성능을 보임
- 사전 학습(Pretraining): Task에 맞춰 학습하기 전, 베이스가 되는 지식을 학습하는 방법
 - 대규모 사전 학습 기반의 “파운데이션 모델”

GPT: Next Token Prediction

● “문맥을 고려하여” 주어진 문장의 다음 단어 예측하기

– Generative Pre-trained Transformer (2018)



GPT부터 GPT-3까지

● GPT(GPT-1) : 0.1B

- 특정 도메인에 적용하기 위해 후처리(Fine-tuning) 필요
- Ex) 리뷰 데이터+ 레이블 추가 학습 후 긍정/부정 분류

● GPT-2 : 1.5B

- 일반 도메인에 범용적으로 적용하려는 시도
- Ex) 간단한 요약, 번역 등의 작업을 언어로 전달

● GPT-3 : 175B

- 대량의 데이터를 바탕으로, 높은 수준의 지식과 언어 능력
- In-Context Learning과 Few-Shot Learning 능력 발견
- 질의응답 능력은 부족

GPT-3.5부터 GPT-4까지

● GPT-3.5 : 175B

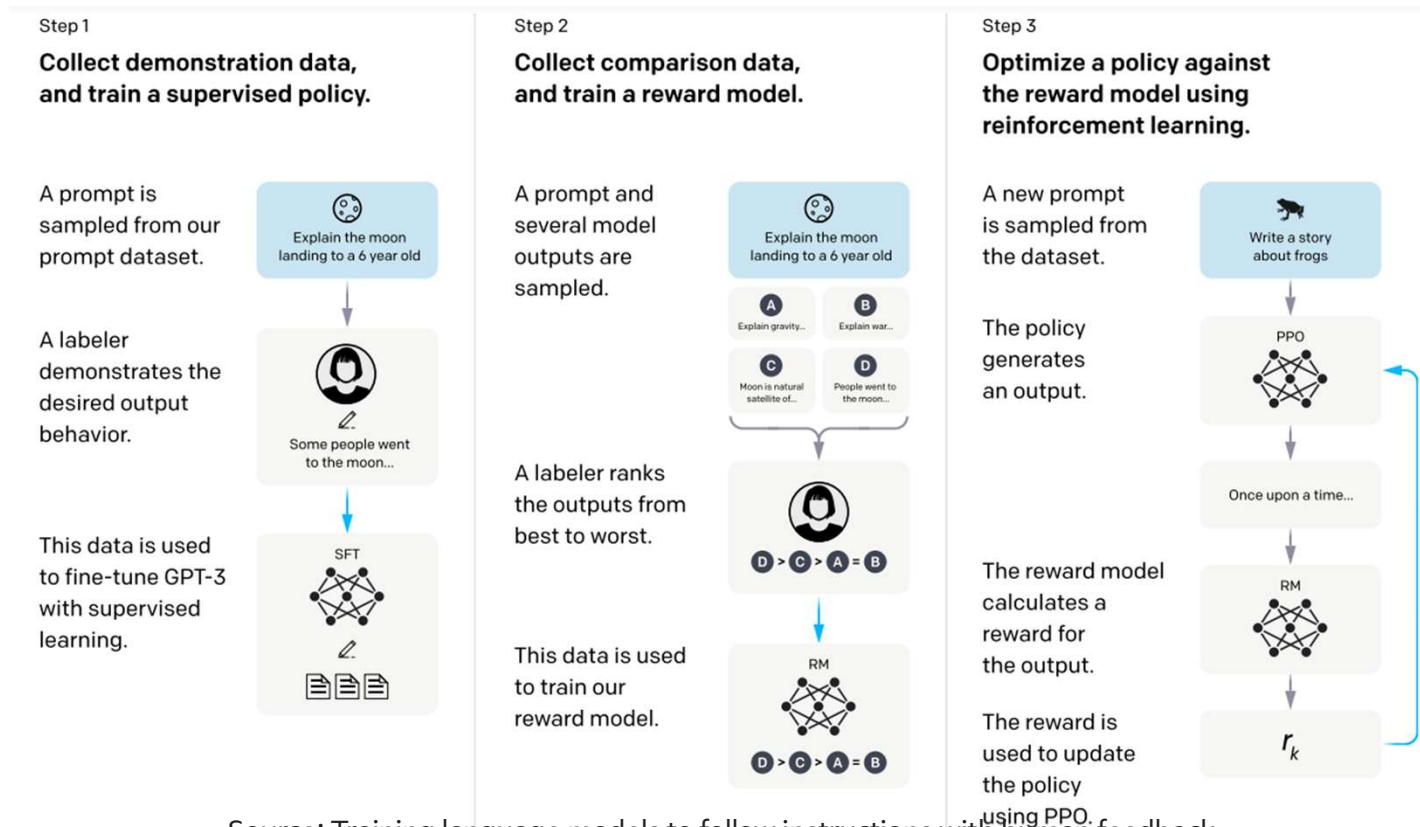
- ChatGPT 모델
- 질의 응답 포맷 학습 + 강화 학습을 이용해 인간의 선호도 반영

● GPT-4 : 1760B (110B * 16 추정)

- 16개의 모델 혼합: Mixture of Experts 기법 사용
- 이미지 데이터 인식
 - Vision Encoder를 이용하여 이미지 인코딩 생성 후 결합
- 웹 검색, Dall-E 3, Code Interpreter 등의 기능 추가

ChatGPT 성능의 핵심: Instruction 기반의 학습

● Supervised Fine Tuning (SFT) + Reinforcement Learning with Human Feedback (RLHF)



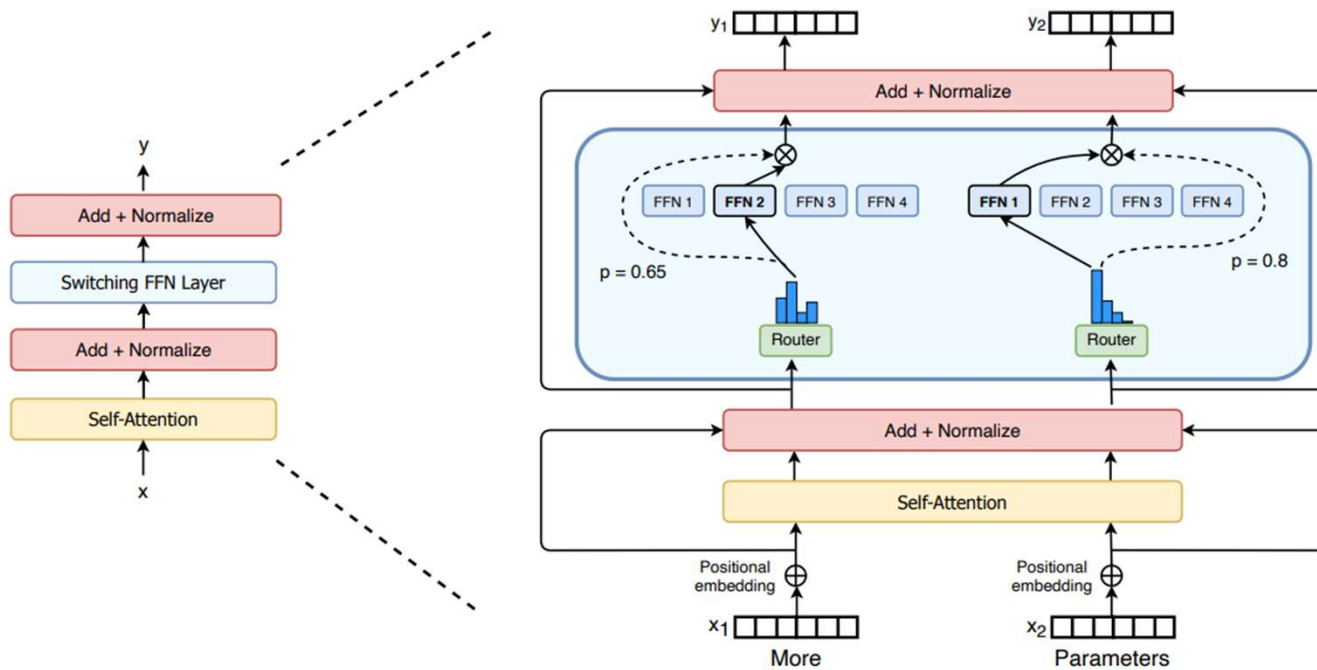
Source: Training language models to follow instructions with human feedback.

<https://arxiv.org/abs/2203.02155>

Ch.2 AI for LLM

GPT 3.5 to GPT 4: Mixture of Experts

- MoE 구조: 작은 모델 여러 개의 집합 + 이들 중 누가 대답할지를 정하는 라우터 네트워크
 - 이후 DeepSeekV2 등에서 개선된 구조로 계속 활용



Source: Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,
<https://arxiv.org/abs/2101.03961>

GPT-4o

● GPT to GPT-4o : Early Fusion

- GPT-4o: Omnimodal (옴니모달) 모델
- 텍스트/이미지/오디오/비디오를 멀티미디어 토큰으로 변환 → 초기에 융합하여 학습
- Any-to-Any 모델 형태: 현재는 텍스트/오디오/이미지 생성이 개별 공개

● 토큰나이저 개선

- 한국어 등의 다국어 데이터 처리 속도 대폭 증가
- <https://openai.com/index/hello-gpt-4o/>

Ch.2 AI for LLM

OpenAI의 추론 전문 모델

● 25.2. OpenAI-O3-mini 출시

- 200K 토큰 입력, 100K 토큰 출력 가능

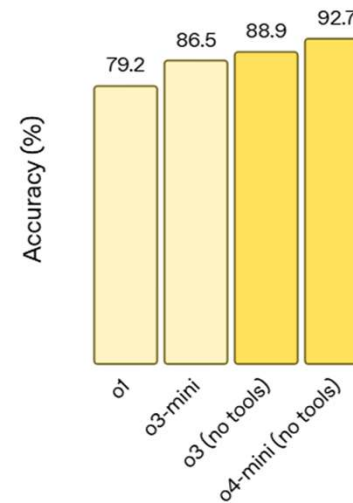
● 25.4. OpenAI O3, O4-mini 출시

- 매우 뛰어난 추론 및 도구 사용 능력
- <https://openai.com/index/introducing-o3-and-o4-mini/>

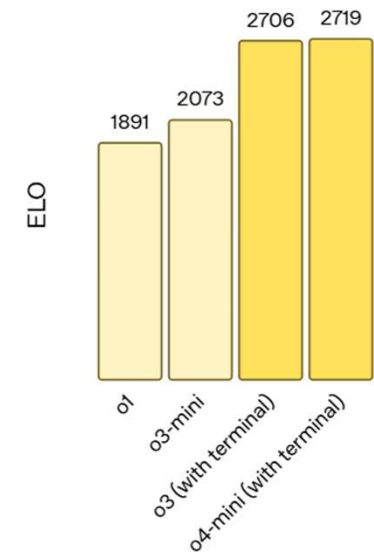
● 추론 강도를 지정할 수 있음

- Low-medium-high

AIME 2025
Competition Math



Codeforces
Competition Code



FAQ) 왜 지금은 디코더 기반의 모델이 분류도 잘 하나요?

- 인코더는 양방향 맥락을 참고하고, 디코더는 단방향 맥락을 참고
 - 이론적으로는 인코더 모델의 문장 이해력이 더 뛰어남
- 그렇지만, 어느 정도의 파라미터가 주어지고, 데이터를 충분히 학습한 디코더 모델도 높은 수준의 이해력을 갖게 되므로, 최근의 디코더 기반 모델은 다양한 언어 Task에 모두 활용 가능
- 인코더 모델은 디코더 모델보다 활용 범위가 낮으므로, 크게 만들 필요가 적음
 - 인코더 모델의 발전 방향인 임베딩 모델에서도, LLM을 활용하려는 시도 중

Prompt Engineering

프롬프트 엔지니어링 (2023 ~ 2025?)



SBS Biz

<https://biz.sbs.co.kr> > article

AI 덕분에 뜬 연봉 '4억4천만원'인 '이 직업'...나도 해볼까?

2023. 3. 30. — ... 프롬프트 엔지니어'가 주목을 받고 있습니다. 블룸버그통신은 현지시각 29일 프롬프트 엔지니어(prompt engineers)가 연봉 33만5천 달러, 우리 돈 4억 ...



AI타임즈

<https://www.aitimes.com> > news > articleView

"몇년 뒤 프롬프트 엔지니어링은 사라질 것" < 말

2024. 5. 12. — "2026년이나 2027년에 프롬프트 엔지니어라는 직업이 남아 있을까. 실제로 여기서 이야기를 주고 받는 우리는 프롬프트 기술을 사용하지 않는다.



WSJ

<https://www.wsj.com> > articles

2023년 가장 인기 있는 AI 직업은 이미 구식이다

2025년 4월 25일 — 대규모 언어 모델에 보낼 완벽한 입력을 만드는 것을 목표로 하는 프롬프트 엔지니어링이 가장 인기 있는 직업 중 하나가 될 것으로 예상되었습니다.

프롬프트 엔지니어링은 2.5년만에 어떻게 바뀌었는가?

● 모델 자체의 발전

- 1) 최근의 고성능 LLM 모델들은 프롬프트의 품질에 크게 영향받지 않음
→ 스스로 Reasoning하는 과정을 통해 좋은 답변을 생성
- 2) 모델 자체가 훌륭한 프롬프팅 능력을 가져,
메타 프롬프팅(Meta Prompting) 성능이 크게 향상

● 직무에서 역량으로

- 특정 직군에 맡기기보다는, 내부 교육을 통해 모든 구성원에게 역량을 기르는 쪽으로 변화

● 하지만, 아직도 고성능의 프롬프트를 최적화하고 구성하는 것은 어려움

- 모델 자체가 빠르게 변화하는 점에서 오는 경험과 노하우의 영역

프롬프트 엔지니어링이란?

- 자연어를 입력받는 언어 모델에 대해,
원하는 결과를 얻기 위한 프롬프트를 작성하는 기술
- 같은 목적의 요청이라도,
효과적인 입력을 설계하고 최적화하는 과정이 필요
- LLM의 작동 원리를 고려하여,
좋은 프롬프트의 특징을 이해하고 활용하는 방법

Google Gemini Prompting Guide 101 (2024)

- “업무용 최적의 프롬프트는 평균 21개 단어로 구성되었으나, 일반 이용자들의 평균 프롬프트 길이는 9단어였다.”
- 효과적인 프롬프트를 위해서 4개의 요소를 포함
 - Persona : 구체적인 역할 설정
 - Task : 명확하고 구체적인 작업 제시
 - Context : 참고할 만한 관련 정보를 포함
 - Format : 원하는 출력 형식 명시

Ch.3 Prompt Engineering

Example) 4개의 요소를 포함한 프롬프트 (최소 75단어)

- 당신은 기업의 COO입니다. 분기 미팅을 주최하고 있습니다.
- 직원들의 어려운 질문에 대응할 수 있는 전략을 브레인스토밍하고, 답변을 작성하세요.
- 회사의 주요 이슈는 다음과 같습니다.
 - 연말 투자자 리포트: 개발 실적 정리 관련 (자세한 내용 첨부)
 - CTO 퇴사: 기술 리더십 변화로 인한 우려 (자세한 내용 첨부)
 - 초기 계획 대비 회사 비전 변화에 대한 장기적 목표 재설정 (자세한 내용 첨부)
- 각 주제에 대해, 직원들이 질문할 가능성이 높은 내용을 2~3개씩 작성하세요.
- 각 질문과 함께 자신감 있고 단호한 어조의 답변을 작성해주세요.

Prompt Chaining: 복잡한 작업은 나눠서 처리하기

- LLM은 이전 맥락을 기억하므로, 이를 활용하자
- 이전 프롬프트에서 수정)
 - 다음 프롬프트에 회사의 주요 이슈를 전달하겠습니다.
 - 각 주제에 대해, 질문과 답변을 작성해 주세요.
 - (1번 안건:연말 투자자 리포트 입력 → 답변 생성)
 - (2번 안건: CTO 퇴사 입력 → 답변 생성)
 - ...

Ch.3 Prompt Engineering

프롬프트 구조화하기

● LLM이 프롬프트의 어느 부분에 집중해야 하는지 강조

- ---, [], <> 등을 이용하여 각 요소를 명확하게 분리/명시
- Ex) 아래의 [Context]를 활용하여 [Question]에 답하세요.

[Context]:

[Question]:

● 마크다운 문법을 통한 강조 (#, **)

- Ex) GPT를 통해 생성한 프롬프트

고객의 VOC가 주어집니다.
고객 서비스, 제품 개발, 운영 지원팀 중 올바른 팀에 전달할 수 있도록 분류하세요.

Steps

1. VOC를 분석합니다.
2. VOC 내용이 고객 서비스, 제품 개발, 운영 지원팀 중 어떤 팀의 역할에 부합하는지 확인합니다.
3. 적합한 팀을 출력합니다.

Output Format

- 담당 팀: `[고객 서비스/제품 개발/운영 지원팀]`

Examples

(예시 생략)

Notes

- VOC 내용이 모호할 경우, 가장 관련 깊어 보이는 팀을 선택하세요.

Ch.3 Prompt Engineering

Few-Shot Prompting

● 프롬프트 내에 참고할 답변 예시 포함

- GPT-3에서 주로 사용하던 방법
 - Ex) 호랑이:Tiger, 사자:Lion, 쌍둥이: → Twin
- Ex) GPT-3.5 삼행시 만들기

NO

한국어로 삼행시를 작성해 주세요. 삼행시란, 단어가 주어지면 해당 단어의 2문장을 이용하여 전체적으로 연결되는 문장을 만드는 놀이입니다.

제시어는 '지피티'입니다.



지금 이 순간, 지피티와 함께
풍부한 지식을 나누며 학문의 문을 열어
더 나은 미래를 향해 함께 나아갑시다.

GPT-3.5 & GPT-4

지 ? ? ? ? ?

GPT-4o & GPT-4o mini

지 피 티

한국어로 삼행시를 작성해 주세요. 삼행시란, 제시어가 주어지면 해당 단어의 각 글자로 시작하는 문장을 이용하여 전체적으로 연결되는 문장을 만드는 놀이입니다.

제시어: '청바지'

청: '청춘은'

바: '바로'

지: '지금'

제시어: '지피티'

제시어: '지피티'

지: '지금은'

피: '피곤하지만'

티: '티비를 켜고 쉬자'

Ch.3 Prompt Engineering

Few-Shot Prompting

● 퓨 샷 프롬프팅은 예제의 다양한 특징을 참고한다

- Ex) 삼행시: 길이, 스타일, 어휘, 주제 등

한국어로 삼행시를 작성해 주세요. 삼행시란, 제시어가 주어지면 해당 단어의 각 글자로 시작하는 문장을 이용하여 전체적으로 연결되는 문장을 만드는 놀이입니다.

제시어: 청바지

청: 청춘의 뜨거운 열정으로 꿈과 희망을 안고, 어둠 속에서도 빛을 찾아 나서며,

바: 바람을 가르며 앞으로 나아가는 우리의 발걸음, 어떤 어려움도 이겨낼 수 있는 강한 의지를 가지고,

지: 지금 이 순간을 놓치지 말고, 지나가는 시간의 소중함을 깨달으며, 모든 순간을 소중히 여기고, 우리 인생의 주인공으로서 후회 없이 살아가자.

제시어: 지피티



지: 지식의 나무처럼 자라가는 우리의 미래를 준비하며,

피: 피어나는 재능과 끊임없는 노력으로 성장해 나가는 모습을 보여주고,

티: 티끌 모아 태산이라는 말을 새기며, 끝없는 도전을 멈추지 않는다.

프롬프트 엔지니어링 원리는 항상 그대로일까?

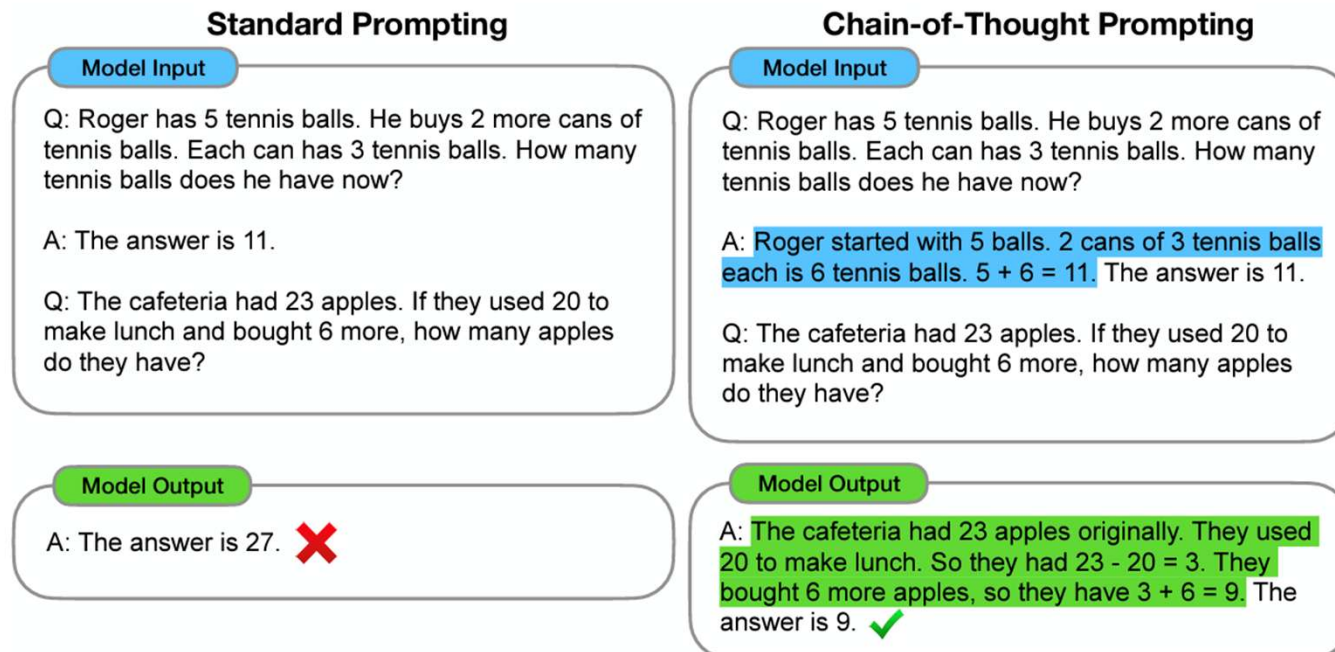
- 모델이 달라져도, 대부분의 LLM 프롬프팅 기술은 유지됨
 - LLM의 학습 방식과 데이터의 형태가 유사하기 때문
- 그러나, 추론 능력이 뛰어난 모델일수록 프롬프트에 덜 국한되는 성능을 보임
 - 예시) 문제 풀이 과정을 알려줘야 하는 sLLM / 알아서 해결하는 Big Model
 - 페르소나 프롬프팅의 경우 최근 모델에서는 덜 중요하다는 연구 결과도 있음
- LLM의 추론 과정은 어떻게 이루어지는가?
 - Chain-of-Thought

Ch.3 Prompt Engineering

Chain-of-Thought (CoT)

●언어 모델의 논리적 추론 과정 유도하기

- 문장형 수학 문제가 주어지면, 풀이과정을 먼저 출력해야 맞힐 수 있다



Source: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, <https://arxiv.org/abs/2201.11903>

Zero-shot Chain-of-Thought (CoT)

- “Let’s Think Step by Step”
- CoT 예시를 제공하지 않고, 출력 형식을 변화시키는 방법

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let’s think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Source: Large Language Models are Zero-Shot Reasoners, <https://arxiv.org/abs/2205.11916>

최근의 모델들은 Zero-Shot CoT 형식의 출력을 주로 수행

 ChatGPT 4o ▾

내가 지갑에 2만원이 있었는데, 1만 2천원 배달음식을 주문하고, 2천원 배달팁을 냈어.
이후, 은행에 들어서 5만원권 20장을 인출했어.
내가 가진 현금은 얼마일까?



현재 가지고 있는 현금을 계산해 보겠습니다:

1. 초기 현금:

2만 원

2. 배달음식 비용:

1만 2천 원 + 2천 원(배달팁) = 1만 4천 원 지출

남은 금액:

2만 원 - 1만 4천 원 = **6천 원**

3. 은행에서 인출한 금액:

5만 원권 20장 = 5만 원 × 20 = **100만 원**

4. 최종 금액:

6천 원 + 100만 원 = **100만 6천 원**



추론을 금지시킨다면?

- 언어 모델의 특성상, 오답이 발생함

 ChatGPT 4o ▾

내가 지갑에 2만원이 있었는데, 1만 2천원 배달음식을 주문하고, 2천원 배달팁을 냈어.
이후, 은행에 들어서 5만원권 20장을 인출했어.
내가 가진 현금은 얼마일까? 풀이과정을 쓰지 말고, 정답만 출력해줘.
정답:00원



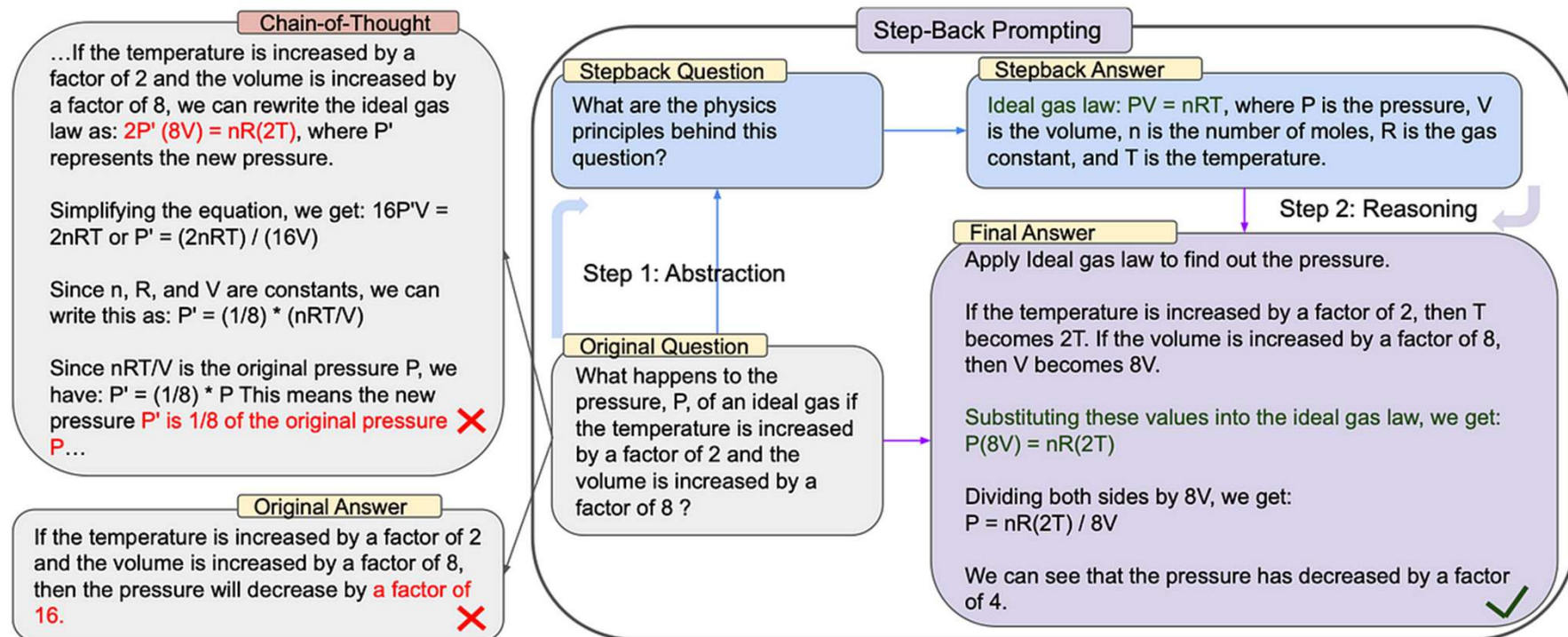
정답:1,002,000원



Ch.3 Prompt Engineering

파생) Step-back Prompting

●답을 내기 전에 답변에 필요한 것을 먼저 도출한다



Source: Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models <https://arxiv.org/abs/2310.06117>

프롬프트 길이와 비용-속도 관계

- 상용 LLM: 토큰당 비용 발생
- 오픈 LLM: 추론 속도 및 GPU 소모 증가
- 예시) GPT-4o 사용 비용 (2025년 기준)

Model	Pricing
gpt-4o	\$2.50 / 1M input tokens
	\$1.25 / 1M cached** input tokens
	\$10.00 / 1M output tokens

프롬프트 개선: Rule of Thumb

- OpenAI Prompt Guide에서 권장하는 일반적 기준
 - 감지할 차이가 작을수록, 더 많은 데이터가 필요함

DIFFERENCE TO DETECT	SAMPLE SIZE NEEDED FOR 95% CONFIDENCE
30%	~10
10%	~100
3%	~1,000
1%	~10,000

프롬프트 작성 Tip

- 오픈 LLM 모델의 경우, 한국어 성능의 편차가 심할 수 있음
 - Try: 지시 사항을 영어로 작성하기
- 알려진 다국어 모델이나, 최신 모델의 한국어 파인 튜닝 버전 찾기 (라이선스 주의)
 - 2025년 기준 대표적 한국어 성능 높은 모델
 - Alibaba Qwen 2.5, 3 / Google Gemma 3 (Commercial) / Microsoft Phi-4 14B
 - Cohere Aya-Expanse / LG ExaOne (Non-Commercial) /
- 한국어 성능 확인 지표: 호랑이 리더보드
 - <https://wandb.ai/wandb-korea/llm-leaderboard3/reports/W-B-Korean-LLM-Leaderboard-v3--Vmlldzo5NTM4MjU0>
- 메타 프롬프팅을 잘 활용하기

그 외의 프롬프트 엔지니어링 가이드

● 프롬프팅 가이드

- <https://www.promptingguide.ai/kr>

● OpenAI

- <https://platform.openai.com/docs/guides/prompt-engineering>

● Claude

- <https://docs.anthropic.com/claude/docs/prompt-engineering>

● Gemini

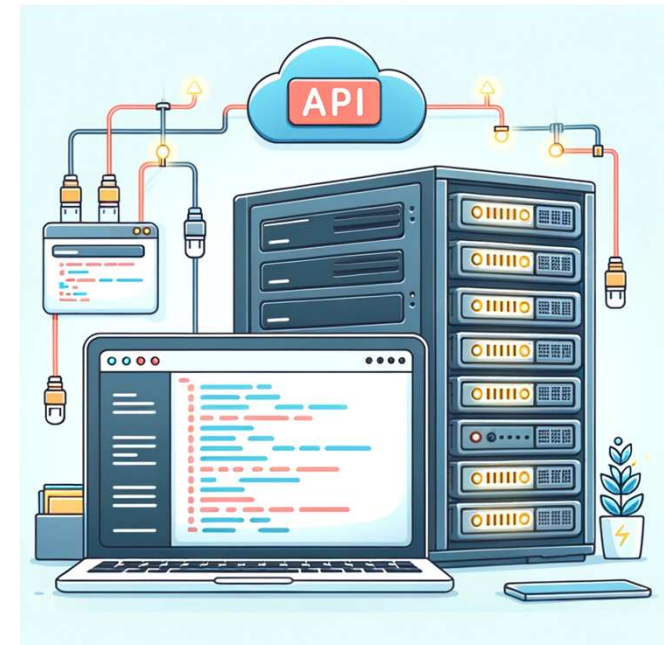
- <https://ai.google.dev/gemini-api/docs/prompting-intro?hl=ko>

OpenAI API

변형호

OpenAI API

- OpenAI의 모델을 호출하여 사용 가능
 - GPT, DALL-E, TTS, Whisper, Embedding, ...
- 웹 어플리케이션 형태가 아닌,
GPT의 능력을 프로그램에 탑재하는 방법



OpenAI Chat API

● OpenAI SDK Library (Python, Node.js)

- <https://github.com/openai/openai-python>
- 2024.11 이후 Gemini도 액세스 가능
- Ollama, vLLM과 같은 오픈 모델용 라이브러리,
Groq, Cerebras의 클라우드 라이브러리와도 호환

```
1 curl https://api.openai.com/v1/chat/completions \  
2   -H "Content-Type: application/json" \  
3   -H "Authorization: Bearer $OPENAI_API_KEY" \  
4   -d '{  
5     "model": "gpt-4o-mini",  
6     "messages": [  
7       {  
8         "role": "system",  
9         "content": "You are a helpful assistant."  
10      },
```

Ch.4 OpenAI API

OpenAI API Key

● 서버 통신을 위한 인증 키

- ChatGPT Plus, Pro 가입과 별개로 요금 부과
- <https://platform.openai.com/api-keys>

API keys

+ Create new secret key

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	PERMISSIONS	
Secret key	sk-...M14A	All	 
Secret key	sk-...xZcA	All	 

OpenAI API Billing

- Billing 등록 후 결제 가능

- <https://platform.openai.com/settings/organization/billing/overview>

Pay as you go

Credit balance ⓘ

\$32.22



Auto recharge is on

When your credit balance reaches \$10.00, your payment method will be charged to bring the balance up to \$50.00.

[Modify](#)

Add to credit balance

Cancel plan



Payment methods

Add or change payment method



Billing history

View past and current invoices



Preferences

Manage billing information



Usage limits

Set monthly spend limits



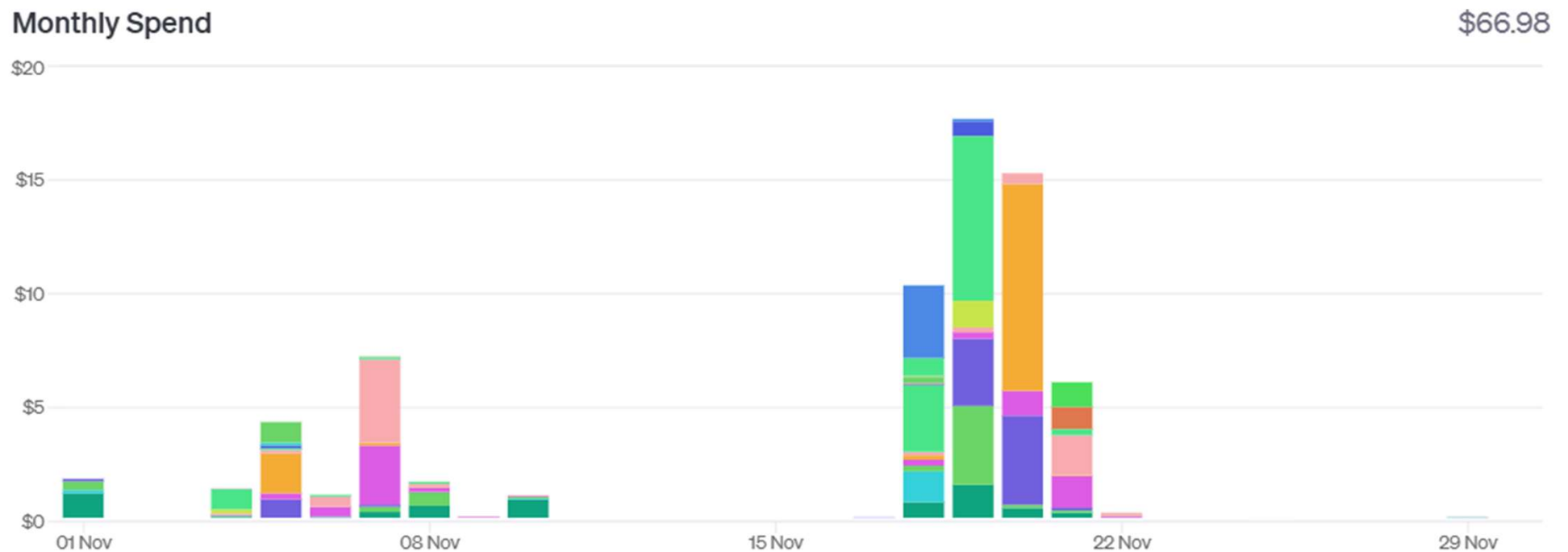
Pricing

View pricing and FAQs

OpenAI API Key

● 대시보드를 통한 사용량 트래킹

- <https://platform.openai.com/settings/organization/usage>



Ch.4 OpenAI API

등급별 사용량 상한선

● <https://platform.openai.com/docs/guides/rate-limits>

● Free Tier의 경우 사용량 제한이 매우 낮음

TIER	QUALIFICATION
Free	User must be in an allowed geography
Tier 1	\$5 paid
Tier 2	\$50 paid and 7+ days since first successful payment
Tier 3	\$100 paid and 7+ days since first successful payment
Tier 4	\$250 paid and 14+ days since first successful payment
Tier 5	\$1,000 paid and 30+ days since first successful payment

Free tier rate limits

This is a high level summary and there are per-model exceptions to these limits (e.g. some legacy models or models with larger context windows have different rate limits). To view the exact rate limits per model for your account, visit the **limits** section of your account settings.

MODEL	RPM	RPD	TPM	BATCH QUEUE LIMIT
gpt-4o-mini	3	200	40,000	-
text-embedding-3-large	100	2,000	40,000	-
text-embedding-3-small	100	2,000	40,000	-
text-embedding-ada-002	100	2,000	40,000	-
omni-moderation*	250	5,000	10,000	-
whisper-1	3	200	-	-
tts-1	3	200	-	-
dall-e-2	5 img/min	-	-	-
dall-e-3	1 img/min	-	-	-

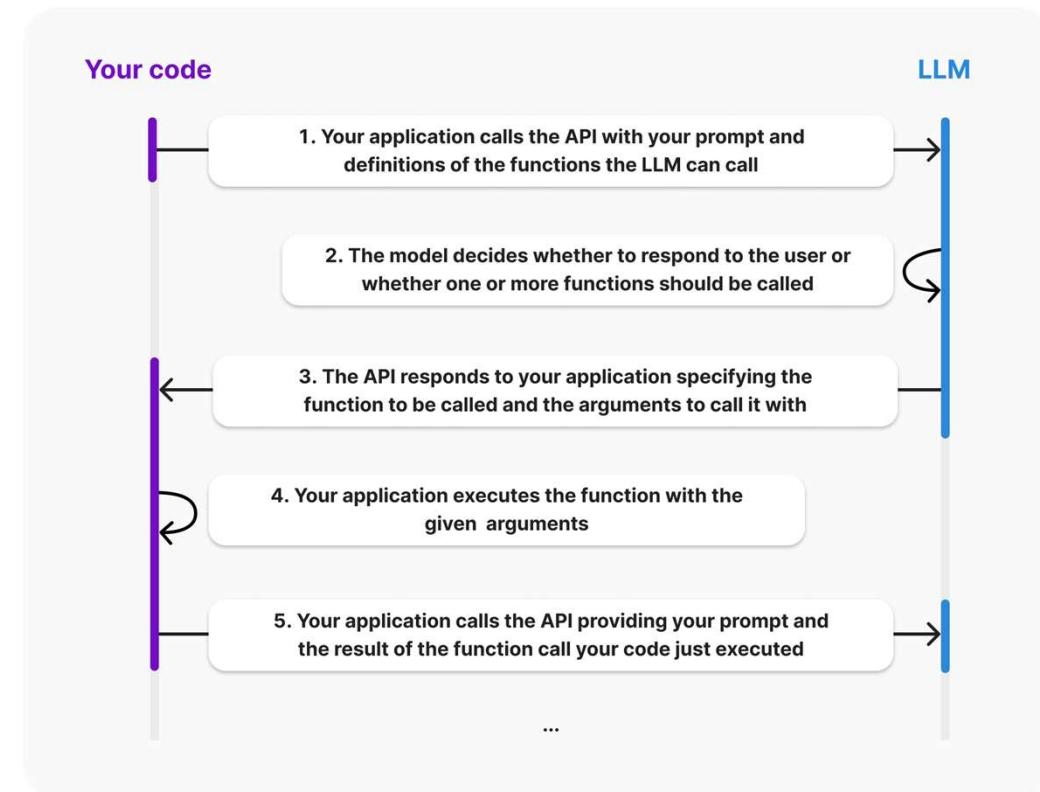
[실습] OpenAI API 활용

- Chat API를 통한 GPT 프롬프팅(텍스트/이미지/보이스)
- 임베딩, DALL-E , Voice API 사용해 보기

```
response = client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages = messages,  
    temperature=0.2,  
    # temperature: 무작위 출력을 조절: (0-2) 0에 가까울수록 정해진 답변을 수행  
    max_tokens = 512,  
    # 출력 최대 토큰 수 조절: 초과할 경우 자름  
    # n = 1  
    # 여러 개의 출력 가능  
)  
print(response)
```


OpenAI Tool Calling

- LLM Agent를 위해 최근 매우 주목받는 기능
- Tool Calling: 외부 함수나 API를 LLM에 연결하는 방법
 - LLM에게 툴 정보를 전달하면, 시스템 메시지에 포함
 - Ex) “다음의 툴을 사용할 수 있습니다. <툴 1번 사용법>, ...
- 툴이 필요하다고 판단하는 경우
Chat Message 대신 Tool Call Message를 Return
 - 툴 실행 결과를 Tool Message로 GPT에 다시 전달하면
다음 응답 생성



Ch.4 OpenAI API

[실습] OpenAI API와 Tool Calling을 이용한 뉴스 요약 봇 만들기

- 웹 검색 결과를 프롬프트로 전달하여 요약하기
- 질문에 대해 적절한 검색 방법 스스로 판단하기

```
result = news_bot([
    {
        "role": "user",
        "content": news_json
    },
    {
        "role": "user",
        "content": ""
    }
])

---
위 전체 내용을 종합하여 회사 동향에 대
---
'제목':
'본문':
""
```

 뉴시스 PICK · 1일 전 · 네이버뉴스

20대 리즈 시절로 돌아간 국민 대배우...'AI 나문희' 탄생
인공지능(AI) 기술을 통해서다. AI 엔터테인먼트 기술 기업 엠씨A)는 한국마사회와의 협업으로 'AI 나문희 배우'와 함께 하는 브텐츠를 선보였다고 9일 밝혔다. '나문희의 첫사랑'이라는 제목의

 한국경제 PICK · 13시간 전 · 네이버뉴스

"논리적으로 생각해 계획 세우는 인공지능 나온다"

논리적으로 추리하고 계획까지 세우는 인공지능(AI) 기술이 곧 기존의 AI가 광범위한 데이터에서 직접적인 결론을 도출하는 커ce)에 그쳤다면, 이제는 여러 단계의 논리적 사고와 정보의 통합

| "오픈AI·메타, 진화된 인공지능 언어모델 곧 출시" KBS · 7시간 전

OpenAI Assistant API

● Assistant : OpenAI API로 GPTs와 유사한 기능 수행 가능

- Instruction 기반의 행동 양식 설정
- Code Interpreter, File Search, Function Call 등의 기능 수행
 - Code Interpreter: 질문을 해결하기 위한 파이썬 코드를 실행함
 - File Search: 주어진 파일을 탐색하여 정보를 활용함
 - Function Call: API 이외의 함수를 활용함(사전 정의 필요)

● 2025년 3월 이후 Response API로 통합 예정

실습) OpenAI Response API 사용하기

- Chat Completion API의 개선 방향 실제 코드로 확인하기

LangChain

변형호

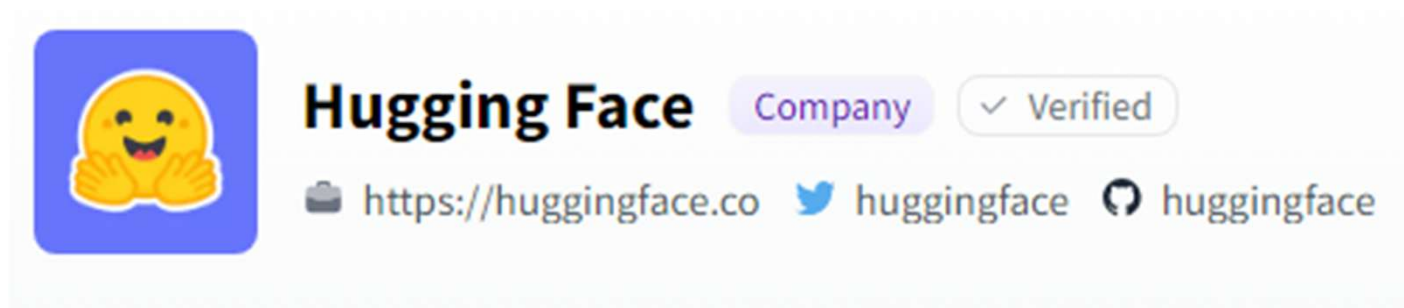
LangChain이란?

- LLM 어플리케이션을 쉽게 개발하게 해 주는 프레임워크
- Main Point: 효과적인 Abstraction
 - 새로운 기능을 추가하기보다는, 쉬운 구현을 돕는 역할
- LLM Orchestration 프레임워크
 - 다양한 LLM 모델 입/출력
 - 프롬프팅, 데이터 연결, 메모리
 - 외부 툴 사용과 에이전트



LangChain + HuggingFace

- HuggingFace: 오픈 모델, 데이터셋 공유 커뮤니티
 - langchain-huggingface 라이브러리 기반의 높은 호환성
 - <https://huggingface.co>



LangChain VS OpenAI API

- OpenAI API만으로도 어플리케이션 구현 가능하나...
- LangChain은
 - OpenAI와 오픈 모델을 포함한 대부분의 모델에서 동일한 인터페이스로 사용 가능
 - OpenAI 내부 기능도 대부분 사용 가능
 - 코드가 API SDK에 비해 간결함

ChatOpenAI in LangChain

- *langchain-openai* 라이브러리를 통해 연결

```
from langchain_openai import ChatOpenAI  
  
llm = ChatOpenAI(model = 'gpt-4o-mini', temperature=0.5, max_tokens=1024)
```

- *invoke()* 를 통해 실행

- 대부분의 LangChain 구성요소는 *invoke*로 실행됨

```
question = '''전 세계적으로 흥행한 영화에 나오는 유명한 명대사를 하나 알려주세요.  
대사가 나온 배경과 의미도 설명해 주세요.'''  
  
llm.invoke(question)
```

실습) LangChain 기초

LangChain 어플리케이션의 구성 방식

- LangChain 체인의 기본 구성

- 프롬프트 - LLM

- 파서: LLM의 출력을 변환하는 모듈 (json, string, custom, ...)

- 프롬프트 - LLM - 파서
- 프롬프트 - LLM - 파서 - 프롬프트 - LLM - 파서

- 이외에도 함수, 툴, 검색기 등의 모듈을 결합할 수 있음

- 프롬프트 - LLM - 파서 - 함수
- 함수 - 프롬프트 - LLM - 파서

LCEL

● LangChain Expression Language

- 각각의 LangChain 모듈을 파이프(|)를 통해 체인으로 연결
 - Ex) *Prompt* | *LLM* | *Parser*
 - Invoke()를 통해 한 번에 실행 가능
- 다양한 체인을 구성하여, 입/출력 및 중간 과정을 직관적으로 구현할 수 있음

Runnables

● LangChain의 기본 요소

- LLM, Prompt, Parser, Tool, ...
- Invoke()로 실행 가능

● 특수 Runnable

- RunnablePassthrough() : 직전 Runnable의 출력을 그대로 전달
- RunnableParallel(): 1개 이상의 Runnable을 실행하고, 그 결과를 Dict로 저장
- .assign() : Runnable 값을 전달하고, 그 결과를 가져와 결합하는 기능

실습) LangChain Expression Language

- Prompt | Llm 의 간단한 체인부터, Parser를 포함한 다양한 체인 구성하기
- 다양한 Runnable의 의미 이해하기

Ch.5 LangChain

LangChain 개발 프레임워크

● LangChain 연계 라이브러리

- LangSmith : 성능 모니터링
- LangGraph: 에이전트 구현
- LangServe: API 서빙

● LLMOps를 고려한다면

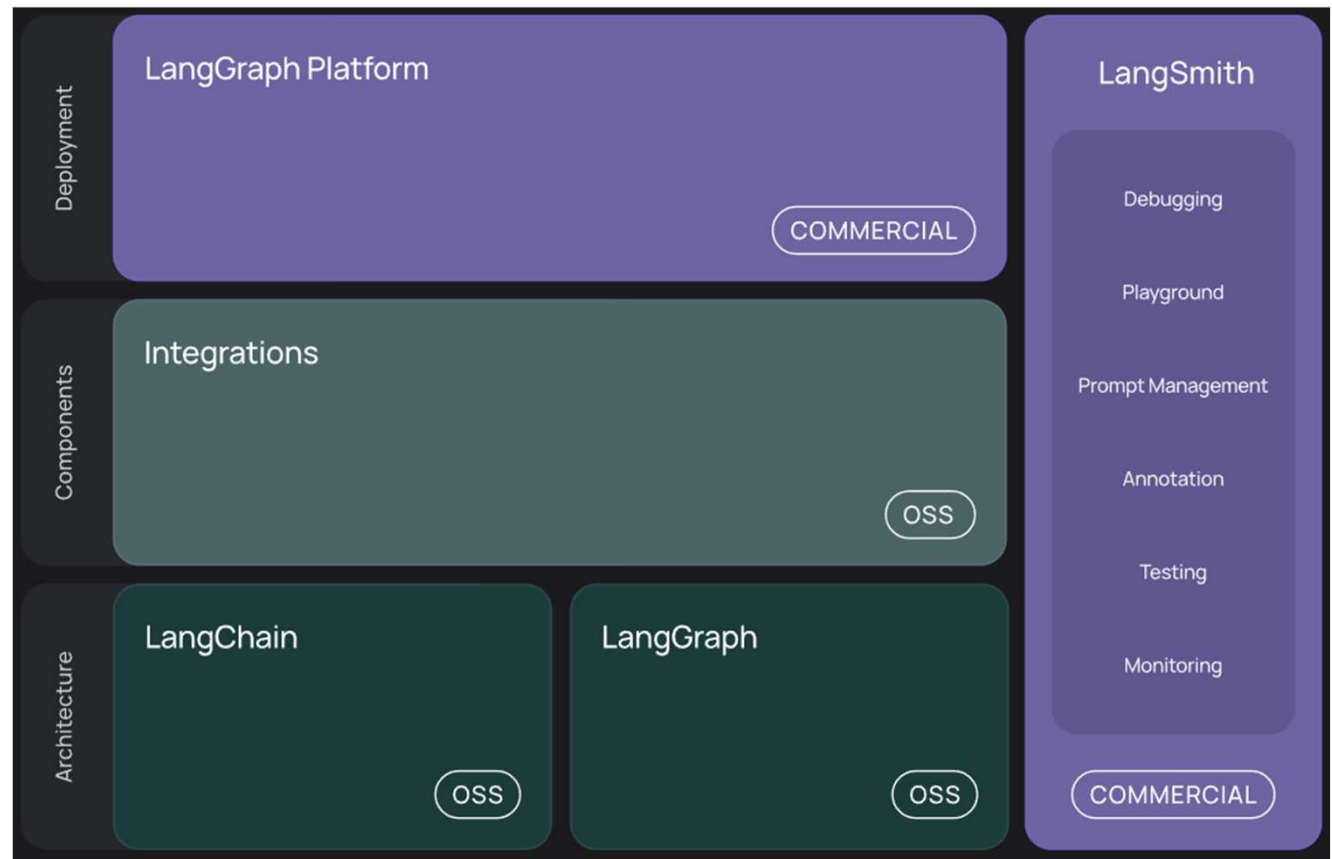
- LangSmith

● 복잡한 워크플로우/에이전트 구현

- LangGraph

● API 형태 서빙

- LangServe(다른 대체도 많음)



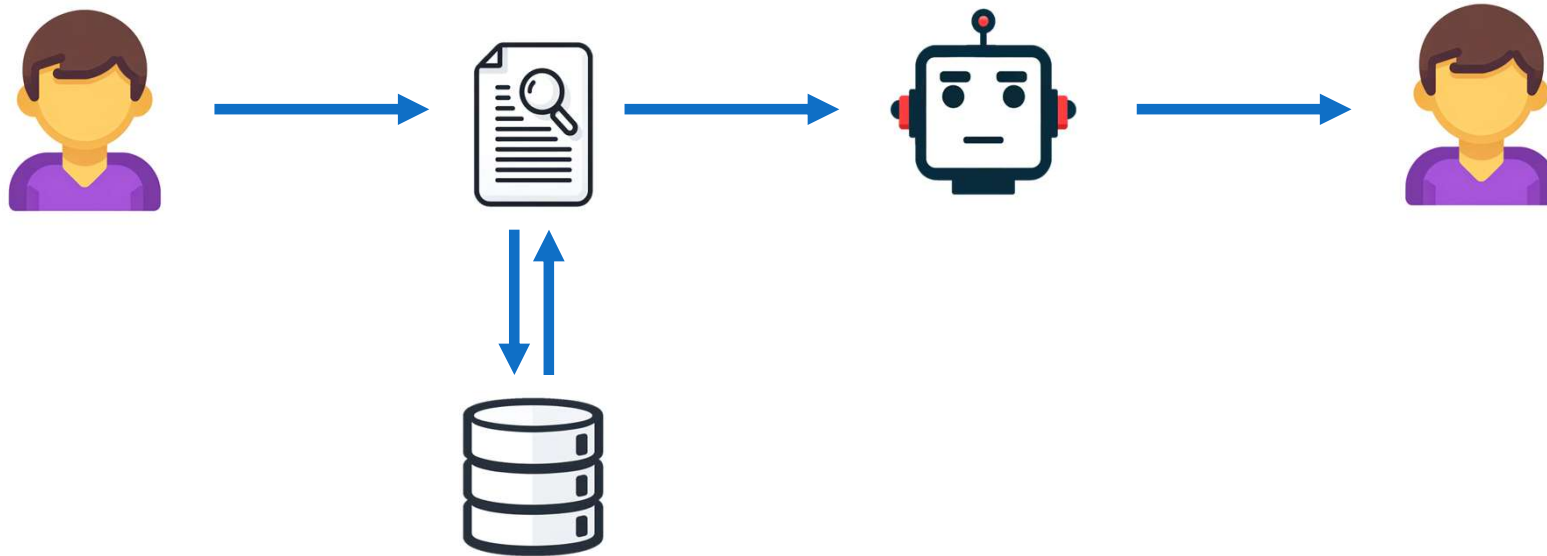
Source: <https://python.langchain.com/docs/introduction/>

Retrieval-Augmented Generation

검색 증강 생성 (Retrieval-Augmented Generation)

● LLM+ 정보 검색

- LLM의 In-Context Learning 능력 기반
- 질의와 관련된 정보를 검색하여 이를 프롬프트에 함께 전달하는 방법



Ch.6 Retrieval-Augmented Generation

RAG 예시 프롬프트

● Instruction + Context + Question

- 할루시네이션을 완화하고, 최신 데이터/ 내부 데이터에 대해서 답변할 수 있음

다음의 [Context]를 참고해 [Question]에 답하세요.

[Context]의 정보만으로 답변할 수 없는 경우, '답변할 수 없습니다'를 출력하세요.

[Context]:

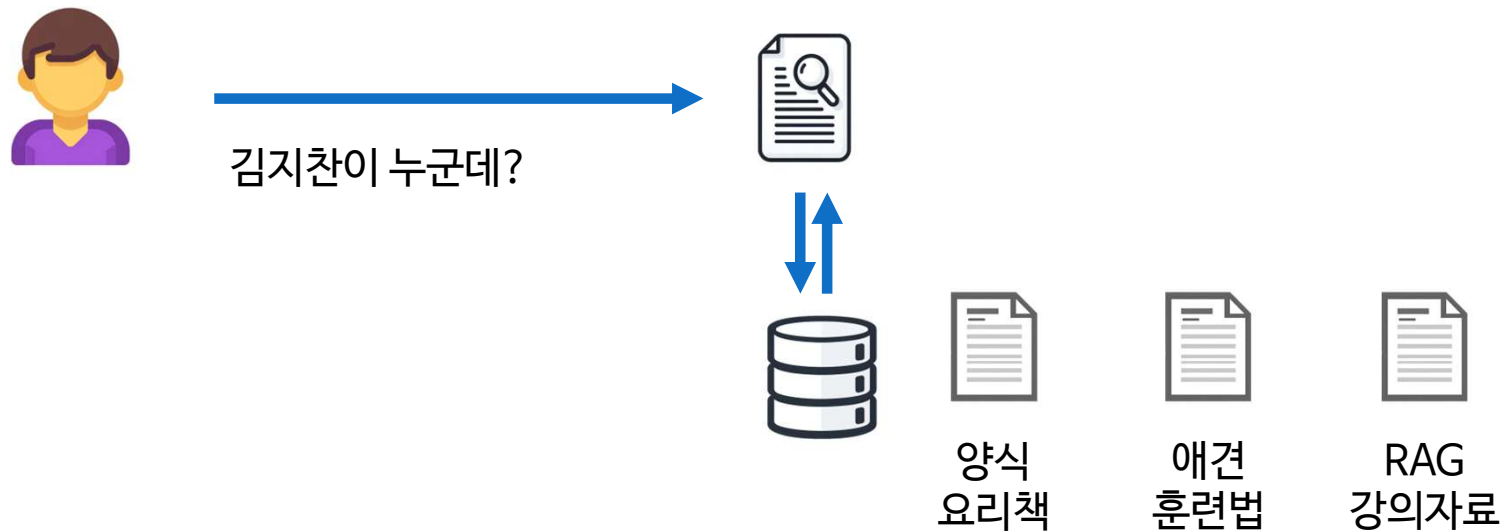
김지찬 / 개요 삼성 라이온즈 소속 우투좌타 중견수. 주 포지션은 2루수 및 중견수이다. / 전형적인 슬랩 히터이며 컨택, 선구안에 강점을 보인다. 통산 도루 성공률 1위(88.1%)를 기록할 정도로 발이 빠르다 2024년 중견수로 전향하며 매우 높은 성장을 보였다.

[Question]:

김지찬의 통산 도루 성공률이 얼마나 돼?

Retrieval : 정확한 Context 제공하기

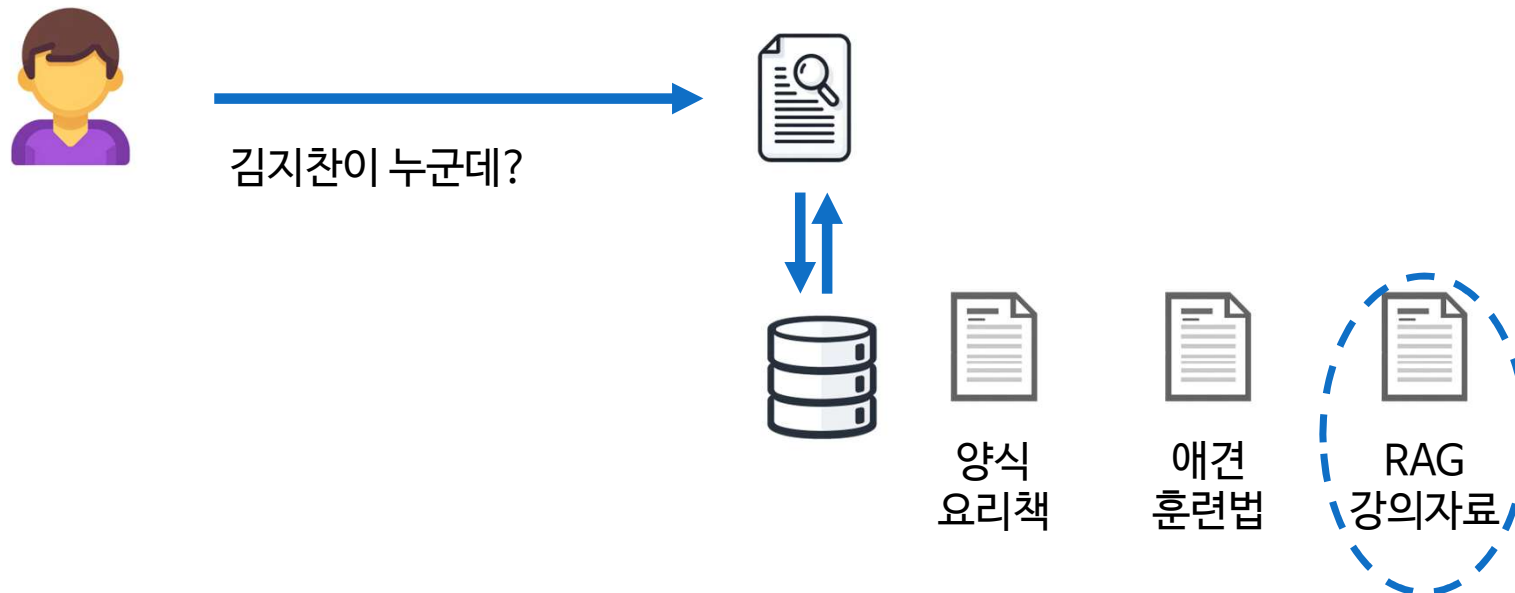
- Retrieval의 결과가 나쁜 경우
RAG의 성능은 기대할 수 없음



Context를 어떻게 구성하는가?

● 임베딩 벡터/ 키워드 기반의 유사도 Top K Search

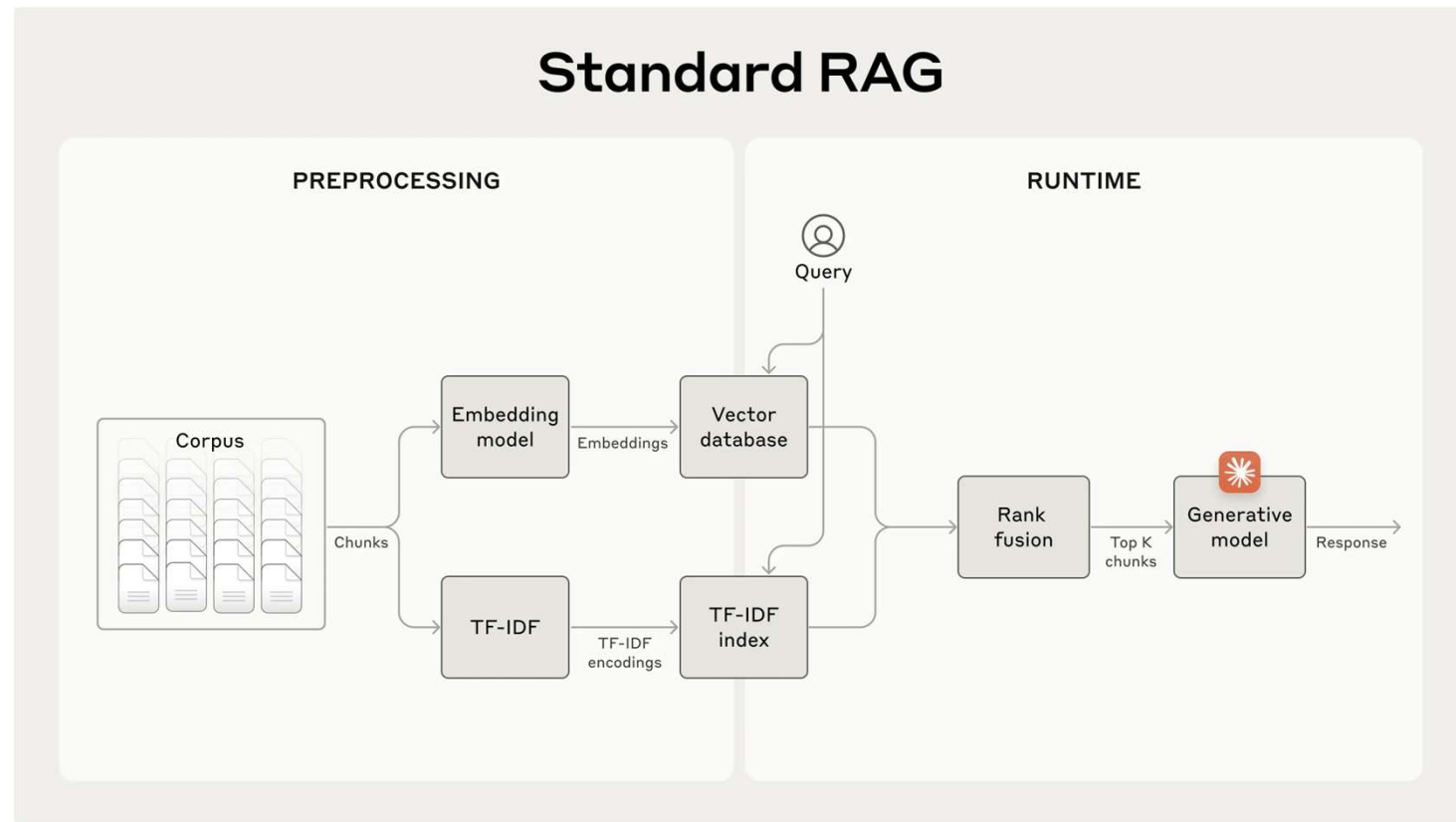
- Top K가 모두 필요한 경우는 적기 때문에,
Relevant 정답 문서 + 일부 Irrelevant 노이즈의 형태로 주로 구성



Ch.6 Retrieval-Augmented Generation

RAG의 5-step Process

- Indexing
 - 데이터베이스 구성
- Processing
 - 질의 전처리
- Searching
 - 질의와 적합한 데이터 검색
- Augmenting
 - RAG 프롬프트 생성
- Generating
 - LLM 답변 생성



Source: <https://www.anthropic.com/news/contextual-retrieval>

Indexing: 데이터 준비하기

- Context의 길이는 제한적이므로 전체 문서를 입력하는 것은 효과적이지 않음
 - 필요한 부분만 검색할 수 있도록 분리(Chunking)
 - Ex) 페이지 단위로 데이터베이스에 저장
- 일반적인 텍스트의 경우, 문자나 토큰 단위의 청킹 수행
 - 청크 사이즈: 각 청크의 크기
 - 최적의 청크 사이즈는 도메인/데이터마다 다를 수 있음

Ch.6 Retrieval-Augmented Generation

Indexing: 적절한 청크 사이즈 선택하기

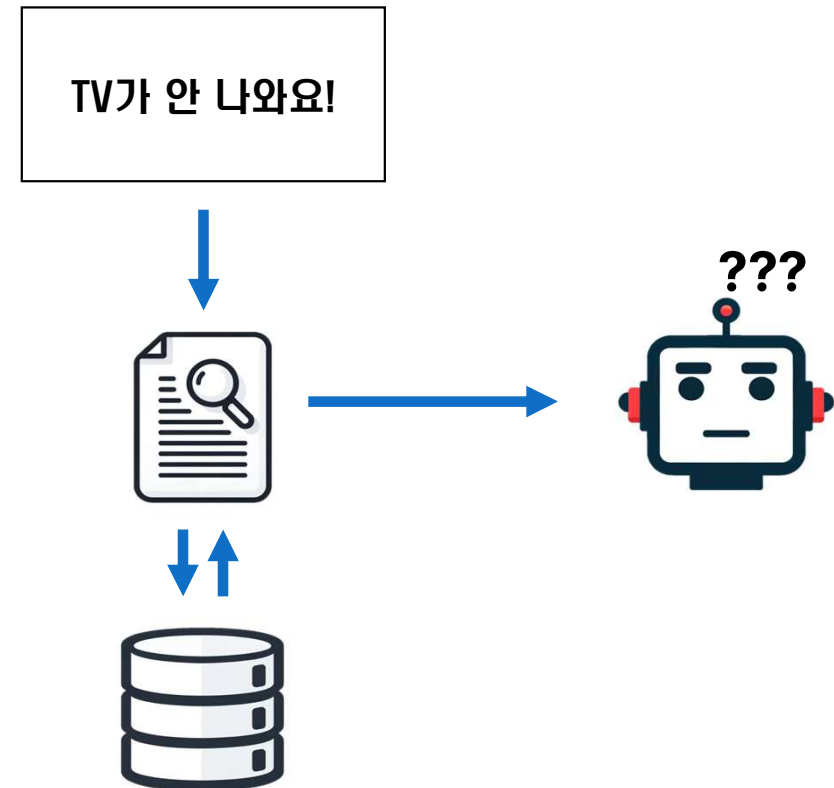
- 청크 크기가 작은 경우
 - 주변 정보 활용 어려움
- 청크 크기가 큰 경우
 - 불필요한 정보 포함
 - 임베딩의 정확도 감소
- 청크 오버랩
 - Context의 의미 보존

제목: RAG의 이해
 RAG를 활용하면, LLM이
 사용자의 입력에 대해 보다
 정확한 답변을 생성하도록
 유도할 수 있습니다. 그 이유는
 프롬프트 내에
 데이터베이스의 내용을
 포함하여 이를 활용할 수 있기
 때문입니다.
 데이터베이스는 청크 단위로
 저장되며, 적절한 청크
 사이즈를 찾는 것이
 중요합니다.

Ch.6 Retrieval-Augmented Generation

Processing : 사용자 쿼리 처리하기

- 검색이 잘 되도록 하기 위한 전처리
 - RAG로 답변할 문제인지를 사전에 파악하기
- 너무 짧은 쿼리는
 - 의미 이해를 위한 정보가 불충분
 - 이전 대화 내역을 고려하여 Contextualize
- 너무 긴 쿼리는
 - 불필요한 정보가 포함
 - 요약하거나 일부 내용 삭제



Searching: 쿼리와 적합한 데이터 검색하기

● Semantic 검색과 Lexical 검색

- Embedding 기반의 Semantic 검색
 - 정확하게 일치하지 않아도 유사한 의미를 탐색
- Keyword 기반의 Lexical 검색 (BM25, TF-IDF,...)
 - 정확하게 일치하는 경우에 높은 가중치
 - 모델명, 법령 등 용어 검색에 유리

● Semantic 검색의 경우, 벡터 데이터베이스를 활용

Searching: 벡터 데이터베이스 = 임베딩 저장 공간

- 벡터 형태의 데이터 저장 및 검색에 최적화된 소프트웨어
 - 비정형 데이터를 임베딩으로 변환하고, 이를 저장/검색
 - 트랜스포머 기반 기존 Text Embedding 기술 활용
- 자연어, 그래프, 이미지 등의 데이터가 많아지면서 중요도 증가
 - LLM뿐만 아니라 데이터베이스 자체로도 활용

Searching: 벡터DB 활용 예시

- 검색: 질문(Query)과 청크(Chunks) 비교
 - 질문의 임베딩과 청크들의 임베딩 유사도 계산
 - Top K Search 후 Return
- 청크 검색 후
 - Task 에 따른 활용
 - RAG : LLM 프롬프트에 추가하여 답변 생성
 - Recommendation : 해당 상품 혹은 문서 전달

Ch.6 Retrieval-Augmented Generation

Searching: 대표적 Vector Database

- Pinecone : 클라우드 기반의 유료 서비스
- Milvus, Qdrant, Chroma, Weaviate : 무료 사용 지원
- Popularity 순위
 - Elastic Search와 같이 Vector 지원되는 DB를 사용해도 됨

Rank			DBMS	Database Model	Score		
Dec 2024	Nov 2024	Dec 2023			Dec 2024	Nov 2024	Dec 2023
1.	1.	1.	Elasticsearch	Multi-model ⓘ	132.32	+0.68	-5.43
2.	2.	2.	Kdb +	Multi-model ⓘ	6.99	-0.07	-1.29
3.	3.	3.	Aerospike	Multi-model ⓘ	5.34	+0.01	-1.84
4.	4.	↑ 5.	Pinecone	Vector	3.23	+0.15	-0.13
5.	5.	↑ 8.	Milvus +	Vector	2.82	-0.04	+1.33
6.	6.	↓ 4.	DolphinDB	Multi-model ⓘ	2.54	-0.05	-1.37
7.	7.	↓ 6.	Chroma	Vector	1.87	+0.05	-0.40
8.	8.	↑ 10.	Qdrant	Vector	1.85	+0.08	+0.60
9.	9.	↓ 7.	Weaviate +	Vector	1.70	+0.25	-0.33

Source: <https://db-engines.com/en/ranking/vector+dbms>

Searching: 대표적 Vector Database

● Vector Database의 Metric Types

- Cosine Distance
 - 1- Cosine Similarity
- Euclidean Distance (L2 Distance)
 - 벡터 간의 직선거리
- MMR (Maximal Marginal Relevance)
 - 다양성을 고려한 방법
 - $\text{Alpha} * \text{쿼리와의 유사성} - (1-\text{Alpha}) * (\text{Context 중 가장 가까운 청크와의 유사성})$ 순으로 검색

Searching: 대표적 Vector Database

- 모든 벡터를 1:1로 계산하는 것은 시간이 오래 걸림

- 검색 공간을 근사하여, 분할하거나 압축하고 계산량을 줄이는 방식 선택

- 벡터 데이터베이스의 검색 알고리즘

- ANN: 근사 최근접 이웃법(Approximated Nearest Neighbors)
 - ANNOY (ANN Oh Yeah) : 전체 벡터들을 트리로 변환하여 검색
- HNSW: 계층적 작은 세상 탐색 (Hierarchical Navigable Small World)
 - 전체 벡터들을 그래프로 변환하여 검색

실습) LangChain으로 RAG 어플리케이션 만들기

Advanced RAG

변형호

현대 RAG의 요구사항

- 많은 정보가 필요한 문제 풀기

- Single Retrieval로는 해결하기 어려운 문제

- 특수한 도메인에서 응답하기

- RAG 이외의 지식이 필요하기도 함

- 빠른 시간 안에

- Accuracy/Latency Tradeoff

- 정확하게 답변

- Self Correction/Factual Groundedness

Ch.6 Retrieval-Augmented Generation

Indexing: Small2Big Chunking/ Sliding Window

- 검색 정확도를 위해 청크의 사이즈를 최대한 작게 구성하고, 검색 이후 해당 청크의 주변 정보를 포함해 전달하기
- Big Chunk/Small Chunk의 2단 구성
 - 작은 청크로 검색, 연결된 큰 청크 전달
- Sliding Window
 - Small Chunk의 위치 기준으로 앞뒤 문맥 포함하여 전달

제목: R-G Decoupling (디커플링)
 텍스트의 길이가 길어질수록 임베딩
 알고리즘은 해당 내용을 모두
 포함하게 되므로 검색에서의
 정확도가 감소할 수 있습니다.

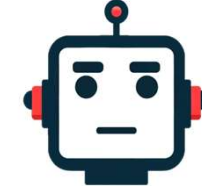
이를 해결하기 위한 Small2Big
 Chunking/ Sliding Window
 는 작은 크기의 청크로 검색을 하고,
 실제 Augmenting 단계에서는 큰
 청크를 포함하는 방법입니다.
 예를 들어, (중략)
 이 방법은 임베딩의 효과적 검색과 긴
 Context를 모두 활용할 수 있는
 방법으로 활용되고 있습니다.

Ch.6 Retrieval-Augmented Generation

Contextual Retrieval

- 청킹의 앞부분 단절을 완화하기 위해,
전체 맥락을 고려해 청크의 앞부분을 추가
- 키워드 없는 대명사/ 상대적 표현/ 앞부분 분절과 같은 부정확한 청킹 완화 효과
- Claude, OpenAI, Gemini의 Context Caching을 통해 비용을 절약할 수 있음 (오픈 모델 사용도 가능)
 - Claude의 Report에서, Hybrid 검색 성능이 대폭 향상

이용자 수가 전년 대비 37% 증가하며, 해당 서비스의 성공적인 시스템 통합을 입증했습니다.



헤더 추가 요청
전체 문서
청크

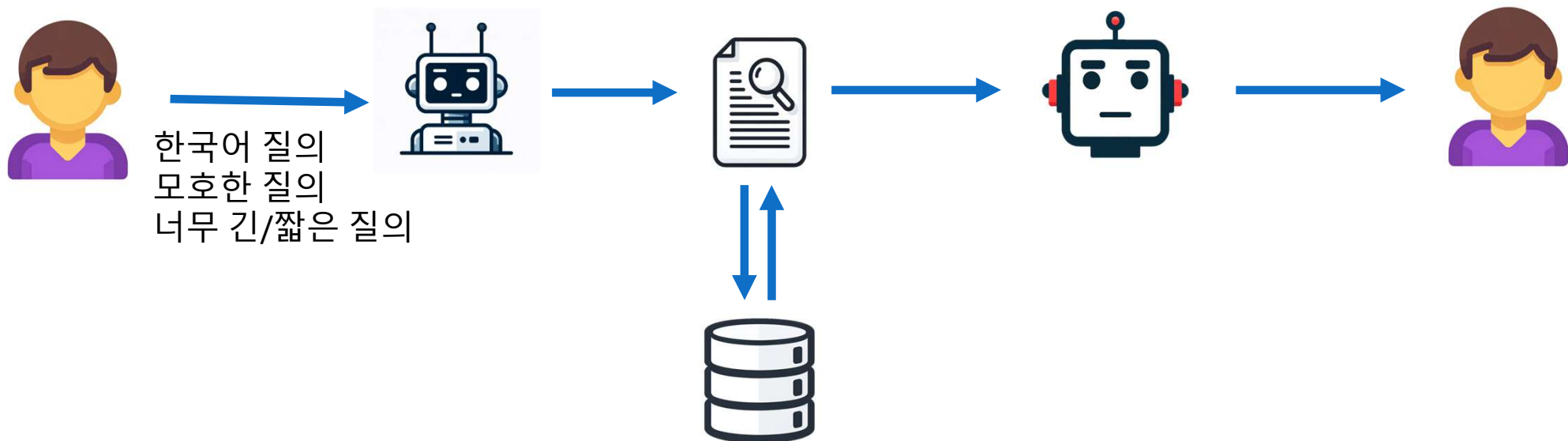
이 청크는 A회사의 LLM 기반 노트 작성 앱에 대한 2025년 보고서의 내용입니다. 2024년 이용자 수는 00명이었습니다. [기존 청크 위치]

Ch.6 Retrieval-Augmented Generation

Processing: 사용자의 질문 전처리하기

● 단순 Top-K 비교는 실사례의 복잡한 쿼리에서 효과가 떨어짐

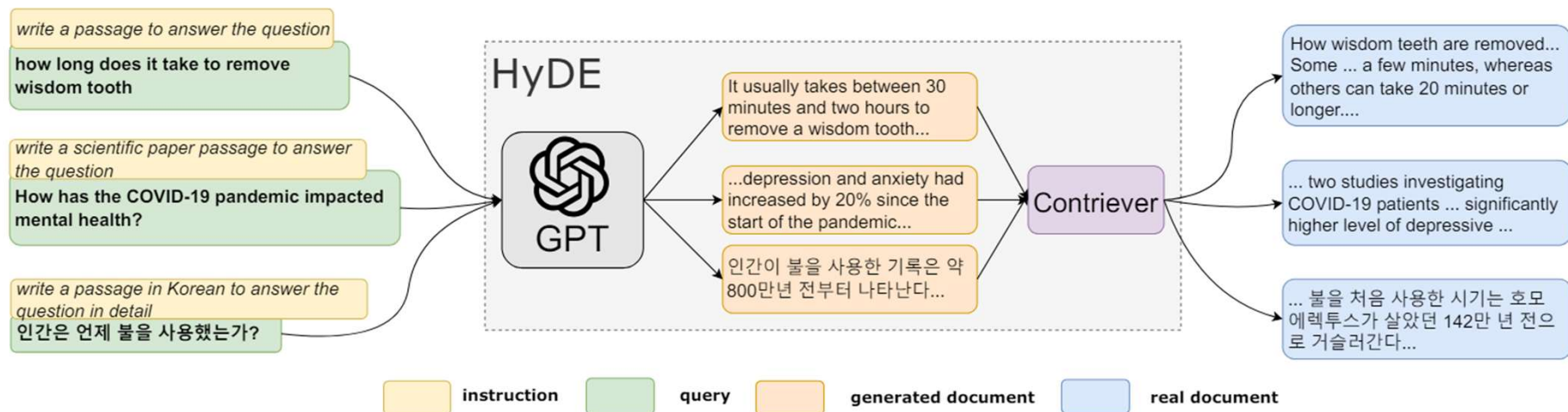
- 질문자의 정보/이전 맥락을 고려한 맥락화(Contextualize)
- 쿼리를 실제 DB와 유사하게 재구성하는 HyDE
- 쿼리를 세부 질문/단계별 질문으로 분할하는 재구성(Reform)



Ch.6 Retrieval-Augmented Generation

HyDE: Hypothetical Document Embedding

- 사용자의 질문은 특정 용어나 세부사항이 포함되지 않는 경우가 있음
 - LLM을 이용해, 질의에 대한 가상의 답변이나 관련 문서를 생성
 - 해당 문서를 통해 다시 검색하면, 질의로 검색하는 것에 비해 정확도가 높다는 결과
 - 이후 RAG 파이프라인에 연결

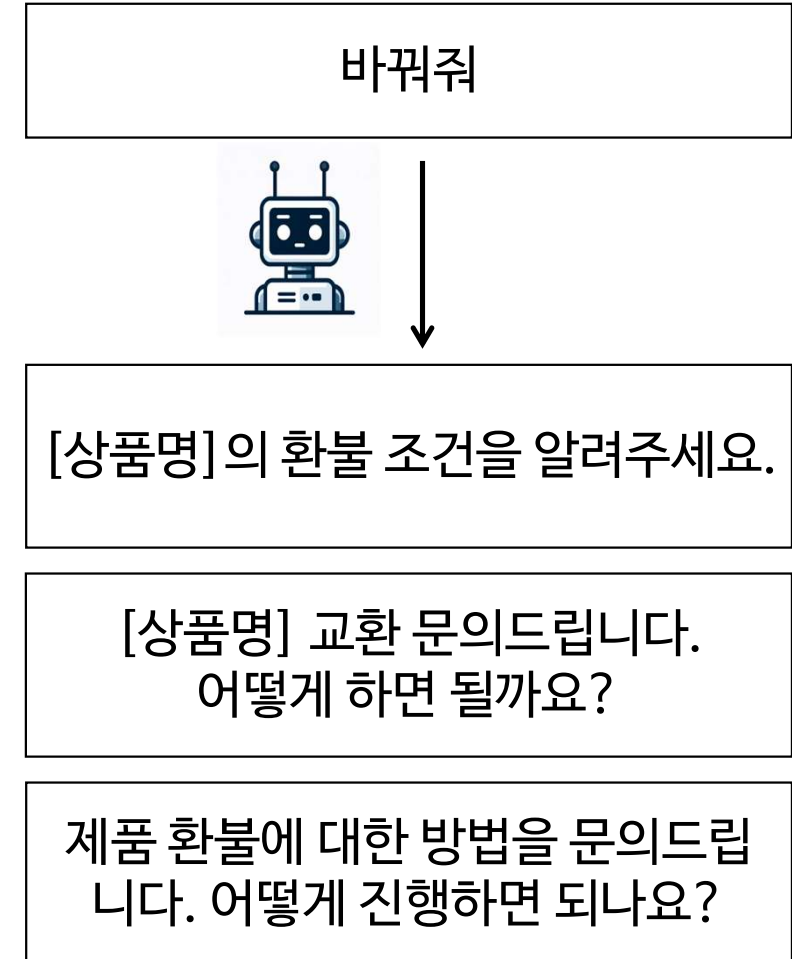


Source: Precise Zero-Shot Dense Retrieval without Relevance Labels <https://aclanthology.org/2023.acl-long.99.pdf>

Ch.6 Retrieval-Augmented Generation

Query Reformulation: 다중 검색 수행하기

- 하나의 질문을 다각도의 질문으로 변환하는 Multi-Querying
- Chain-of-Thought와 같이 순차적 질문으로 변환할 수도 있음
 - 중간 검색 결과/답변을 보고, 다음 질문을 생성하는 방식
 - 이 부분은 Agentic RAG에서 자세히 다룰 예정



Ch.6 Retrieval-Augmented Generation

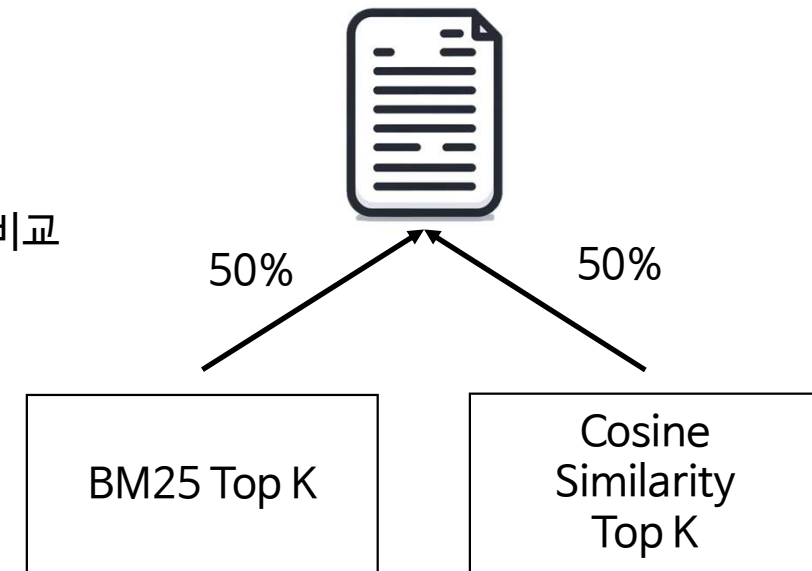
Searching / Augmenting: 찾아서 프롬프트 만들기

● Hybrid Search가 일반적

- Semantic Search: 임베딩 기반 Dense Vector 비교
→ Customer Service/상담 작업
- Lexical Search: BM25/TF-IDF 키워드 인덱싱 기반 Sparse Vector 비교
→ 법령/조항/모델명 검색
- Hybrid Search: Rank Fusion 등의 방법 사용

● 최근에는 Context 길이를 늘리는 추세

- Top K 실패시 RAG 실패 → Larger K로 실패율 감소
- Long Context Model에는 많은 K를 넣기만 하면 될까?

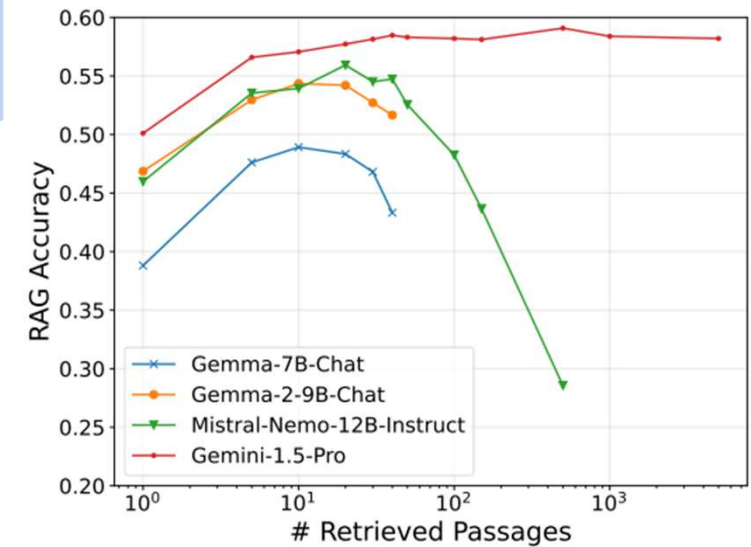


Ch.6 Retrieval-Augmented Generation

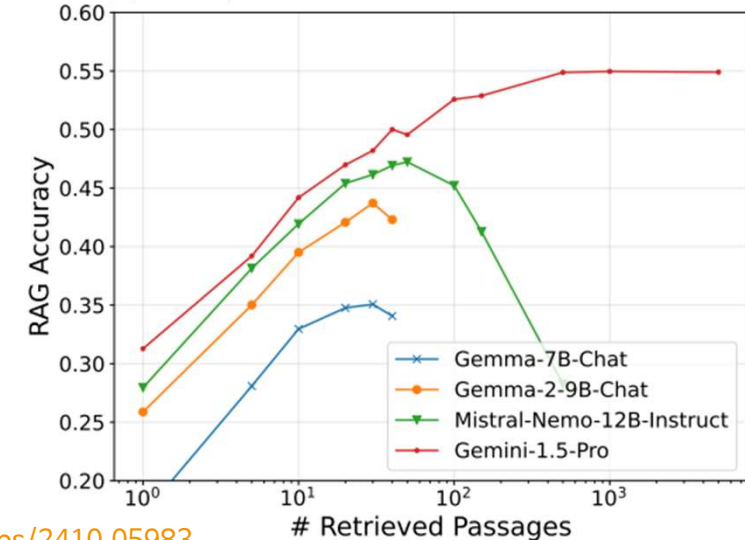
Context Optimization: 정보 가공하고 배열하기

- Top K에 정답이 포함되어 있어도, 전체 Context 길이가 증가하면 성능이 감소하는 현상
 - [ACL 2024] [Lost in the Middle: How Language Models Use Long Contexts](#)
 - 특히, 전체 Context의 중간 위치의 정보에 대해 감소하는 경향이 큼
- 좋은 답변을 생성하기 위해, Context의 추가 조정 필요
 - Reranker를 이용한 재정렬/필터링 기법

Source: Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG <https://arxiv.org/abs/2410.05983>



(a) RAG performance with e5 retriever



(b) RAG performance with BM25 retriever

Ch.6 Retrieval-Augmented Generation

Reranker: Retriever보다는 똑똑한 모델로

- Retrieval 과정 이후의 2차 작업에 해당

- 주로 BERT/T5 기반의 모듈을 학습하여 사용 (Input: Query/Chunk , Output: Score)
- 최근에는 LLM/ Reasoning LLM으로 Reranker를 구성하려는 시도가 유효
 - Rank-R1: Enhancing Reasoning in LLM-based Document Rerankers via Reinforcement Learning
<https://arxiv.org/abs/2503.06034>

Reordering: 중요 정보를 어디에?

- 중요 정보를 맨 앞/맨 뒤에 지그재그로 배치하거나, 관련 없는 정보를 필터링

- 특정 문제에 대해서는 원래 순서대로의 배치가 유용할 수도 있음
 - In Defense of RAG in the Era of Long-Context Language Models
<https://arxiv.org/abs/2409.01666>

Generation : LLM에게 강건한 능력 가르치기

● 다양한 RAG 시나리오에 대한 파인 튜닝

- RAFT: Adapting Language Model to Domain Specific RAG
- 텍스트의 내용을 인용하며 답변하는 방법 학습
- <https://arxiv.org/abs/2403.10131>
- Long-Context LLMs Meet RAG: Overcoming Challenges for Long Inputs in RAG
- Hard-Negative Context를 추가하여, 정답 관련 정보만 추출하고 답변하게 학습
- 답변하기 전, 문서 중 어떤 청크가 관련 있는지 추론하는 단계를 생성하도록 학습
- <https://arxiv.org/abs/2410.05983>

Ch.6 Retrieval-Augmented Generation

Advanced RAG (Evaluation)

● 수행된 RAG의 성능 평가하기

- Retrieval과 Generation에 대한 평가가 각각 이루어져야 함
 - 더 중요한 것은 Retrieval
- RAGAS: <https://docs.ragas.io/en/stable/>
 - RAG 평가 프레임워크 제공

- Generation 평가 지표

- 1) Faithfulness : 답변이 Context에 근거하여 추출되었는가?
- 2) Answer Relevancy : 답변이 질문과 관련되었는가?

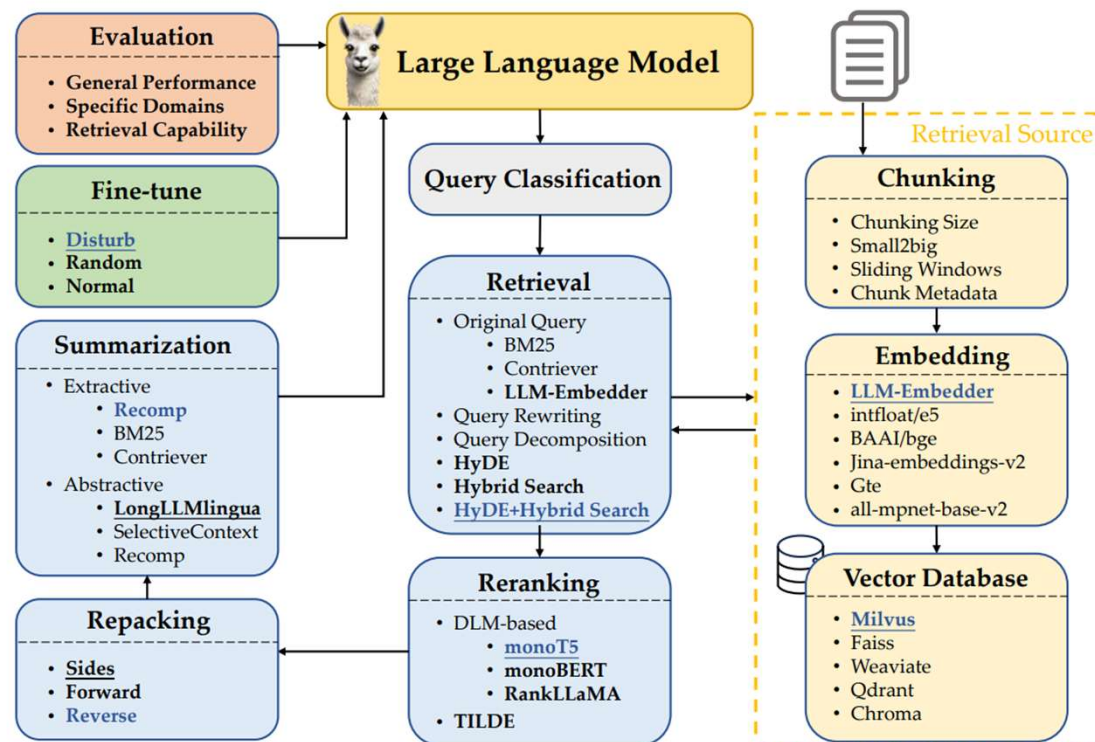
- Retrieval 평가 지표

- 1) Context Precision: Context가 Ground Truth와 연관이 있는가?
- 2) Context Recall: Context가 Ground Truth와 일치하는가?

Ch.6 Retrieval-Augmented Generation

RAG Module

● 최근의 RAG는 다양한 모듈을 조합하여 최적화하는 형태로 성능 개선



Source: Searching for Best Practices in Retrieval-Augmented Generation <https://arxiv.org/abs/2407.01219>

실습) LangChain의 Advanced RAG 만들기

AI Agent

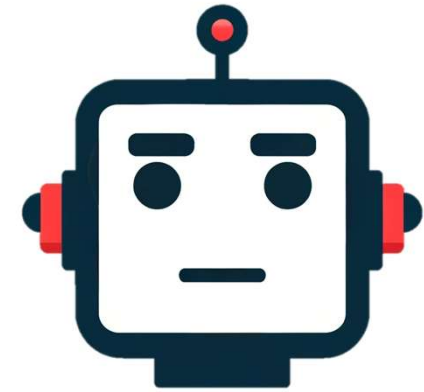
Large Language Model만 있으면 될까?

●LLM의 구조적 한계

- LLM이 이해하는 것은 ‘텍스트로 이루어진 세계’
- 구조적으로는 데이터 입력 → 데이터 출력에 한정

●LLM 구성 방식의 한계

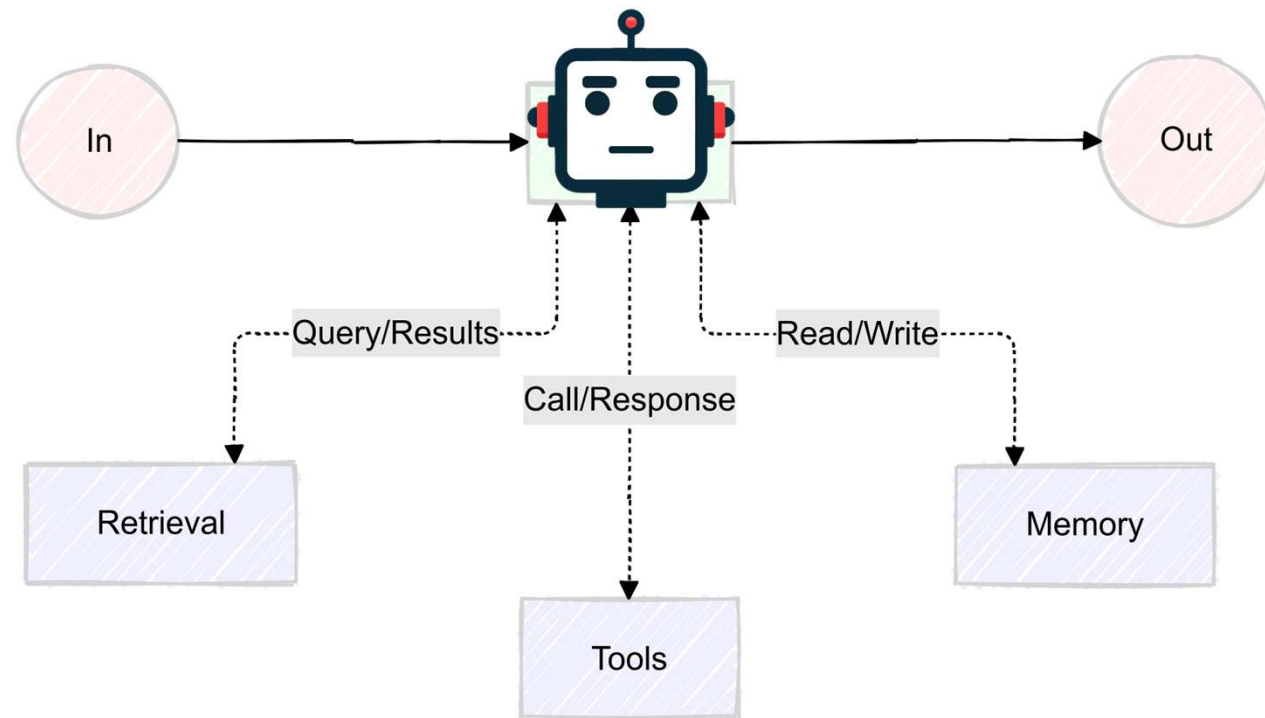
- Train Data 범위 내의 데이터만을 학습
- 학습하지 않은 지식에 대해서는 답변이 부정확할 수 있음



LLM의 한계 극복 3요소

● LLM의 외연을 확장하기 위한 방법 제시

- Retrieval : 검색 모듈 붙이기
- Tools : 외부 도구 연결하기
- Memory : 기존의 대화 기억하기

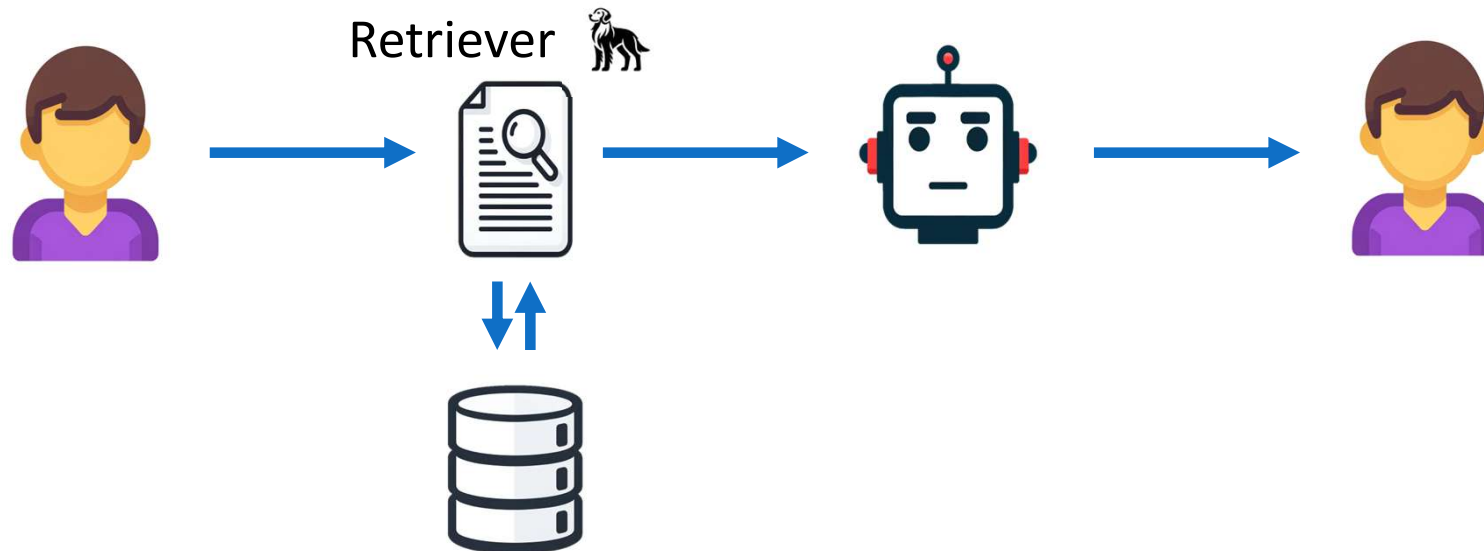


Source: <https://www.anthropic.com/research/building-effective-agents>

1) Retrieval Augmented Generation

● 기존의 LLM Workflow에 검색 툴을 추가

- Context의 내용을 활용하여 답변
- 할루시네이션을 완화하고, 모르는 질문에 대한 답변 가능



2) Tool: LLM에게 도구 들려주기

- LLM이 사용할 수 있는 외부 함수나 모듈

- 검색, 파이썬 코드 실행, API 호출 등
- 함수(Function)로 정의하기도 함

- 대표적 Tool

- SerpAPI, Tavily: 웹 검색 기능
- Code Interpreter: 파이썬 코드 실행 기능
- Slack, Github, Gmail, ...



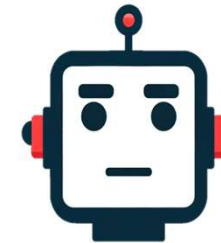
2) Tool: LLM에게 도구 들려주기

- LLM이 제어를 할 수 있을까?

- 제어는 할 수 없지만, 제어 명령은 내릴 수 있다.

- 프롬프트를 통해 도구의 설명을 포함하고, 파싱 가능한 출력을 생성하도록 한다.

User: 61번 서버 상태 알려줄래?



Call {Tool: Retrieve_Status,
Query: '61'}

2) Tool: LLM에게 도구 들려주기

● Tool이 탑재된 LLM의 판단

- Tool이 필요한 경우, Tool 실행을 요청
- 프로그램 상에서 Tool 요청을 파싱하여 실제 실행

● Tool 실행 결과: Tool Message

- 툴 실행 결과를 LLM에게 전달
- LLM은 답변 생성

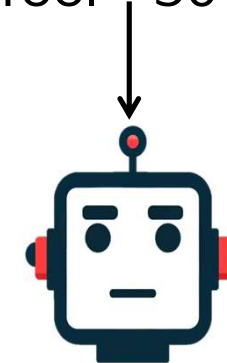
User: 61번 서버 상태 알려줄래?

+

LLM: Call {Tool: Retrieve_Status, Query: '61'}

+

Tool: "504"



504 Error로, 게이트웨이 타임아웃으로 추정됩니다.

Tool 탑재 예시: 뉴스 검색 증강 생성

●프롬프트 예시

- “너는 뉴스 검색을 할 수 있어.”
- “실행 방법은 get_news(query) 야.”
- “query는 문자열이고,
세 단어 이하로 작성해.”

```
from pydantic import BaseModel, Field

class get_news(BaseModel):
    """query를 이용하여 뉴스 검색"""
    query: str = Field(
        description="검색어(3단어 이하)"
    )

tools = [openai.pydantic_function_tool(get_news)]
```

Tool 탑재 예시: 뉴스 검색 증강 생성

● Tool의 Schema 형식

- 툴 사용법과 형식에 대한 정보
- 시스템 메시지에 포함

```
{
  'type': 'function',
  'function': {
    'name': 'get_news',
    'strict': True,
    'parameters': {
      'description': 'query를 이용하여 뉴스 검색',
      'properties': {'query': {'description': '검색어(3단어'
      'required': ['query'],
      'title': 'get_news',
      'type': 'object',
      'additionalProperties': False
    },
    'description': 'query를 이용하여 뉴스 검색'
  }
}
```

Tool 탑재 예시: 뉴스 검색 증강 생성

● 프롬프트 구성

- 유저 메시지: “오픈AI 뉴스 검색 결과 요약해줘.” → LLM에 입력
- LLM 출력:
<FUNCTION CALL> {function: get_news, argument: {query:오픈AI}} <END OF TEXT>
- Python 레벨: 파싱하여 get_news(query=오픈AI) 실행 → LLM에 입력
- LLM 출력: (오픈AI 검색 결과 요약)

● LLM 호출은 총 2번

- 더 복잡한 문제라면? Tool을 순차적으로 실행

대표적 Tool Calling Data: Glaive-function-calling-v2

● <https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2>

SYSTEM:

You are a helpful assistant with access to the following functions.
Use them if required -

```
{ "name": "get_news_headlines", "description": "Get the latest news headlines",  
  "parameters": { "type": "object", "properties": { "country": { "type":  
"string",  
"description": "The country for which to fetch news" } }, "required":  
[ "country" ] } }
```

└─ 함수 스키마

USER: Can you tell me the latest news headlines for the United States?

ASSISTANT: <functioncall> {"name": "get_news_headlines", "arguments":
'{"country": "United States"}' } <|endoftext|>

└─ STOP! (함수 실행 후 다시 입력)

대표적 Tool Calling Data: Glaive-function-calling-v2

● <https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2>

USER: Can you tell me the latest news headlines for the United States?

ASSISTANT: <functioncall> {"name": "get_news_headlines", "arguments":
'{"country": "United States"}'} <|endoftext|> **STOP! (함수 실행 후 다시 입력)**

FUNCTION RESPONSE: {"headlines": ["Biden announces new vaccine mandates",
"Hurricane Ida devastates Louisiana", "Apple unveils new iPhone", "NASA's
Perseverance rover collects first Mars rock sample"]} **함수 실행 결과 포맷**

ASSISTANT: Here are the latest news headlines for the United States: 1. Biden
announces new vaccine mandates 2. Hurricane Ida devastates Louisiana 3. Apple
unveils new iPhone 4. NASA's Perseverance rover collects first Mars rock sample
<|endoftext|>

Tool Calling LLM: 기대 효과

- LLM의 텍스트 출력을 실제 행동으로 연결 가능
 - 상황을 판단하고, 지시하고, 제어할 수 있는 능력
 - 단순 챗봇이 아니라, LLM의 출력을 분석하여
지능형 판단을 수행하는 컨트롤 타워의 역할로 활용할 수 있다
- Ex) Robotics, 업무 자동화, 스케줄링, ...

Tool Calling LLM: 나쁜 예시

● Realtime API와 Tool Calling의 결합

- https://www.youtube.com/watch?v=XYVrbFYHjOY&ab_channel=TodayNews



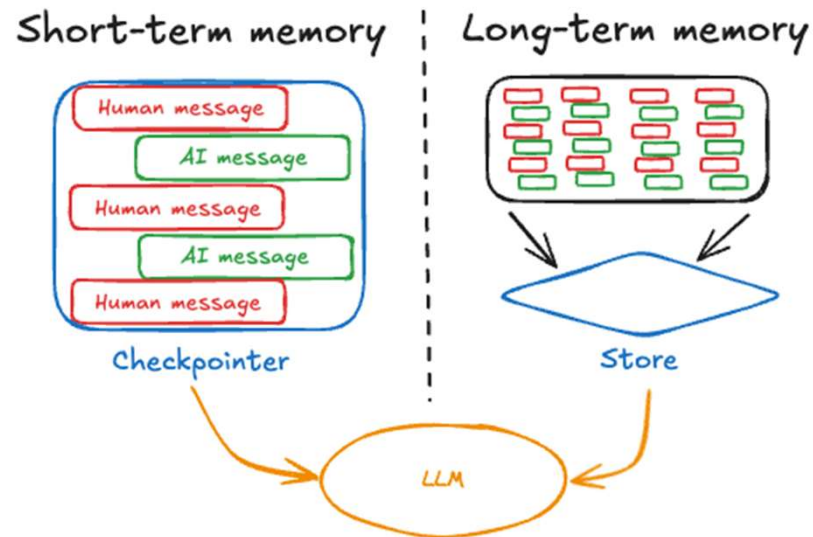
오픈AI, '챗GPT'로 자동 소총 만들던 개발자 차단

✎ 박찬 기자 ⌚ 입력 2025.01.13 18:00 ⌚ 수정 2025.01.13 19:19 💬 댓글 1 ❤️ 좋아요 0



3) Memory: 서비스화를 위한 필수 요소

- 세션 단위의 짧은 메모리 + Store 형식의 긴 메모리에 대한 이해 필요



Source: <https://langchain-ai.github.io/langgraph/concepts/img/memory/short-vs-long.png>

3요소 + 또 하나의 필요한 능력

- Computer Agent 예시에서 확인했듯, 복잡한 작업은 Tool의 연속적인 사용이 요구됨
 - Context가 복잡해지기 때문에, LLM이 전체 과정에서 목표에 대한 집중이 필요함
- 실제 상황에서는 모든 중간 스텝이 성공하는 것은 아님
 - 중간에 오류가 발생할 경우, 이를 LLM이 수정하거나 우회할 수 있어야 함

다단 Tool Calling 예시

● 예시) 계산기와 인터넷 검색

- “디카프리오의 탄생년도를 검색한 뒤, 각 숫자를 순서대로 곱해서 알려줘.”

● 프롬프트: 1. 유저) “디카프리오의 탄생년도를 검색해서, 각 숫자를 곱해봐.”

2. LLM) <FUNCTION CALL> (검색) <END OF TEXT>

3. Tool) “레오나르도 디카프리오 1974년 ...”

4. LLM) <FUNCTION CALL> (곱하기: 1*9) <END OF TEXT>

5. Tool) “9”

6. LLM) <FUNCTION CALL> (곱하기: 9*7) <END OF TEXT>

7. Tool) “63”

8. LLM) <FUNCTION CALL> (곱하기: 63*4) <END OF TEXT>

9. Tool) “252”

10. LLM) “디카프리오의 생년월일을 모두 곱한 결과는 252입니다!”

다단 Tool Calling 예시

● 예시) 계산기와 인터넷 검색

- “디카프리오의 탄생년도를 검색한 뒤, 각 숫자를 순서대로 곱해서 알려줘.”

● 프롬프트: 1. 유저) “디카프리오의 탄생년도를 검색해서, 각 숫자를 곱해봐.”

2. LLM) <FUNCTION CALL> (검색) <END OF TEXT>

3. Tool) “레오나르도 디카프리오 1974년 ...”

4. LLM) <FUNCTION CALL> (곱하기: 1*9) <END OF TEXT>

5. Tool) “9”

6. LLM) <FUNCTION CALL> (곱하기: 9*7) <END OF TEXT>

7. Tool) “63”

8. LLM) <FUNCTION CALL> (곱하기: 63*4) <END OF TEXT>

9. Tool) “252”

10. LLM) “디카프리오의 생년월일을 모두 곱한 결과는 252입니다!”

복잡한 Tool Calling 예시) 계산기와 인터넷 (2)

- 프롬프트: 1. 시스템) 너는 인터넷 검색과 두 수 곱하기를 할 수 있어. 그리고 ~~~
2. 유저) “디카프리오의 탄생년도를 검색해서, 각 숫자를 곱해봐.”
3. LLM) <FUNCTION CALL> (검색) <END OF TEXT>
4. Tool) “레오나르도 디카프ριο는 1974년 ...”

- 실제로 발생하기 쉬운 전개

- 5. LLM) <FUNCTION CALL> (곱하기: $1*9$) <END OF TEXT>
 - 6. Tool) “9”
 - 7. LLM) <FUNCTION CALL> (곱하기: $1*9$) <END OF TEXT>
 - 8. Tool) “9”
 - 9. LLM) <FUNCTION CALL> (곱하기: $1*9$) <END OF TEXT>
 - 10. Tool) “9”
- (이하 무한 반복)

!?

복잡한 Tool Calling 예시) 계산기와 인터넷 (3)

- 1*9는 긴 작업을 수행하는 중간 과정이므로, LLM은 다음 단계로 넘어가야 함
- 그러나, Tool 요청-실행의 나열만으로는 집중력 있는 추론이 어려움
 - 툴과 툴 사이의 연결을 강하게 이해시키는 방법이 필요
- 정답은 Prompt에 있다?
 - 중간중간의 상황을 파악하게 하면 어떨까?

복잡한 Tool Calling 예시) 계산기와 인터넷 (4)

- Solution: 중간 Thought 출력하기

4. LLM) 디카프리오의 생년은 1984년이다. 이제 1, 9, 8, 4를 차례대로 곱하자.

5. LLM) <FUNCTION CALL> (곱하기: $1*9$) <END OF TEXT>

6. Tool) “9”

7. LLM) $1*9$ 의 값은 9다. 이제 9와 7을 곱하자.

8. LLM) <FUNCTION CALL> (곱하기: $9*7$) <END OF TEXT>

9. Tool) “63”

10. LLM) $9*7$ 의 값은 63이다. 이제 63과 4를 곱하자.

11. LLM) <FUNCTION CALL> (곱하기: $63*4$) <END OF TEXT>

12. Tool) “252”

13. LLM) 이제 답을 알겠다.

14. LLM) <FUNCTION CALL> (최종답변: 결과는 252입니다!) <END OF TEXT>

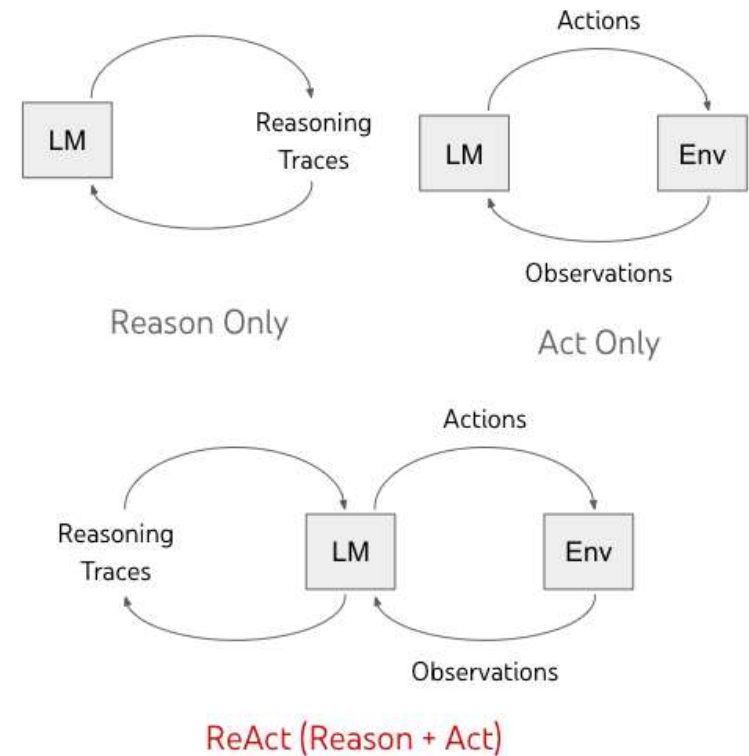
ReAct Agent (2022) : Reasoning and Acting

- ReAct: Synergizing Reasoning and Acting in Language Models

- <https://arxiv.org/abs/2210.03629>

- Action을 수행하고, 결과를 반영하는 프롬프트 방식

- 생각-행동-관찰을 통해 다음 행동을 계획하고 실행



<https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/>

ReAct Agent (2022) : Reasoning and Acting

● 한국어 ReAct 프롬프트 예시

프롬프트: ‘강아지와 고양이 중 누가 더 평균적으로 똑똑한가요? 뇌세포의 수를 비교하세요.’

응답:

생각) 두 동물의 뇌세포 수를 비교하는 것이 중요하다.

행동) 검색 ‘Dogs and Cats Brain neurons comparison’ **STOP!** (함수 실행 후 다시 입력)

관찰)

URL: <https://dogs-cats-brains-neurons-study-vanderbilt/>

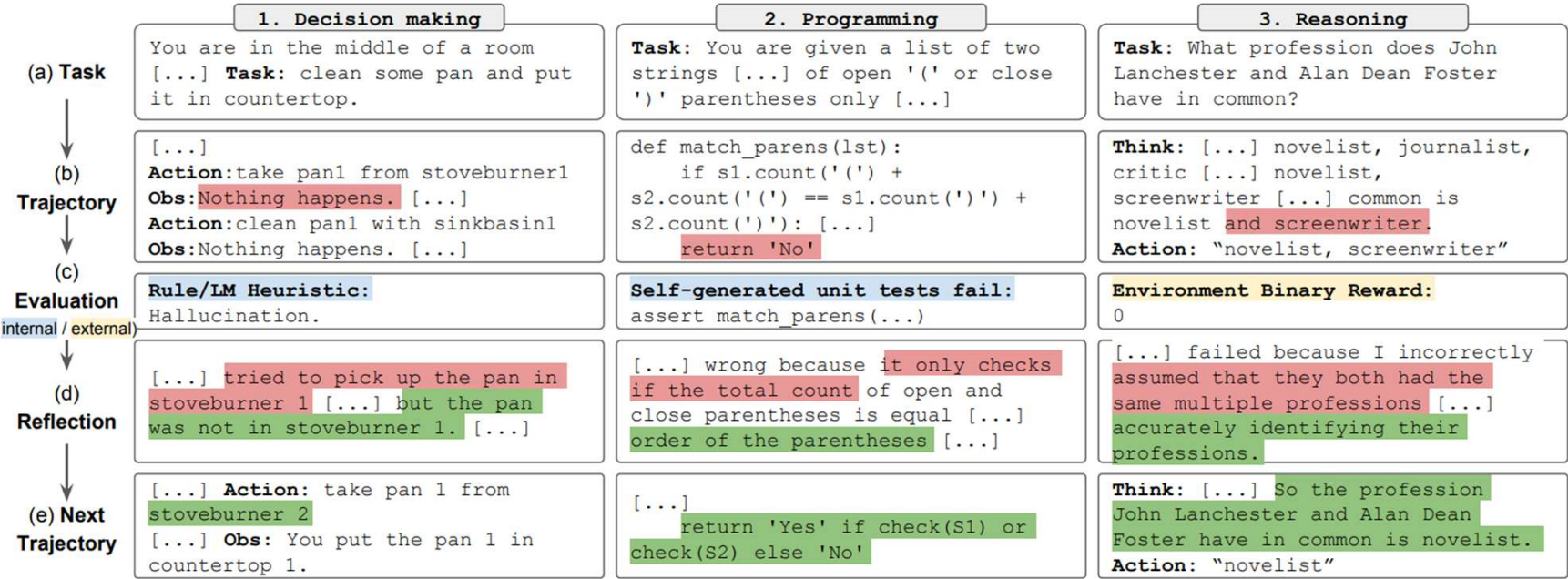
cats have 250 million cortical neurons, while dogs have about 530 million. (...)’ **LLM에 입력되는 부분**

생각) 250 million보다 530 million이 더 크니까, 강아지가 고양이보다 더 똑똑한 것 같다.

행동) 최종답변 ‘강아지가 평균적으로 더 많은 뇌세포 수를 가지고 있으므로 고양이보다 똑똑합니다.’

Reflexion (2023) : 실패한 이유를 저장하여 다시 실행할 때 활용

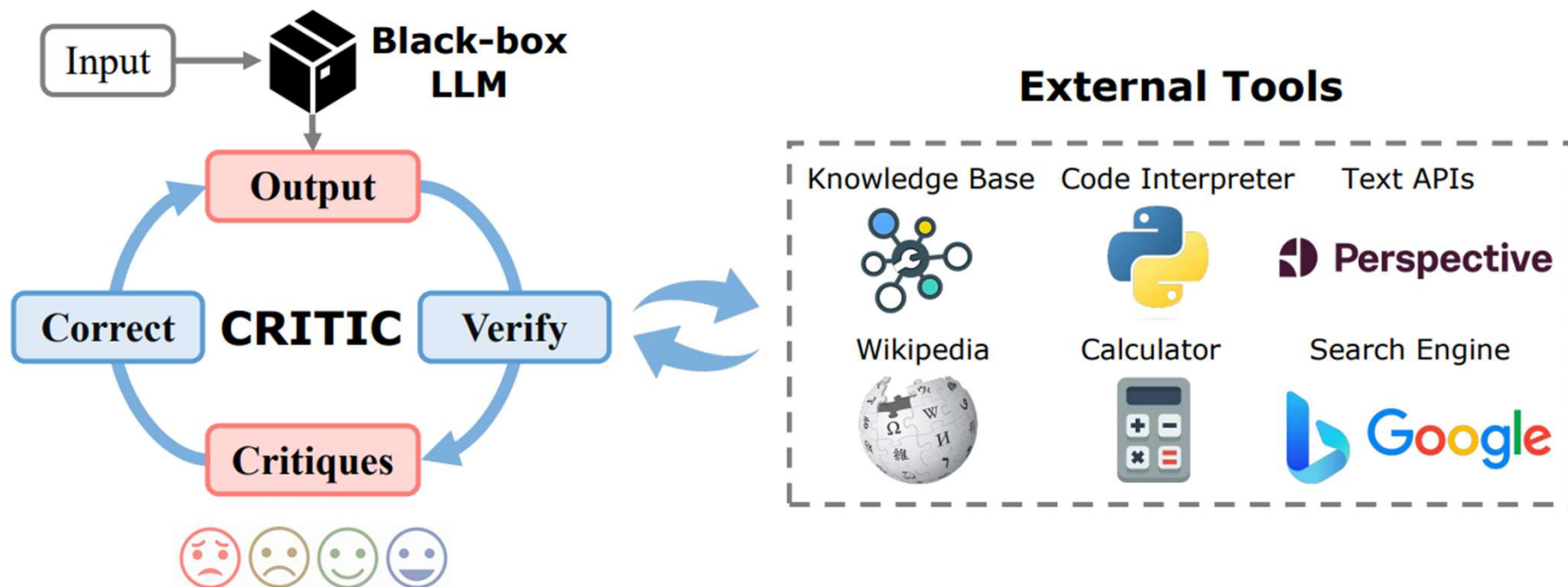
● Reflexion: Language Agents with Verbal Reinforcement Learning <https://arxiv.org/abs/2303.11366>



CRITIC (2023) : 실행 결과를 비판하고 평가하는 모듈 연결

● CRITIC: LARGE LANGUAGE MODELS CAN SELF-CORRECT WITH TOOL-INTERACTIVE CRITIQUING

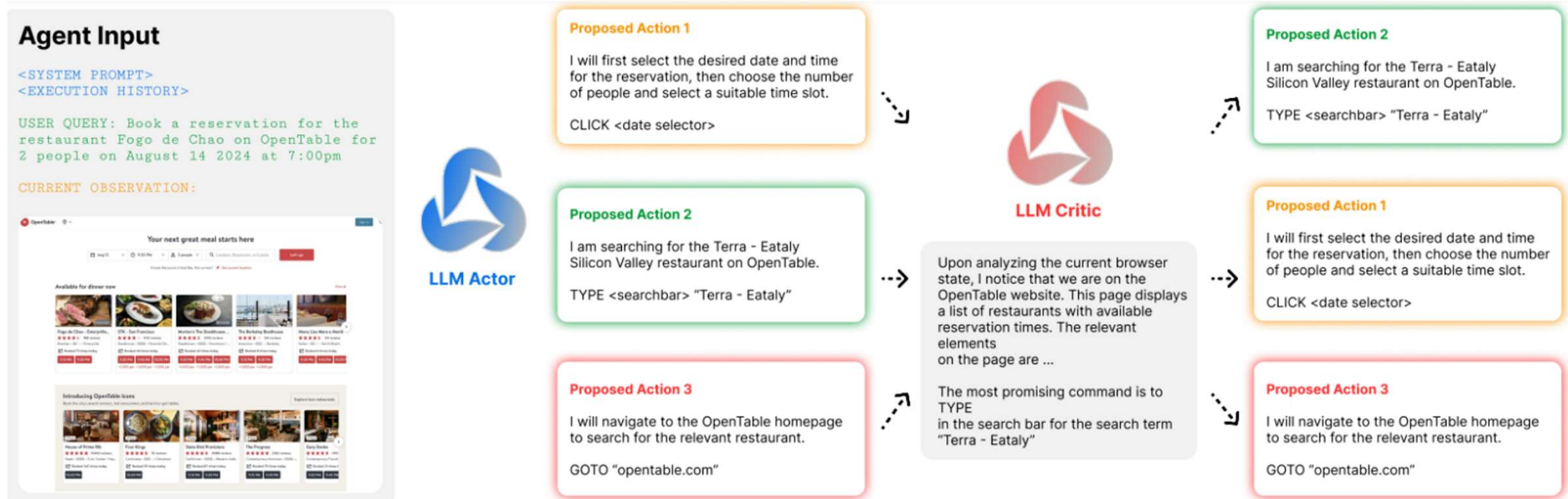
<https://arxiv.org/abs/2305.11738>



Ch.7 Agent

Agent Q (2024) : 실행 후보 중 적합한 경로를 선택하는 CRITIC LLM

● Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents <https://arxiv.org/abs/2408.07199>



Agent에서 Agentic Work를 분리시키자

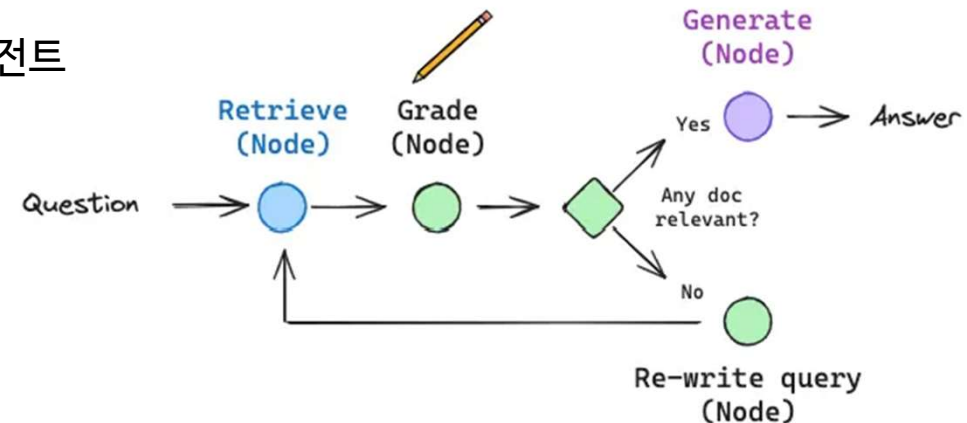
● LLM이 모든 경로를 매번 판단하는 것은 비효율적임

- 정해진 Flow 안에 LLM을 포함하게 하는 프레임워크의 필요성
- 완전 자율화 Agent가 아닌, Agentic Work를 고려한 강건한 에이전트

● 대표적 Agentic Work 라이브러리: LangGraph

- <https://github.com/langchain-ai/langgraph>
- 단순 일방통행이 아닌, 반복이나 Divide & Conquer 등의 구성도 가능

Self-Reflective RAG with LangGraph
(Reflection and self-correction)

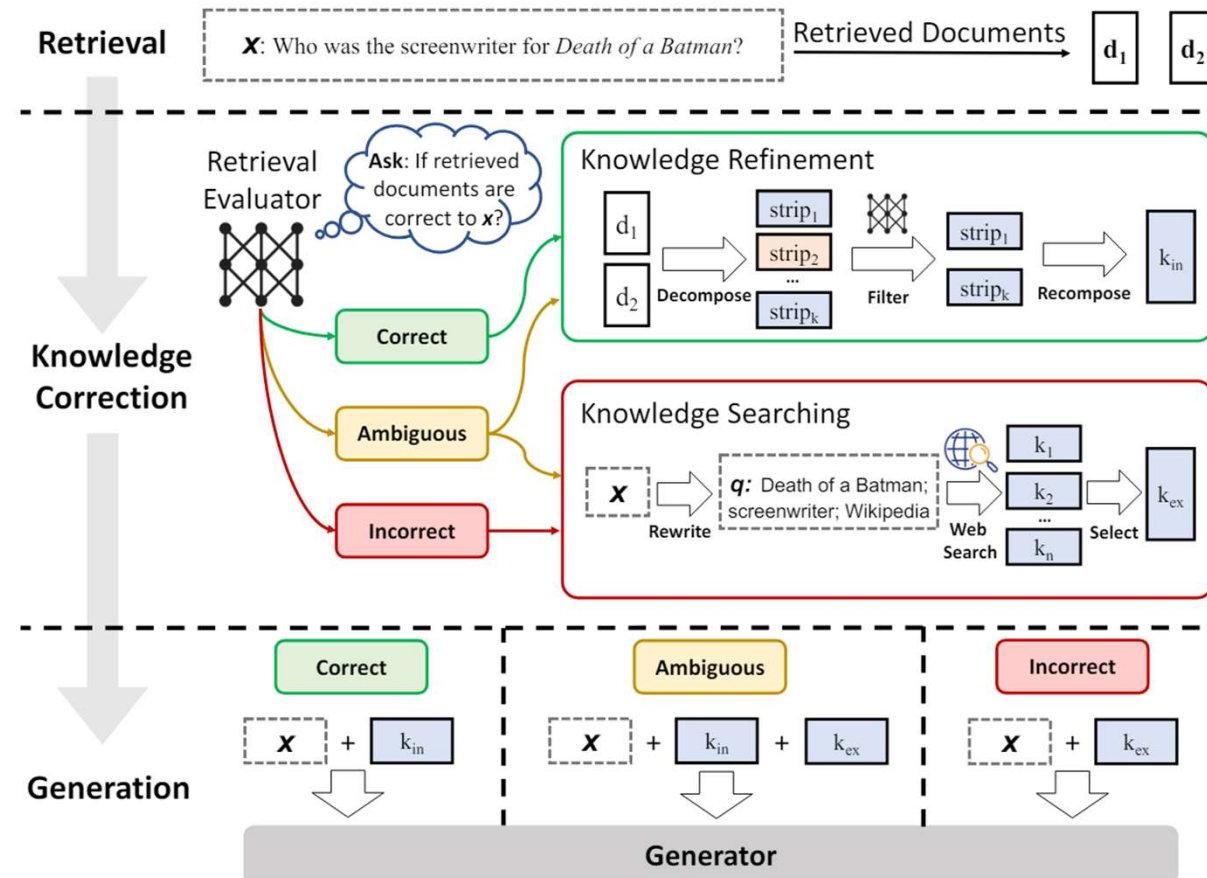


실습) LangGraph의 Agentic Workflow 만들기

Ch.7 Agent

Agentic RAG: Corrective RAG

- RAG의 검색 결과를 판단하고,
그 결과가 정확하지 않으면 웹 검색을 활용



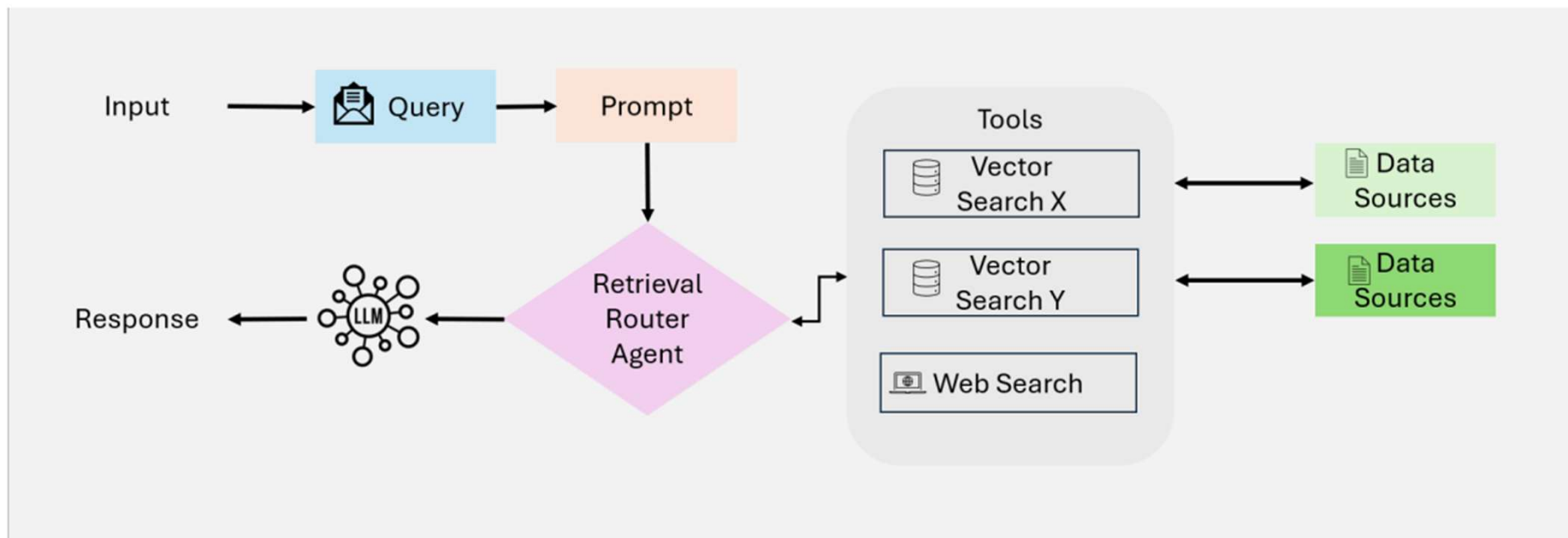
Source: <https://arxiv.org/abs/2401.15884>

Memory와 Human-in-the-loop

- 과거 내용을 바탕으로 재개하거나, 필요한 정보를 바탕으로 전처리
- 에이전트의 작업에서 중요한 부분이나 수정이 필요한 부분을 인간이 개입
- LangGraph는 이런 측면에서 매우 효율적인 구성을 지원

예시) Multi-Source RAG Agent

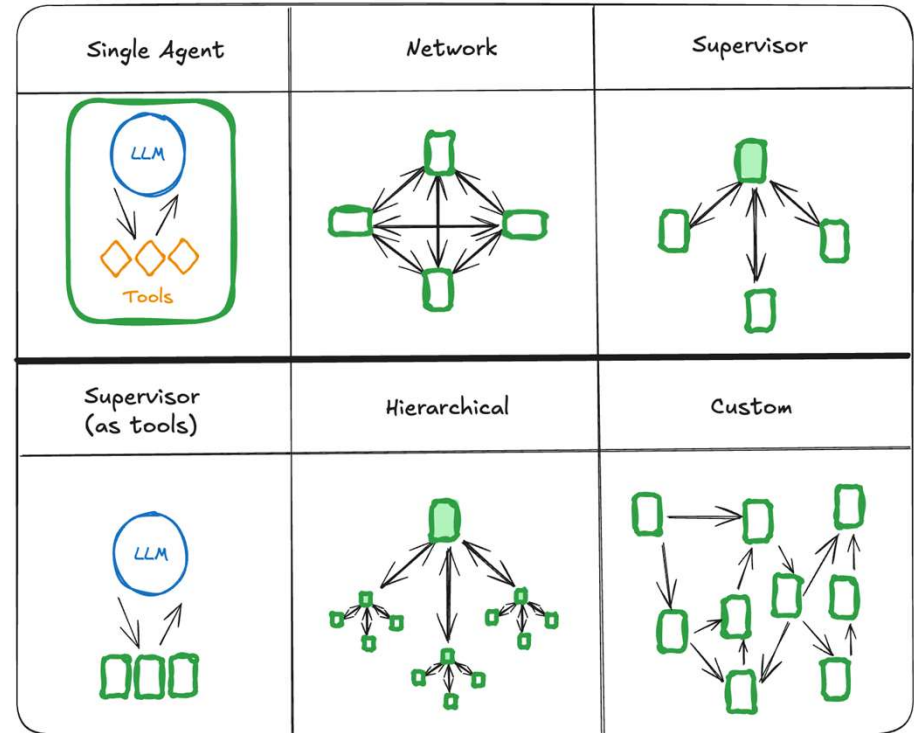
- Tool 형태로 각각의 검색 연결



Ch.7 Agent

멀티 에이전트: 에이전트들의 상호 작용

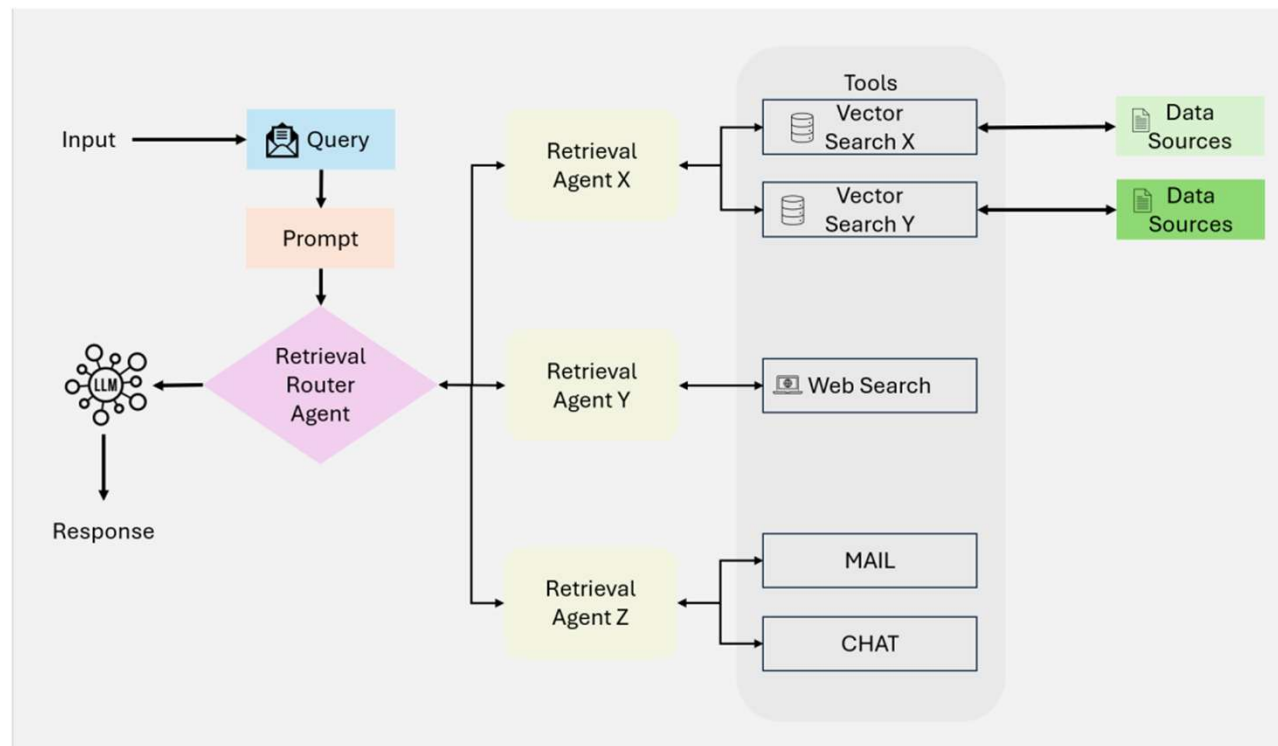
- 1) Supervisor / Worker
 - 여러 에이전트를 병렬로 실행하고 중앙 제어
- 2) Network 구조
 - 정해진 루트보다 그때그때 구성이 필요할 때
- 3) Hierarchical 구조
 - 하나의 Supervisor로 구성하기 복잡할 때



Source: https://langchain-ai.github.io/langgraph/concepts/img/multi_agent/architectures.png

Multi-Agent Retrieval 구조

- 작업을 세분화하여 개별 Agent가 처리
 - 결과만 받아 Agent가 최종 결과에 활용



Multi Agentic RAG Use Case

Tool: Arxiv 검색, 뉴스 검색

입력: “Agentic RAG의 최근 동향을 알려줘.”

→ Arxiv 검색 에이전트: 충분한 정보가 모일 때까지 검색 조건을 변경하며 검색

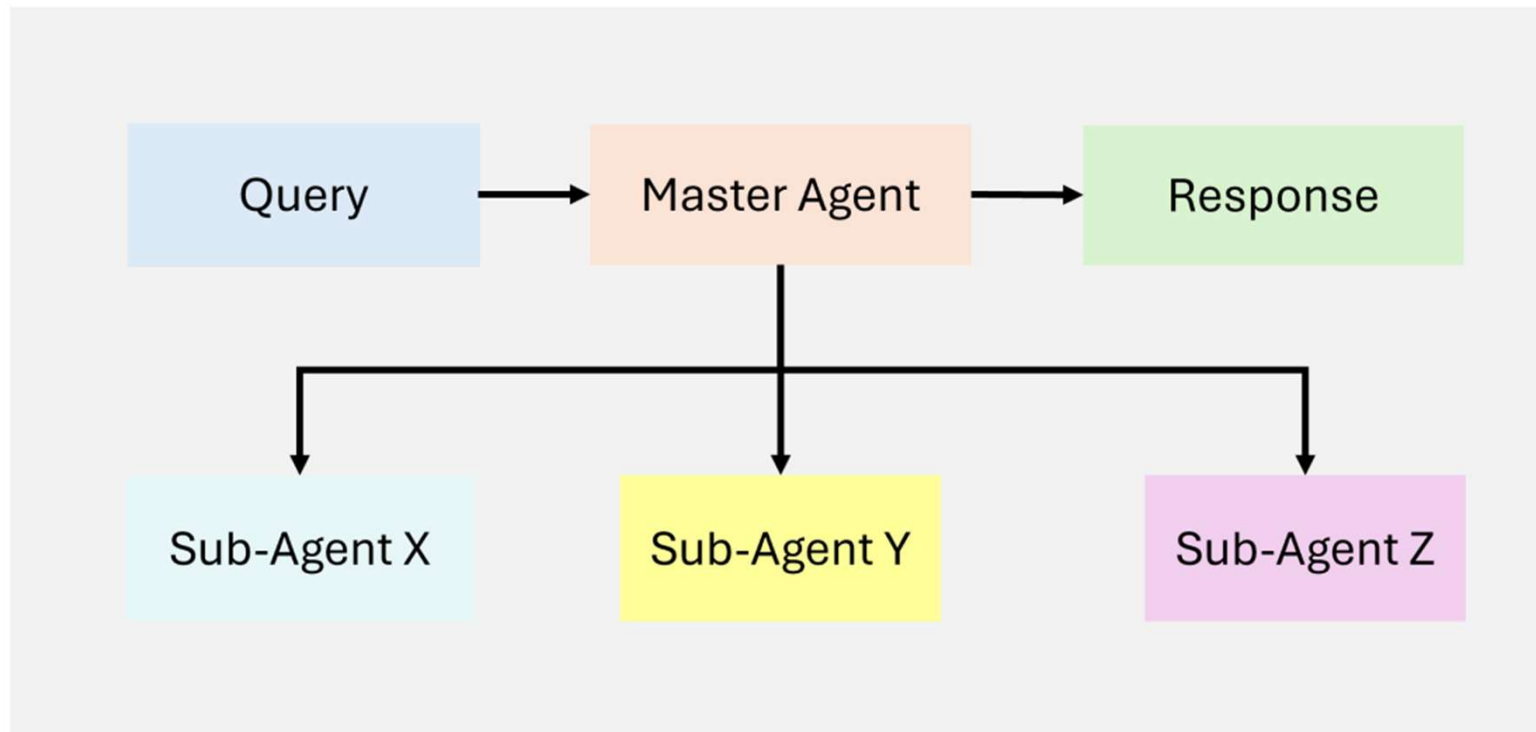
→ 뉴스 검색 에이전트: 충분한 정보가 모일 때까지 검색 조건을 변경하며 검색

→ 최종 정보 취합

→ 과정은 생략하고 정보만 모아서 LLM에 전달하고 답변 생성

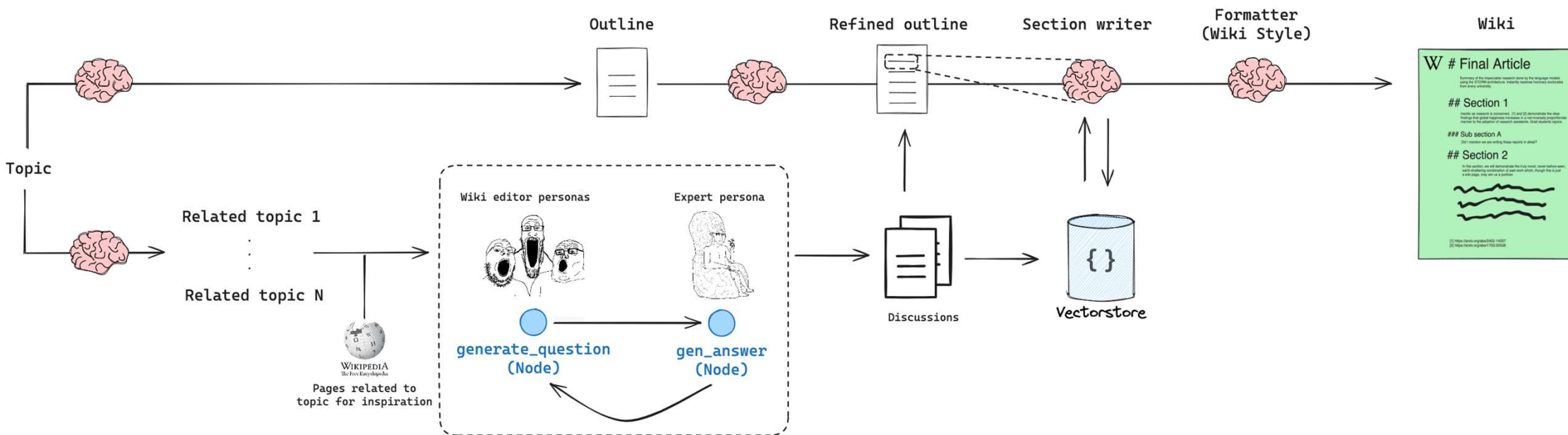
Hierarchical Agentic RAG

- 계층 구조를 통한 우선순위 처리 가능

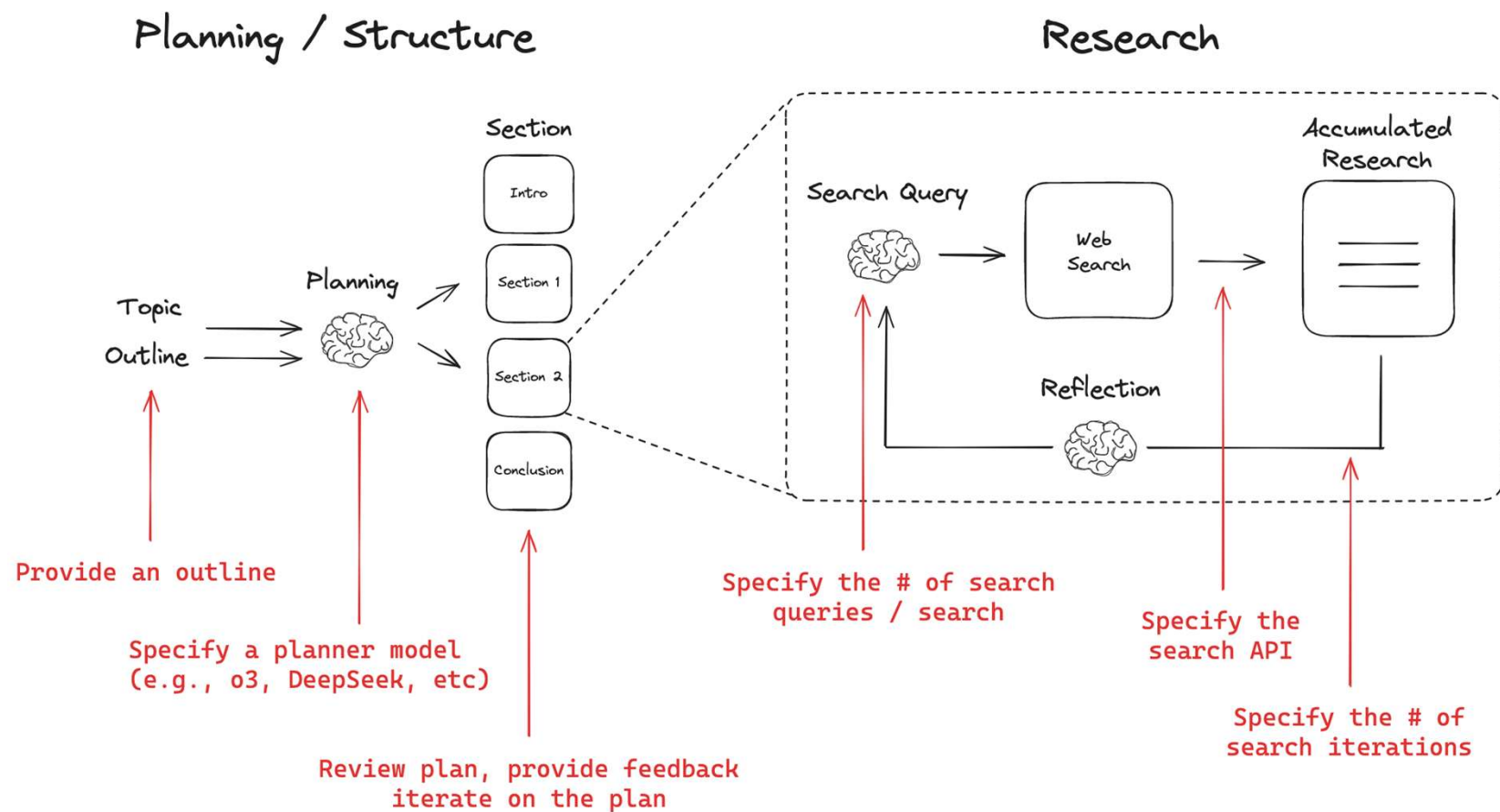


Stanford STORM (<https://arxiv.org/abs/2402.14207>)

Synthesis of Topic Outlines through Retrieval and Multi-perspective Question Asking



LangChain² | Open Deep Research (https://github.com/langchain-ai/open_deep_research)



Open-Weight LLMs

변형호

오픈 모델 = 오픈 웨이트 모델

- 구조와 파라미터가 전부 공개된 모델

- 학습 데이터는 비공개

- 오픈소스 모델?

- 재현이 어려우므로 완전한 오픈 소스로 보기는 어려움
- **Open-Weight LLM**, Open-Sourced LLM , Open LLM을 혼용

상업적 이용이 가능한가?

● LLM의 다양한 라이선스

- Qwen 2.5: 3B/72B 모델에만 상업적 이용 제약 존재
 - 3B: 라이선스 요청 필요, 72B: 월간 활성 사용자(MAU) 1억 명 이상인 경우 라이선스 필요
- Llama 3 시리즈: MAU 7억 명 이상인 경우 라이선스 필요
- Cohere Aya-Expanse, LG-AI exaone : NC License

● 모델을 파인튜닝하는 경우

- 해당 모델에서 파생되었음을 꼭 명시해야 하는 조건 (Qwen, Llama)

● 모델을 이용해 다른 모델을 학습시키는 경우

- Llama는 금지, Qwen은 허용하나 명시 필요

Open 모델 구성의 이점

● Privacy

- 민감한 데이터를 자체 인프라에서 처리
- 대안) Azure OpenAI 등의 서비스

● Pricing

- API 호출 비용 절약, 예측 가능한 고정비용

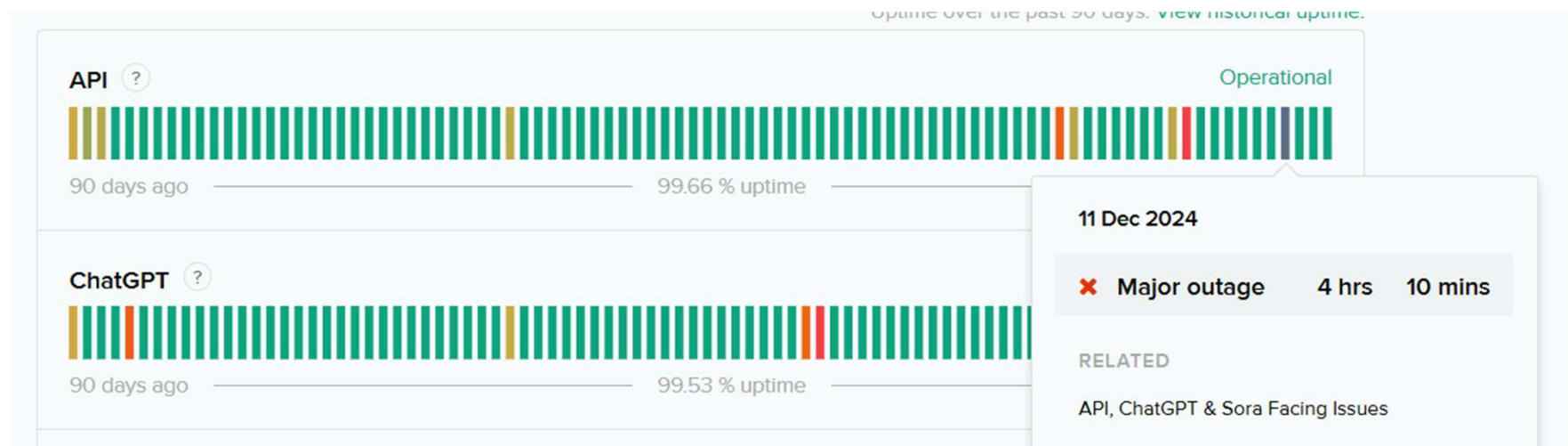
● Fine Tuning

- 특수한 목적에 맞춘 추가 학습, 맞춤형 모델 구성

Open 모델 구성의 이점

● Safety

- Sora 출시 직후, 4시간 가량 정전된 OpenAI 사례
- <https://status.openai.com/>



Ch.8 Open-Weight LLM

왜 Open-Weight LLM을 안 사용하는가?

- Closed Model의 지속적인 업데이트
 - 파인 튜닝의 경우, 모델이 바뀌면 재학습 필요

Announcements

NEW Claude 3.5 Haiku and a new Claude 3.5 Sonnet 2024년 10월 22일

Our updated version of Claude 3.5 Sonnet is now state-of-the-art for real-world software engineering tasks, agentic capabilities, and computer use in public beta.

[Read more](#)

MODEL	CONTEXT WINDOW	MAX OUTPUT TOKENS	KNOWLEDGE CUTOFF
gpt-4o Our high-intelligence flagship model for complex, multi-step tasks. GPT-4o is cheaper and faster than GPT-4 Turbo. Currently points to gpt-4o-2024-08-06.	128,000 tokens	16,384 tokens	Oct 2023
gpt-4o-2024-11-20 Latest gpt-4o snapshot from November 20th, 2024.	128,000 tokens	16,384 tokens	Oct 2023
gpt-4o-2024-08-06 First snapshot that supports Structured Outputs . gpt-4o currently points to this version.	128,000 tokens	16,384 tokens	Oct 2023

제미나이 2.0 출시: 에이전트 시대를 위한
구글의 새로운 AI 모델

왜 Open-Weight LLM을 안 사용하는가?

- 본격적 서비스 인프라 구축을 위한 비용
 - 동시 접속/사용량/파인 튜닝 등의 고려사항

✓ 인기상품순 | 낮은가격순 | 높은가격순 | 신상품순 | 상품평 많은순

☐ 쿠팡와우할인 ? ☐ 배송비 포함 40개 ▼



NVIDIA A100 HBM2e 80GB PCIe

A100 / 스트림 프로세서: 6912개 / PCIe4.0 / HBM2 / 지원기능: 멀티VGA 지원 / 사용전력: 300W / FP16 Tensor Core: 312 TFLOPs / FP32: 19.5 TFLOPs

등록월 2023.07. | 상품의견 4건 | ♥ 관심

주요부품 > 그래픽카드(VGA)

vs검색 담고 스펙 비교하기 ✕

27,500,000원 + 6월 □



NVIDIA H100 HBM3 94GB NVL

H100 / 스트림 프로세서: 16896개 / PCIe5.0 / HBM3 / 지원기능: 멀티VGA 지원 / 사용전력: 400W

등록월 2024.05. | 상품의견 4건 | ♥ 관심

주요부품 > 그래픽카드(VGA)

vs검색 담고 스펙 비교하기 ✕

44,500,000원 +

Ch.8 Open-Weight LLM

왜 Open-Weight LLM을 안 사용하는가?

● 한국어 지원 미비

- Open-Weight LLM은 다국어 데이터의 비중이 적은 편
- Qwen, Gemma와 같은 다국어 모델에 국한된 개발

● 알려진 모델의 한국어 파인튜닝 버전을 활용 가능

44	chatgpt-4o-latest	api	0.8176
43	gpt-4o-2024-05-13	api	0.8153
20	claude-3-5-sonnet-20241022	api	0.8109
12	Qwen2.5-72B-Instruct	30B≤	0.7962
22	claude-3-opus-20240229	api	0.7902
41	gpt-4-turbo-2024-04-09	api	0.7878
15	Qwen2.5-32B-Instruct	30B≤	0.7853
7	Llama-3.1-405B-Instruct-FP8	30B≤	0.7699
39	gpt-4o-mini-2024-07-18	api	0.7654
14	Qwen2-72B-Instruct	30B≤	0.7631
40	gpt-4-0613	api	0.7575
2	EXAONE-3.5-32B-Instruct	30B≤	0.752
32	Qwen2.5-14B-Instruct	10B≤ <30B	0.7362
18	Llama-3.1-70B-Instruct	30B≤	0.7356
23	claude-3-sonnet-20240229	api	0.7354
9	gemini-1.5-pro-001	api	0.7337
5	solar-pro-241126	10B≤ <30B	0.7274
26	gemma-2-9b-it	<10B	0.7225
19	Meta-Llama-3-70B-Instruct	30B≤	0.7202

Source: 호랑이 리더보드

Llama: Meta의 오픈 LLM

- 2024.4.19 Llama 3

- 8B/ 70B

- 2024.7.23 Llama 3.1

- 8B/ 70B/ 405B

- 2024.9.25 Llama 3.2

- 1B/ 3B, Vision (11B/ 90B: 3.1에 Vision만 추가)

- 2024.12.8 Llama 3.3 70B

- 70B 성능이 3.1 405B에 육박

- 2025. 4 Llama 4

- Context Window 10M, 성능 조작 논란



Microsoft Phi

- Phi 시리즈의 핵심 아이디어: 고효율의 합성 데이터를 사용하자

- Textbooks are All You Need <https://arxiv.org/abs/2306.11644>

- 고품질의 데이터 정제와 합성 데이터를 활용해 적은 양의 데이터로도 뛰어난 학습 효과

- Phi-1 : 1.3B
- Phi-2 : 2.7B
- Phi-3 : 3.8B (Mini), 7B (Small), 14B(Medium), 4.15B(Vision)
- Phi-3.5 Mini, MoE, Vision 출시 (24.8)
- Phi-4 (24.12)
- Phi-4-Reasoning (25.4)

- 뛰어난 성능, 한국어 지원은 미흡

- Phi-4에서는 많이 개선되었으나, 토큰 효율이 낮음

Alibaba Qwen Series

● Multilingual Model

- 여러 언어 처리를 위한 높은 압축률
- sLLM부터 Big LLM까지 좋은 성능

Models	Layers	Heads (Q / KV)	Tie Embedding	Context / Generation Length	License
0.5B	24	14 / 2	Yes	32K / 8K	Apache 2.0
1.5B	28	12 / 2	Yes	32K / 8K	Apache 2.0
3B	36	16 / 2	Yes	32K / 8K	Qwen Research
7B	28	28 / 4	No	128K / 8K	Apache 2.0
14B	48	40 / 8	No	128K / 8K	Apache 2.0
32B	64	40 / 8	No	128K / 8K	Apache 2.0
72B	80	64 / 8	No	128K / 8K	Qwen

● Qwen 2.5 (24.9)

- Qwen2.5: 0.5B, 1.5B, 3B, 7B, 14B, 32B, 72B
- Qwen2.5-Coder: 1.5B, 7B, 32B
- Qwen2.5-Math: 1.5B, 7B, 72B

Models	Layers	Heads (Q / KV)	Tie Embedding	Context Length
Qwen3-0.6B	28	16 / 8	Yes	32K
Qwen3-1.7B	28	16 / 8	Yes	32K
Qwen3-4B	36	32 / 8	Yes	32K
Qwen3-8B	36	32 / 8	No	128K
Qwen3-14B	40	40 / 8	No	128K
Qwen3-32B	64	64 / 8	No	128K

● Qwen 3 (25.4)

Mistral Series

● 프랑스 AI 기업 Mistral AI의 오픈소스 모델

- Mistral 7B : Llama 2 13B를 넘는 성능
- Sliding Window Attention을 활용
 - 전체 문맥이 아닌 주변 문맥에 집중
- Mixtral: Mixture of Expert(MoE) 방법을 활용한
 - 8*7B, 8*22B 모델 (2024년 4월까지 파운데이션 모델 1위)
- Mistral Large (24.2), Mistral Large 2 (24.7)
 - Large 2: 123B로 Llama 3.1 405B 수준의 성능, 한국어 지원
- Mathstral, Codestral Mamba 등의 모델도 공개



Ch.8 Open-Weight LLM

Gemma Series

● Gemma 2 (2024.6) : 2B, 9B, 27B

- 어텐션 방식의 개선, 대규모 데이터 사용
- Knowledge Distillation을 이용한 작은 모델 개선 방법
 - 7B to 2B, 9B 모델

● Paligemma: 3B 사이즈의 비전 모델

- SigLIP 인코더 기반의 멀티모달 모델

● Gemma 3 (25.3): 1B, 4B, 12B, 27B

- SigLip 기반의 멀티모달 모델 (1B 제외)
- 높은 범용 성능으로 현재 가장 높은 인기



국내 모델 (1)

- Kakao Kanana 1.5 (25.4)
 - 2.1B / 8B
 - Base/Instruct 모두 공개

- Naver HyperclovaX Seed
 - 0.5B / 1.5B / 3B(MM)



HyperCLOVA X SEED 3B

국내 모델 (2)

● LG-AI Exaone 3.5 : 2.4/7.8/32B

- 24년 12월 9일 공개
- 유사 사이즈 모델 대비 뛰어난 성능
- 타 모델에 비해 효율적인 한국어 토크나이징
 - 단어당 평균 2.46 토크으로 높은 압축 비율
- 2단계 학습
 - 초기 학습 후, 도메인 특화 데이터 2조 토크 추가 학습
 - 데이터를 재사용하여 Catastrophic Forgetting 방지

● Exaone-Deep (25.3)

- Reasoning 능력 탑재



국내 모델 (3)

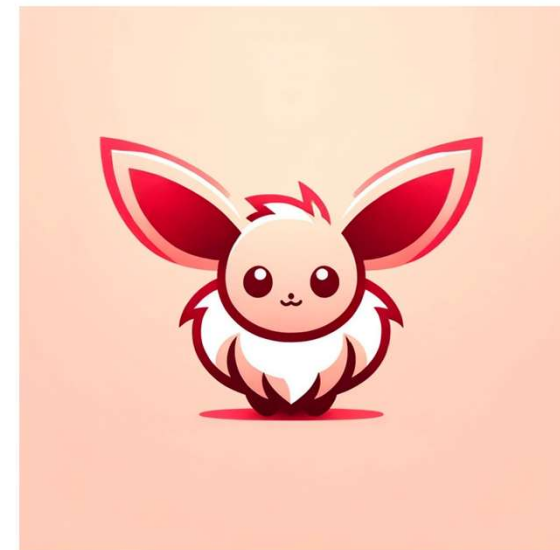
●Upstage SOLAR Pro

- 10.7B (23.12) 22B (24.10) 이후
- Solar Pro 2 31B 출시 예정



●Yanolja Eevee : 10.8B (24.1)

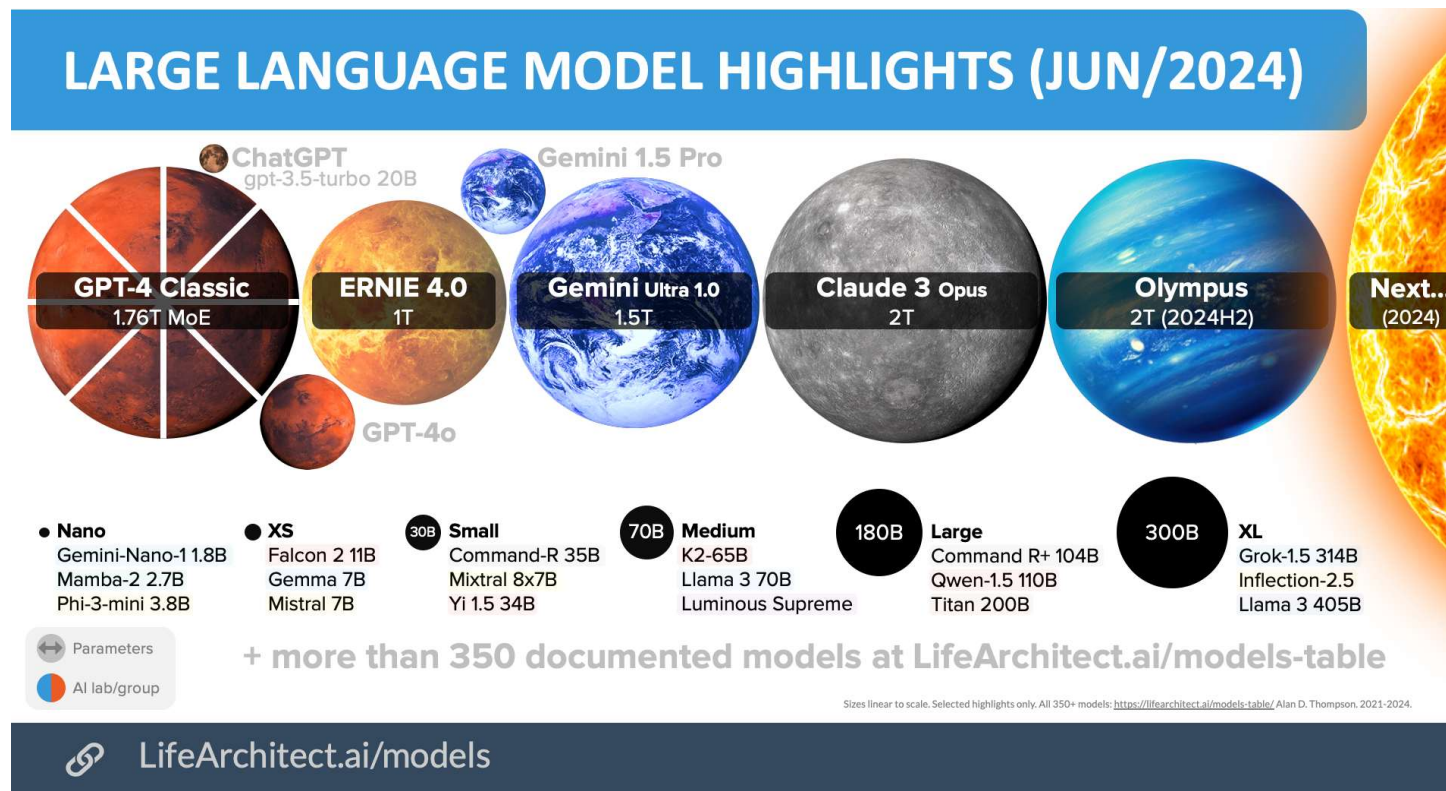
- SOLAR 10.7B 모델에 한국어 토큰을 추가하고 추가 학습하여 성능을 높임
- <https://arxiv.org/pdf/2402.14714>



Ch.8 Open-Weight LLM

오픈 LLM 모델 선택하기: 크기

- GPU 사용량을 고려한 모델 선택 필요

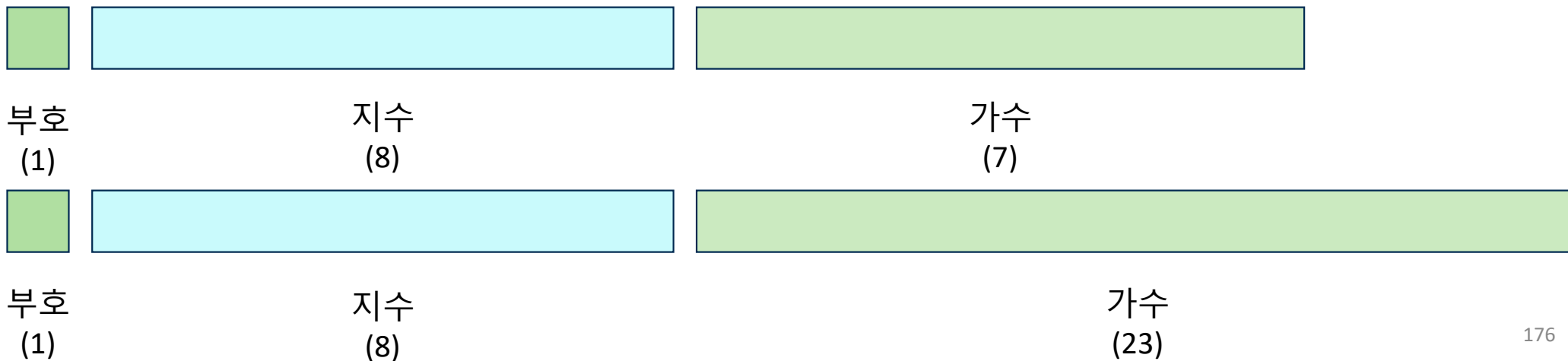


파라미터 수 – VRAM의 관계

● Closed-Source 모델과 다르게 오픈 모델은 충분한 크기의 GPU 인프라 필요

- 큰 모델은 불러오기도 어려울 수 있음
- Ex) 모델의 파라미터 : 7B, 13B, 60B, 180B, ... : 가중치 수가 70억, 130억, ...

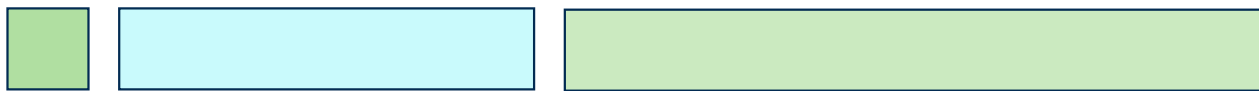
● 각 가중치는 일반적으로 16비트(bfloat16)/ 32비트(float32) 자료형으로 구성



파라미터 수 – VRAM의 관계 (2)

- 16비트= 2바이트이므로 하나의 파라미터는 2바이트의 공간 차지

- 파라미터 7B 모델 → 140억 바이트 → 14GB

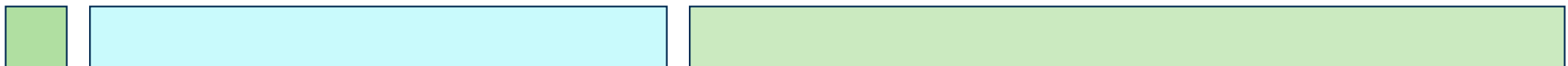


부호
(1)

지수
(5)

가수
(10)

- 참고) float32 자료형



부호
(1)

지수
(8)

가수
(23)

파라미터 수 – VRAM의 관계 (3)

- 실제 사용시에는 입력 Context에 따라 GPU 소모 크게 증가
 - Attention 계산 과정의 Caching 등
- vLLM 등의 서빙 최적화 라이브러리 사용
- 모델의 크기를 압축하여, 활용성을 높이는 양자화(Quantization) 방법 고려

Quantization (양자화) - 고전적

● Weight의 복잡한 값을 압축하여 모델 크기를 줄이는 과정

● Ex) INT8 양자화

- 모델 가중치의 최대/최소를 구한 뒤, 해당 범위를 255개의 균등 가중치로 양자화
- 16비트 모델의 VRAM 사용량 1/2 로 감소
- INT4 양자화는 1/4로 감소하나, 성능 손실이 매우 큼

0.67	0.44	0.56	0.10
0.74	0.89	0.60	0.84
0.83	0.00	0.34	0.30
0.45	0.54	0.88	0.41

→

170	112	142	25
188	226	153	214
211	155	86	76
114	137	224	104

 $\times \frac{1}{255}$

Quantization (양자화) - 이후 발전

● 4비트 양자화 : nf4

- QLoRA 논문에서 제안한 방식(<https://arxiv.org/abs/2305.14314>)
- 단순한 4비트 양자화는 성능이 크게 하락하므로, 최대한 분포를 표현하도록 분위수(Quantile) 기반 양자화 수행
- 전체 범위를 표현하는 16개의 분위수를 이용해
데이터가 특정 구간에 포함되면 가까운 값으로 근사
 - Ex) 0.07 → [0.02, 0.09] → 가까운 값 0.09 으로 근사
- 메모리에는 16개 값의 인덱스를 저장하여 변환하고, 계산시에 인덱스 테이블에서 값을 가져옴
- INT4보다는 적은 성능 저하

Quantization (양자화) - 이후 발전(2)

- GPTQ, AWQ: 성능 저하를 최소화하는 양자화 방법들
- 모든 가중치를 공평하게 양자화하는 대신, 최적의 선택적 양자화 고려
- 기계적 양자화가 아닌 입력 데이터를 통한 Calibration 스타일의 양자화
 - GPTQ: 파라미터들의 중요도를 각각 구하여, 양자화 정도를 조절하며
순차적 양자화를 통해 오류를 다음 가중치에서 보상하는 방식
 - AWQ: 가중치의 활성화(Activation) 값을 기반으로 선택적 양자화 수행
- 보통 개별 라이브러리가 존재하므로, 직접 구현하거나 이해할 필요는 별로 없음

Quantization (양자화) - 그외

- GGUF/GGML: Llama.cpp 기반, CPU 추론에 최적화된 양자화 포맷
 - 전체 모델을 단일 파일에 전부 저장하여 배포와 사용이 간편
- Quantization-Aware Training
 - Google Gemma 3에서 함께 공개된 방식
 - 학습 단계부터 양자화 손실을 줄이는 방향의 학습
- 동적 파라미터 모델을 통한 파라미터 최소화
 - <https://ai.google.dev/gemma/docs/gemma-3n>

Ch.8 Open-Weight LLM

대표적 모델 양자화 방법

● 허깅페이스의 양자화 지원 상황

- 새로운 양자화 방식은 계속 출시되고 있으며, 모델 개발 당시에 함께 포함하는 추세

Quantization Method	On the fly quantization	CPU	CUDA GPU	ROCm GPU	Metal (Apple Silicon)	Intel GPU	Torch compile()	Bits
AQLM	●	●	●	●	●	●	●	1/2
AutoRound	●	●	●	●	●	●	●	2/3/4/8
AWQ	●	●	●	●	●	●	?	4
bitsandbytes	●	●	●	●	●	●	●	4/8
compressed-tensors	●	●	●	●	●	●	●	1/8
EETQ	●	●	●	●	●	●	?	8
GGUF / GGML (.llama.cpp)	●	●	●	●	●	●	●	1/8
GPTQModel	●	●	●	●	●	●	●	2/3/4/8
AutoGPTQ	●	●	●	●	●	●	●	2/3/4/8
HIGGS	●	●	●	●	●	●	●	2/4

성능이 얼마나 차이날까?

● <https://github.com/IST-DASLab/gptq>

● Perplexity 점수 변화

Here is a summary of LLaMa results:

Wiki2 PPL	FP16	4bit-RTN	4bit-GPTQ	3bit-RTN	3bit-GPTQ	3g128-GPTQ
LLaMa-7B	5.68	6.29	6.09	25.54	8.07	6.61
LLaMa-13B	5.09	5.53	5.36	11.40	6.63	5.62
LLaMa-30B	4.10	4.54	4.45	14.89	5.69	4.80
LLaMa-65B	3.53	3.92	3.84	10.59	5.04	4.17

Example: VRAM 소모 비교하기

- 파라미터만 불러왔을 때의 상황

로드 방식	VRAM(MB)
Original(bfloat16)	15637
Load in 8bit	8927
Load in 4bit	6095
Load in 4bit + Double Quant.	5749

오픈 소스 LLM을 불러온 후 확인해야 하는 것

- LLM의 작동 방식, 프롬프트 활용 방법 등은 모두 다름
 - 코드를 그대로 사용하는 것은 적절하지 않을 수 있음
- 모델의 공식 홈페이지나, 프롬프팅에 대한 가이드를 찾기
 - 모델을 불러온 뒤, Tokenizer에서 확인할 수도 있음

실습) 다양한 오픈 LLM의 프롬프트 템플릿 확인하기

Fine Tuning

변형호

Ch.9 Fine-Tuning

HuggingFace 검색: Qwen2.5 7B

● 왜 6개나 있을까?

- Base Model : 채팅 기능 미포함 (7B)
- Instruct Model: 채팅 기능이 탑재된 파인 튜닝 모델 (7B-Instruct)
- Quantized Model: 양자화로 크기 축소 (7B-Instruct-OO)

 Qwen/Qwen2.5-7B

 Text Generation • Updated Sep 25 • ↓ 116k • ♥ 84

 Qwen/Qwen2.5-7B-Instruct

 Text Generation • Updated Sep 25 • ↓ 2.04M • ♥ 364

 Qwen/Qwen2.5-7B-Instruct-GGUF

 Text Generation • Updated Sep 20 • ↓ 19.4k • ♥ 41

 Qwen/Qwen2.5-7B-Instruct-AWQ

 Text Generation • Updated Oct 9 • ↓ 20.3k • ♥ 14

 Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4

 Text Generation • Updated Oct 18 • ↓ 34.2k • ♥ 12

 Qwen/Qwen2.5-7B-Instruct-GPTQ-Int8

 Text Generation • Updated Oct 18 • ↓ 10.6k • ♥ 8

Ch.9 Fine-Tuning

Base Model: Pretraining만 수행한 모델

● Pretraining (사전 학습)

- 대규모 언어 코퍼스를 이용해, 언어 패턴을 학습시키는 과정
- 주어진 문장의 다음 단어 예측하기 (단순 Completion 학습)
LLM은 Large Language Model의 _____ ← 약자입니다.
- 위키피디아, 스택오버플로우, 뉴스 기사 등을 수집하여 학습
→ C4 (Colossal Clean Crawled Corpus)
- 저작권을 위반한 데이터가 학습되는 경우 존재
 - 적발하기가 어려운 특성

Base Model: Pretrained Model

- 다음 단어 예측만을 수행하므로, 질의응답/ 지시사항 능력 부족

입력) 거대 언어 모델은

출력) 대량의 파라미터의 인공 신경망 구조로 구성된 언어 모델입니다. 거대 언어 모델의 발전은 트랜스포머와 ...

입력) 거대 언어 모델이 뭐야?

출력) 요즘 뉴스에 너무 많이 나오더라구, LLM이랑 같은 거야?

입력) 거대 언어 모델이 뭐야?

출력) ChatGPT가 뭐야? Claude가 뭐야?

Instruct Model: 질의응답/ 지시사항 형식의 템플릿 이해

- 단순 Completion에서, 질의응답과 지시사항의 데이터로 파인 튜닝한 모델
- Ex) Llama 3 Series Template
 - `<|start_header_id|>assistant<|end_header_id|>` 이후부터 생성!

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
```

```
당신은 도움을 주는 비서입니다.<|eot_id|><|start_header_id|>user<|end_header_id|>
```

```
What can you help me with? <|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

파인 튜닝이란?

- 학습이 완료된 모델의 가중치를 새로운 학습 데이터를 이용하여 변경시키는 것
 - 다음 단어를 예측하는 확률 분포가 바뀌게 됨
 - Ex) Llama 모델에 법률 판례를 계속 집어넣어 법률 상담에 특화된 봇으로 바꾸겠다
베이스모델에 질의응답 패턴을 가르치겠다

●파인 튜닝의 목적

- 질의응답 패턴 주입
- 도메인 특화 / Task 특화 데이터 주입
- 다국어 지원

파인 튜닝의 영향

- 기존 데이터로 학습된 모델에게 새로운 데이터 제시
 - 새로운 데이터에 Fit 하는 과정: LLM의 이전 결과는 오답이 됨
- Catastrophic Forgetting
 - LLM의 기존 파라미터는 다양한 코퍼스 학습의 결과물
 - 추가 학습을 진행 시 기존의 능력을 잃어버릴 수 있음
 - Ex) 한국어 코퍼스 학습 이후 영어 능력 저하

파인 튜닝: 목적에 따른 분류

● Continuous Pretraining

- Pretrain 모델에 새로운 코퍼스를 학습시키기

● Instruction Tuning

- Pretrain 모델에 질의응답(채팅) 기능을 추가하기

● Domain QA Tuning

- (어떤 모델이든) 도메인 채팅 데이터로 학습시키기

● Refusal Training

- (어떤 모델이든) 비윤리적인 응답 금지시키기

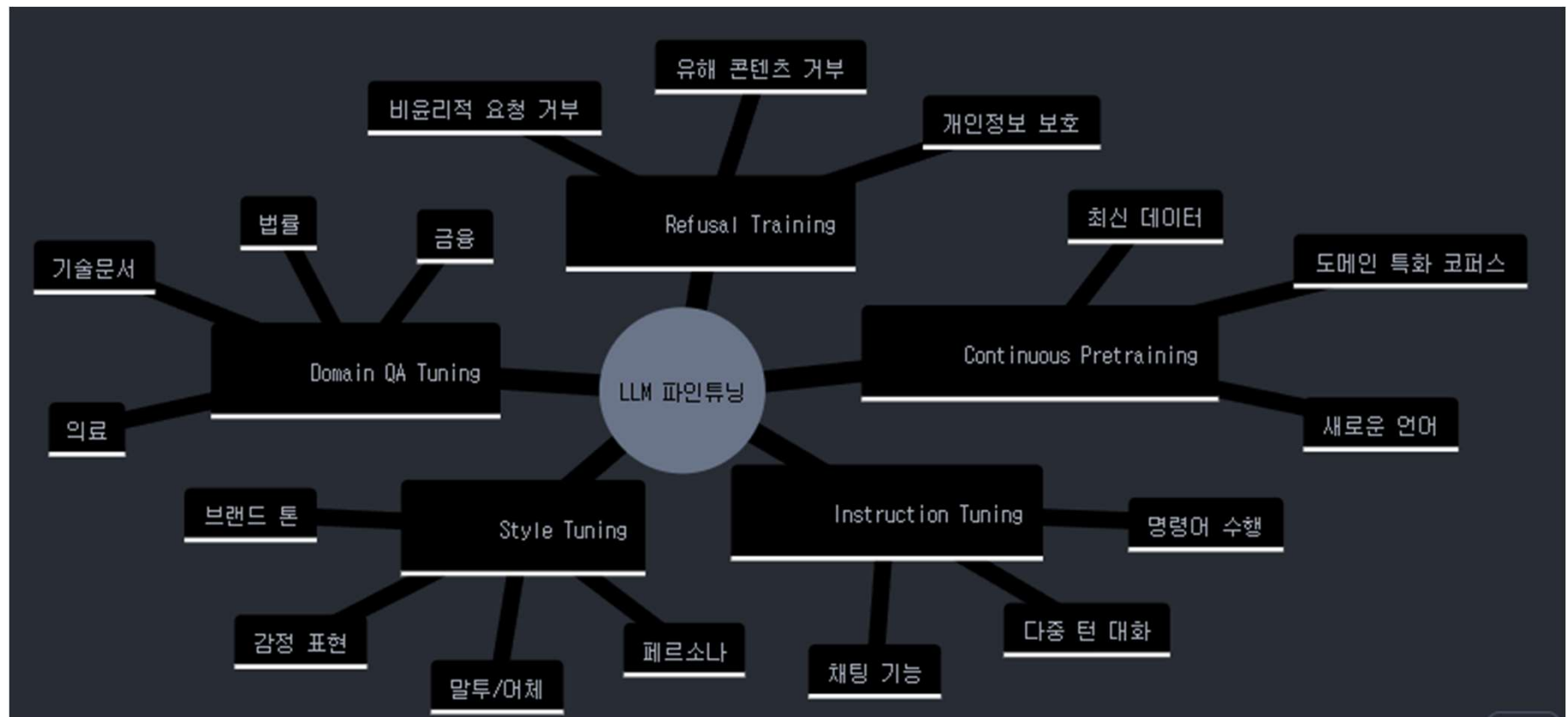
● Style Tuning

- 답변의 형식 변화시키기 (길이, 말투 등)

Ch.9 Fine-Tuning

파인 튜닝: 목적에 따른 분류

● By Claude 3.5 Sonnet



Continuous Pretraining

- Pretrained Model에 주로 Domain Knowledge, Multilingual Corpus 추가 학습
 - 교과서와 같은 지식을 학습하여 이후 질의응답 포맷에서 활용
 - Catastrophic Forgetting의 위험이 가장 큼
 - 다양한 해결 방안 연구 중
 - Ex) LLaMA Pro: Progressive LLaMA with Block Expansion (ACL 2024)
 - Self-Distillation Bridges Distribution Gap in Language Model Fine-Tuning (ACL 2024)
 - Mitigating Catastrophic Forgetting in Large Language Models with Self-Synthesized Rehearsal (ACL 2024)
- Instruct Model에는 수행하지 않음
 - 모델의 Instruction 능력이 손상된다는 연구 결과

Instruction Tuning: 질의 응답 가르치기

- 주로 Instruction의 패턴을 가르치는 것이 목적
 - 채팅이나 지시사항의 템플릿을 만들고, 이를 학습시키는 방법
- 대표적 Instruction Tuning 데이터 : KoAlpaca, KoPlatypus, ...
 - <https://huggingface.co/datasets/beomi/KoAlpaca-RealQA>
- Reasoning Data: 추론 과정이 포함된 데이터
 - <https://huggingface.co/datasets/exp-models/s1K-1.1-Korean>

Instruction 데이터 템플릿 구성하기

- 일반 Completion 입력과 구분되어야 함

- Ex) Alpaca (입력이 없는 경우)

- <https://huggingface.co/datasets/tatsu-lab/alpaca>

프롬프트:

Below is an instruction that describes a task.

Write a response that appropriately completes the request.

Instruction : {}

Response : {}

Instruction 데이터 템플릿 구성하기

- 일반 Completion 입력과 구분되어야 함
 - Ex) Alpaca (입력이 있는 경우)

프롬프트:

Below is an instruction that describes a task, paired with an input that provides further context.
Write a response that appropriately completes the request.

Instruction : {}

Input : {}

Response : {}

Ch.9 Fine-Tuning

Instruction Tuning의 효과

- LLM이 Instruction을 이해하고 결과를 출력

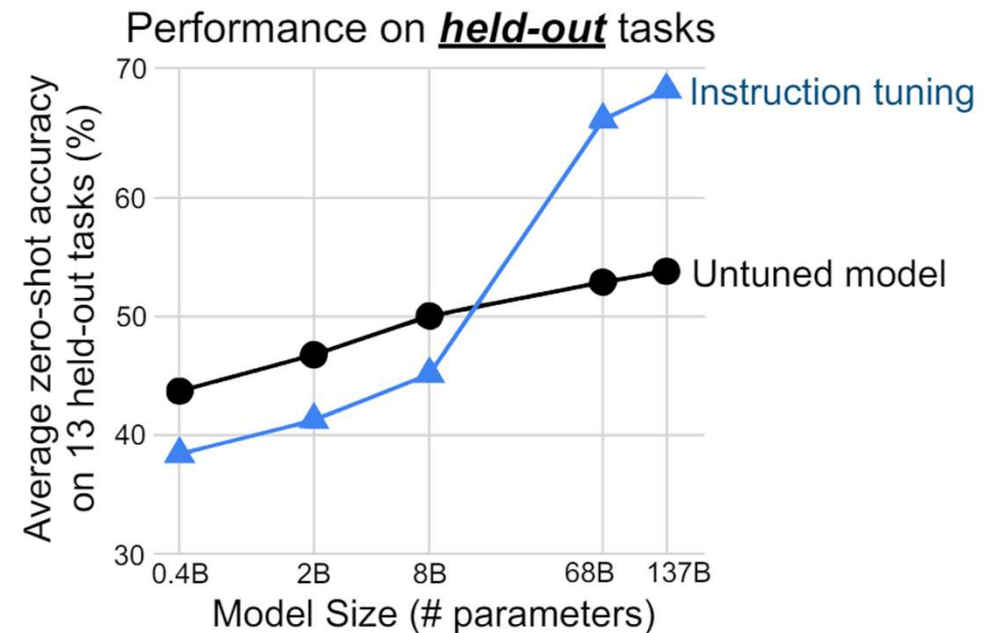
→ 파인 튜닝 데이터의 형식에 따라, 질문도 동일하게 변환되어야 한다

- 심지어 배우지 않은 task에서도 결과를 도출한다

- Finetuned Language Models Are Zero-Shot Learners

(<https://arxiv.org/abs/2109.01652>)

- 모델이 어느 정도 클 때만 그렇다고 관측되었으나(21년)
최근에는 2B 모델 등의 작은 모델도 튜닝을 수행함



Instruction Tuning의 한계에 대한 시각

● 2023년의 시각: Meta의 LIMA: Less is More for Alignment

- 피상적 정렬 가설 (Superficial Alignment Hypothesis)
“대부분의 지식은 Pretrain 과정에서 학습되며,
Instruction Data는 지식을 정리하는 것에 불과하다“

● 2024년의 시각: Instruction Tuning으로도 지식 향상 가능하다! 그러나...

- 해결하고자 하는 문제의 범위가 좁다면 가능할 수 있다
 - 일반화하기가 어렵다
- Continuous Pretrain : 교과서 / I.T : 문제집의 비유

Style Tuning?

- 답변의 형식을 변환시키는 학습 방법
 - 공식 명칭은 아님
- 답변의 말투, 길이, 스타일을 수정하거나, 추론 방식을 변화시킬 수 있다
 - Example) Adapting Language Model to Domain Specific RAG : RAG 성능을 높이는 파인 튜닝

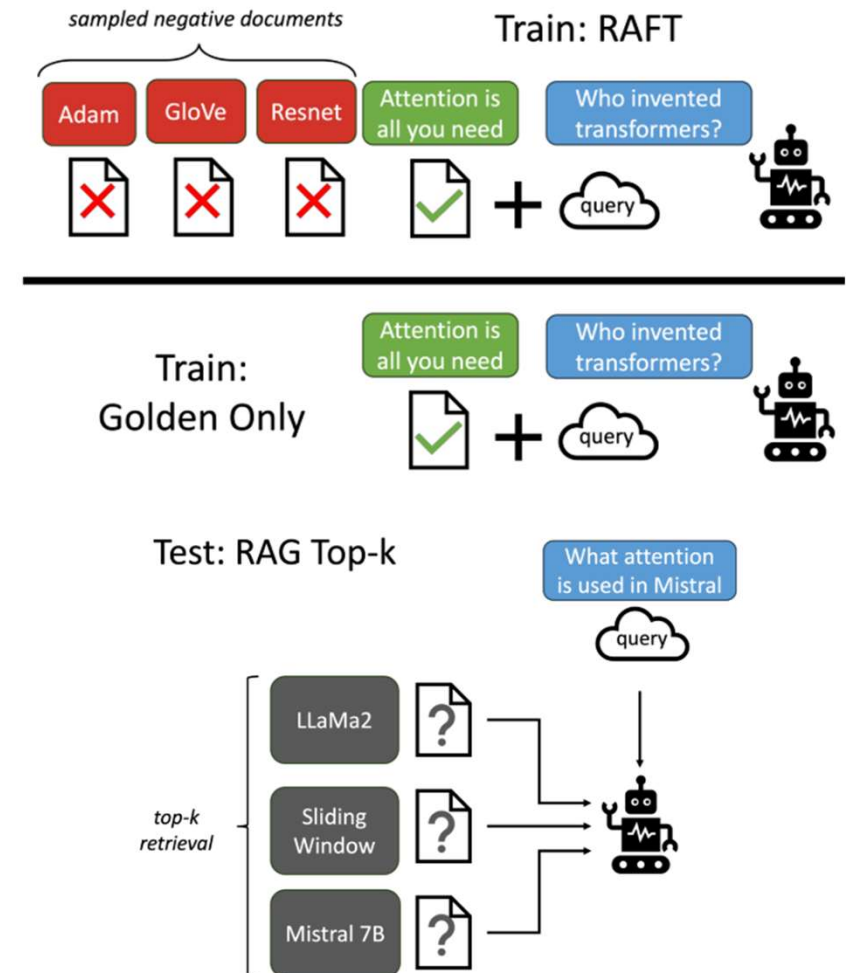
Ch.9 Fine-Tuning

RAFT: 오픈 북 시험 공부

●RAFT의 아이디어: RAG의 포맷 데이터로 모델 파인 튜닝

●2가지 시나리오에 대해 학습

- Negative(관련이 없는)와 Positive(관련이 있는) 문서를 혼합하여 질의응답 데이터를 학습
- Positive 문서만 포함한 경우의 질의응답 데이터를 학습



Source: RAFT: Adapting Language Model to Domain Specific RAG,
<https://arxiv.org/abs/2403.10131>

RAFT: CoT-RAG 데이터 학습시키기

- Context가 주어질 때, RAG를 수행하는 CoT 데이터를 생성하여 파인 튜닝
 - 다양한 상황에 맞춘 데이터가 필요함

CoT Answer :

##근거 : 문서 **##인용시작##** 최초 가입일로부터 90일 이상 경과시부터 보장 **##인용종료##**에 따르면, 현재 1년이 경과했으므로 보장 대상입니다. 또한 문서 **##인용시작##** 연체 3개월 이상 시 보장 중단 **##인용종료##**에 의하면, 연체 내역이 없으므로 보장 중단 가능성도 없습니다.
따라서, 주어진 조건은 보험 보장이 가능합니다. **##정답: 가능**

파인 튜닝의 제약조건

- 모델 파라미터 + 그래디언트 + Optimizer + Activation

- 다양한 테크닉으로 줄일 수 있으나, 속도가 느리거나 학습이 부족
- Ex) Paged Optimizer : CPU 오프로딩을 고려하여 GPU 오버헤드 방지/ 속도 증가

- 반 영구적인 학습: 되돌리기 불가

- Ex) 고객 약관이나 사명 등이 변경된 경우?

파인 튜닝 데이터 구성

● 코퍼스

- Plain Text로 구성된 말뭉치
- Ex) 의료 정보, 법률 사전, 사내 규정집 등

● Instruction Data

- 입력/출력으로 구성된 데이터
- Ex) Text2SQL, 분류, 요약 등

● Q/A Data

- 질문/답변으로 구성된 데이터
- 멀티턴 작업이 필요한 경우 존재
- Ex) 상담, 질의응답

파인 튜닝 데이터 만들기

● 수집

- 관련 도메인의 다양한 데이터 크롤링 및 수집
- HuggingFace (<https://huggingface.co/datasets>)
- AI Hub(<https://www.aihub.or.kr/index.do>) 의 공개 데이터 사용 가능

● 생성

- 고성능 LLM을 통해 생성하기
- 모델마다 규정이 다르므로, 숙지 필요
 - Qwen : 학습 사용시 모델 설명에 명시해야 함
 - OpenAI: “Cannot Use Output to develop models that compete with OpenAI.”
- 데이터의 품질 관리 필요

파인 튜닝 데이터 만들기 (2)

● 어떤 데이터를 준비해야 하는가?

- 풀고자 하는 문제의 분포를 충분히 표현할 만한 데이터
- 코퍼스/QA/Instruction의 길이, 출처, 내용, 형식 다양화 고려

● 코퍼스는 주로 외부 소스를 통해 구하나, 다양한 Instruction은 구하기 어려움

- 코퍼스를 활용한 QA 세트 생성 등의 작업 고려
- 좋은 모델을 사용하면 데이터의 퀄리티가 좋아짐
- 합성 데이터만 사용하기보다는 실제 데이터와 결합하는 것이 중요

[실습] 파인 튜닝 데이터 생성하기

- Corpus, Instruction Tuning을 위한 데이터 생성 과정 알아보기
- Style Tuning을 위한 데이터 생성

[Recap] 파인 튜닝의 제약조건

- 높은 GPU 소모

- 0.5B 풀 파인 튜닝: 15GB에서 겨우 성공

- Catastrophic Forgetting

- 의료 코퍼스로 학습하자, 수학, 상식 능력을 모두 잃어버림
- 기존 데이터의 복습이 필요?

파인 튜닝의 대안

- PEFT(Parameter Efficient Fine Tuning)

- 기존 파라미터를 보존하며, 적은 양의 메모리를 활용하는 파인 튜닝

- 모델에 일부 어댑터(Adapter)를 추가하여 출력의 방향을 조절

- 어댑터의 가중치만 학습시키는 방식으로 파인 튜닝 수행

- LoRA : Low-Rank Adaptation Fine Tuning

- 가장 대표적인 PEFT 알고리즘

Ch.9 Fine-Tuning

LoRA 메커니즘 이해하기

- Pretrained Weights는 그대로 두고,

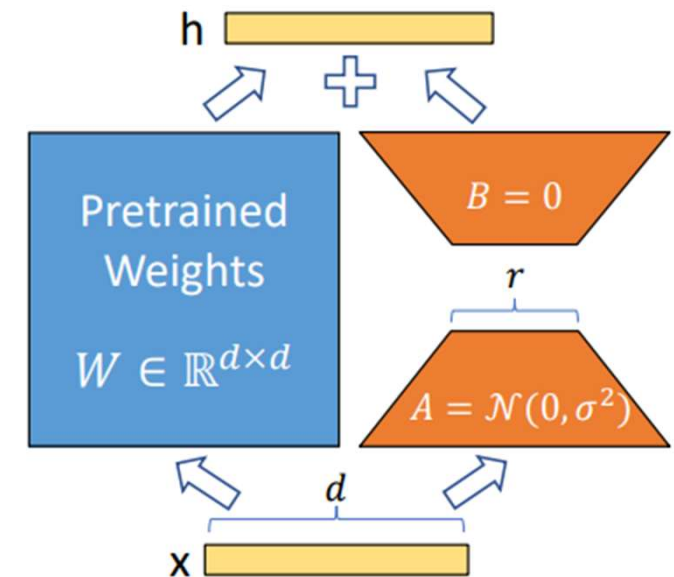
새로운 A,B를 적용하여 결과 변경

$$h = Wx \rightarrow h = (W + BA)x$$

- Concept : W를 다 쓸 필요가 없고, d의 정보를 r차원으로 줄여서

내재된(Intrinsic) 데이터를 추출하고, 이를 활용하는 것만으로도

파인 튜닝의 효과를 낼 수 있다



Source: LoRA: Low-Rank Adaptation of Large Language Models <https://arxiv.org/abs/2106.09685>

Ch.9 Fine-Tuning

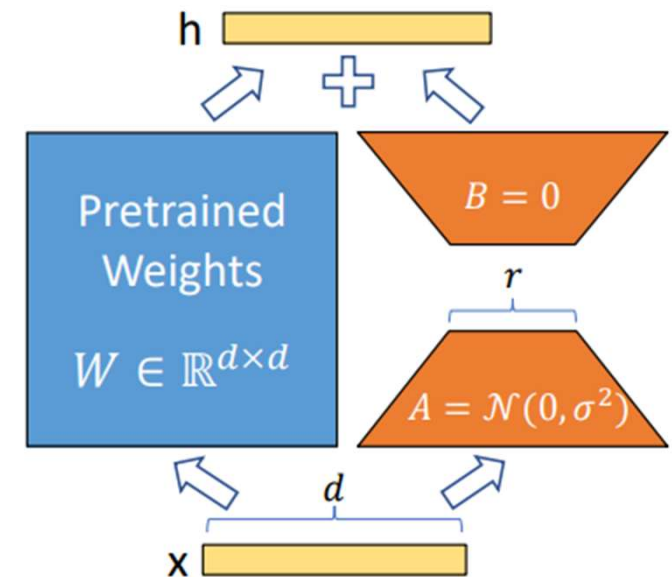
LoRA는 파라미터를 매우 적게 학습시킨다

●파라미터 수 비교 : $d * d$ VS $2 * r * d$

- 학습에 사용되는 부분만 그래디언트가 필요하므로, 큰 메모리 절약

● Fine-Tuning 대비 1% 이하의 파라미터 변경

- Ex) $d = 3584, r = 4$



Source: LoRA: Low-Rank Adaptation of Large Language Models <https://arxiv.org/abs/2106.09685>

LoRA가 아닌 언어 모델의 기본 학습 방식

- x : 입력 문장, y : 출력 문장
- 기존의 언어 모델 : P_Φ

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (P_\Phi(y_t|x, y_{<t}))$$

- 모든 x 에 대해, 각 Step에서 다음 y 를 맞힐 확률이 최대화되는 Φ (파라미터)를 구한다

LoRA의 학습 방식

● x : 입력 문장, y : 출력 문장

● LoRa

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t}))$$

- 원래 Φ 는 그대로 두고, $\Delta\Phi(\Theta)$ 를 추가해서 최적의 Θ 를 찾는다 (A,B로 추가되는 파라미터)

LoRA: 무엇이 좋은가?

- 매우 적은 파라미터로 파인 튜닝 가능
 - A/B 테스트, 버저닝이 쉬움
- 원래 모델의 파라미터를 변화시키지 않음: 탈착 가능
 - 여러 개의 LoRA를 처리하는 Multi-LoRA 구성도 가능
- Catastrophic Forgetting이 덜 발생한다 ?
 - 기본 가중치 Space의 변화량이 적다

Ch.9 Fine-Tuning

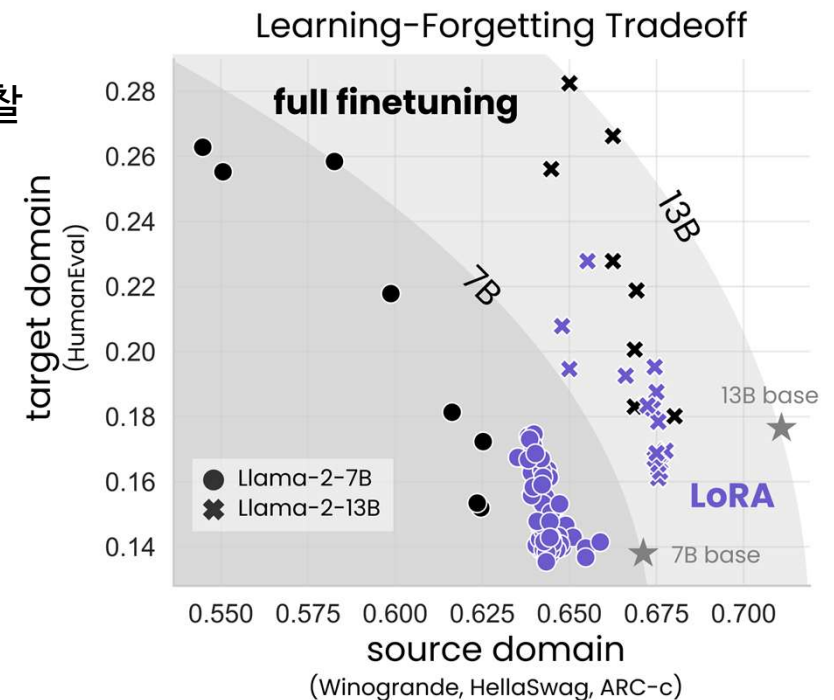
LoRA의 단점은 무엇일까?

● “LoRA Learns Less and Forget Less”

- <https://arxiv.org/abs/2405.09673>
- Full Fine Tuning에 비해, 목표 도메인의 학습 효과가 떨어진다는 관찰
- 그 대신, 기존 도메인의 망각이 적다
- (반박 의견도 있음: <https://unsloth.ai/blog/contpretraining>)

● “LoRA vs Full Fine-tuning: An Illusion of Equivalence”

- <https://arxiv.org/abs/2410.21228>
- 어려운 문제에서는 성능이 감소한다
- 낮은 차원 LoRA는 일반화 성능이 많이 떨어진다



실습) LoRA를 이용한 Style Tuning 학습

Ch.9 Fine-Tuning

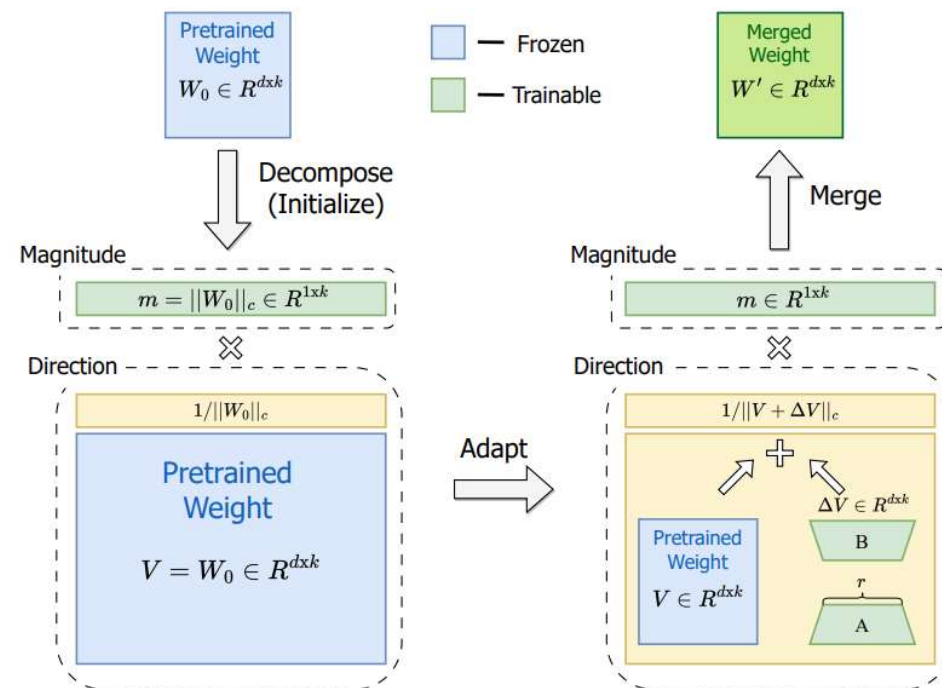
DoRa: Decomposed LoRA

● Pretrained Weight를 $M \times D$ 로 분해

- Magnitude : 크기
- Direction: 방향

● M까지 같이 학습시키면,

더 효율적이고 정확한 fine tuning 가능

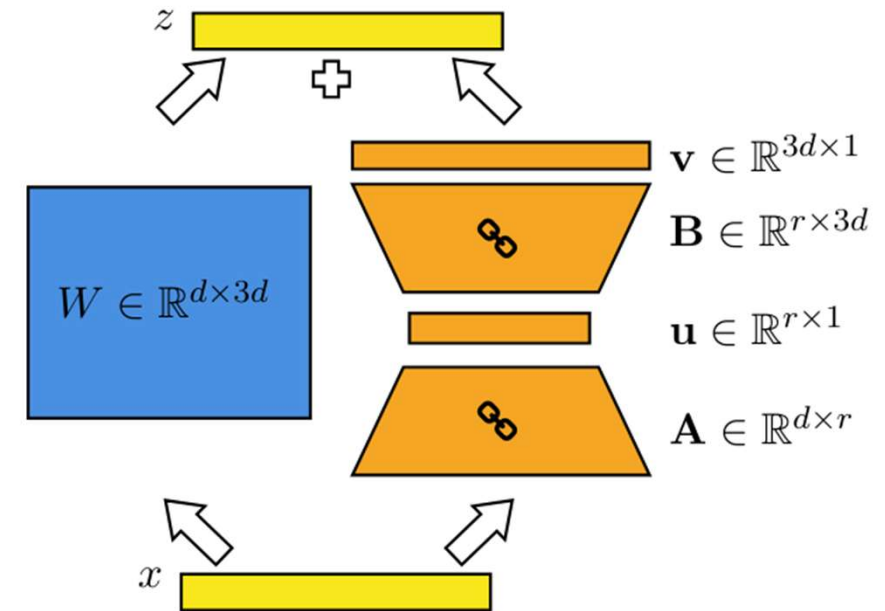


Source: DoRa: Weight-Decomposed Low-Rank Adaptation <https://arxiv.org/pdf/2402.09353>

Ch.9 Fine-Tuning

Tied-LoRA

- 모든 중간 Layer에서, A와 B를 통일하자
 - 파라미터 수가 1/L 로 감소
- 그 대신, u와 v의 scaling을 조절하자
 - 기존 LoRA: 모두 1로 고정
 - TIED: 모든 경우 테스트: TL6가 제일 좋았다



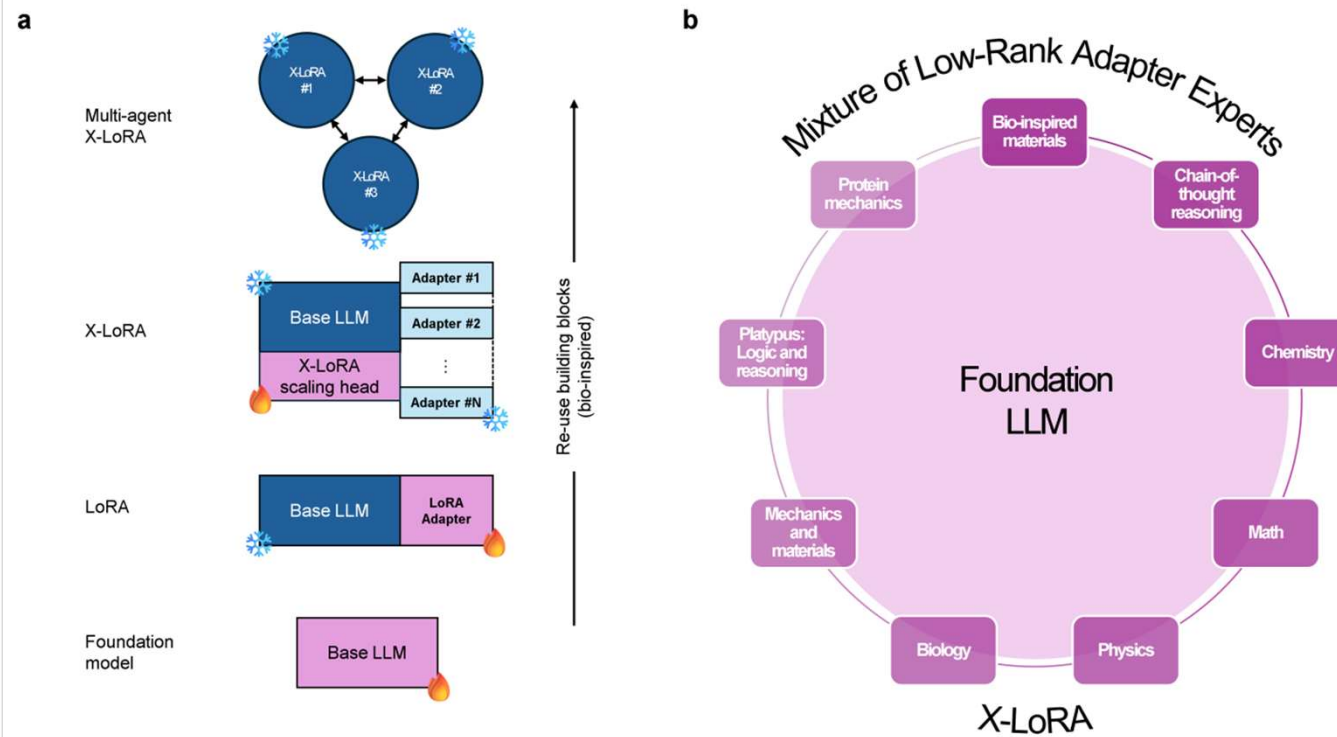
Method	Parameters	Initialization
LoRA ($\mathbf{vBuA}$)	$4Ldr$	$A \sim \mathcal{N}, B = 0, u, v = 1$
Vera ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$L(r + 3d)$	$A, B \sim \mathcal{N}, u = 1, v = 0$
TL ₁ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	dr	$A, B \sim \mathcal{N}, u, v = 1$
TL ₂ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$dr + L(r + 3d)$	$A, B \sim \mathcal{N}, u = 1, v = 0$
TL ₃ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$3dr$	$A \sim \mathcal{N}, B = 0, u, v = 1$
TL ₄ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$(L + 3d)r$	$A \sim \mathcal{N}, B = 0, v, u = 1$
TL ₅ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$4dr$	$A \sim \mathcal{N}, B = 0, u, v = 1$
TL ₆ ($\mathbf{vB}_{\infty} \mathbf{uA}_{\infty}$)	$4dr + L(r + 3d)$	$A, B \sim \mathcal{N}, u = 1, v = 0$

Source: Tied-Lora: Enhancing parameter efficiency of LoRA with weight tying : Optimizing LoRa
<https://arxiv.org/abs/2311.09578>

Ch.9 Fine-Tuning

XLoRA : Mixture of LoRA Experts

- 여러 개의 LoRA 모델을 MoE처럼 쓰는 방법 (2024.2.11)



Source: <https://arxiv.org/abs/2402.07148>

정리) 최신 파인 튜닝 접근법

- 일반화된 지식: Corpus 필요
 - Pretrain 모델에서 수행 가능
- 질의응답: QA Data 필요
 - 지식 주입 능력은 제한적
- 결론) 도메인 지식을 넣는 채팅 모델을 만들기 위해서는
 - Base Model → Continuous Pretrain → Instruction Tuning
- 가장 복잡하지만, 이론적으로 확실한 방법

파인 튜닝: 현실적인 접근법

● 파인 튜닝 대신 RAG

- 전문 지식 부족이 체감되는 경우에 학습시키는 시나리오
- RAFT와 같은 방법을 고려할 수도 있음

● Instruct 모델에 Domain Q/A 만 학습시켜 보기

- 잘 되면 그대로 쓸 수 있다

Take-Home : 어떻게 준비해야 하는가?

● 코퍼스는 얼마나 준비해야 하는가?

- 정해진 값은 없으나, 최대한 넓은 커버리지
- 주로 공개 데이터나 API 등을 통해 수집

● 도메인 QA는 어떻게 만드는가?

- 코퍼스가 있다면, LLM을 통해 생성
- 실제 데이터와의 차이가 존재할 수 있으므로, 감독이 필요함

Post Training Part 2: 강화 학습

- 지금까지의 학습은 정답이 존재하는 Supervised Fine Tuning
 - 정답의 패턴은 잘 이해하지만, 일반화된 성능을 얻기는 어려울 수 있음
- SFT 학습 데이터의 한계
 - 모든 상황에 대한 이상적인 데이터 구축이 어려움
 - 정답을 그대로 정형화하여 학습하므로 일반화 성능이 떨어짐
- Alignment(정렬)의 필요성

Alignment (정렬)

● LLM 모델에게 좋은 답변의 기준을 학습시키는 과정

- 주로 모델이 생성한 데이터를 통해 피드백을 전달하여, 보다 일반화된 학습 가능

● 주로 강화 학습 기반의 방법을 채택

- RLHF : ChatGPT의 초기 버전부터, 지금까지 이어지고 있는 PPO 기반 학습
- DPO : RLHF의 높은 학습 난이도를 낮춘 직접적 선호도 학습 방법
- Rejection Sampling : 강화 학습은 아니지만, 함께 자주 쓰이는 방법
- GRPO : DeepSeek 01 2024년에 제안한 방법으로, R1 시리즈 모델에 활용

Ch.9 Fine-Tuning

RLHF: 보상 모델에 의한 최적화

● 보상 모델 구조: (프롬프트 + 응답) → Score

- Ex) <https://huggingface.co/Skywork/Skywork-Reward-Llama-3.1-8B-v0.2> (Model)
<https://huggingface.co/datasets/Skywork/Skywork-Reward-Preference-80K-v0.2> (Data)

●보상 모델 학습: 레이블러가 매긴 순위에 대해 선호도 학습

- 한 쌍의 데이터를 선택해 좋은 답변이 나올 상대 확률을 높이기
- 주로 Reward에 대한 Bradley-Terry 손실 함수를 사용

Question	Chosen	Rejected
10살짜리 아이와 대화한다고 가정해 보세요. 인공지능이 무엇인지 간단히 설명하세요.	인공지능은 스스로 생각하고 학습할 수 있는 로봇과 같습니다. 그저 시키는 대로 하는 것이 아니라, 정보를 사용하여 ...	좋아요, 놀이터 반대편에 있는 친구와도 대화할 수 있는 멋진 위키토기로 놀고 있다고 상상해보세요. ...

Source: <https://huggingface.co/datasets/NCSoft/offsetbias>

Ch.9 Fine-Tuning

RLHF: 보상 모델에 의한 최적화

● PPO

- Reward 최대화 + Reward Hacking을 방지하기 위한 Term 추가

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[\underbrace{r_{\phi}(x, y)}_{\text{Reward 최대화}} \right] - \beta \underbrace{\mathbb{D}_{\text{KL}} \left[\pi_{\theta}(y | x) \parallel \pi_{\text{ref}}(y | x) \right]}_{\text{KL Divergence (원본 모델과의 차이 완화)}}$$

● PPO의 단점

- 학습 중인 모델 / 레퍼런스 모델/ 리워드 모델
 - 3 LMs 동시 구동으로 인한 오버헤드
- PPO 자체의 불안정성 존재
 - 하이퍼파라미터 민감성 등...

context	And she would always love him for that. And for many other reasons. He had given her so much and she	
	<i>Continuation 1</i>	<i>Continuation 2</i>
zero-shot	had no idea how to release that gift, but she knew	had been unable to give it back.
		"You
no penalty / bonus	! These These These sound flowed instantly easily easily easily easily!	! These These These sound flowed instantly easily easily easily easily!
entropy bonus	initially initiallyprisingly easilyprisingly Liam! Love this ride instantly tightly	' equally particularlyprisinglyprisingly Liam Liam! Love this ride instantly

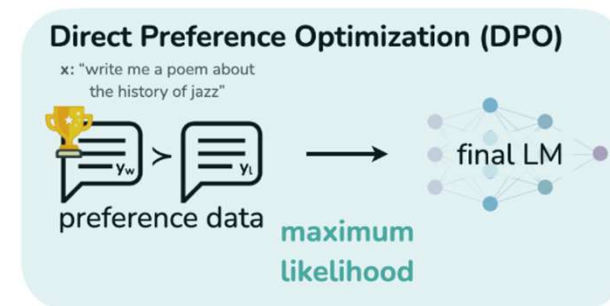
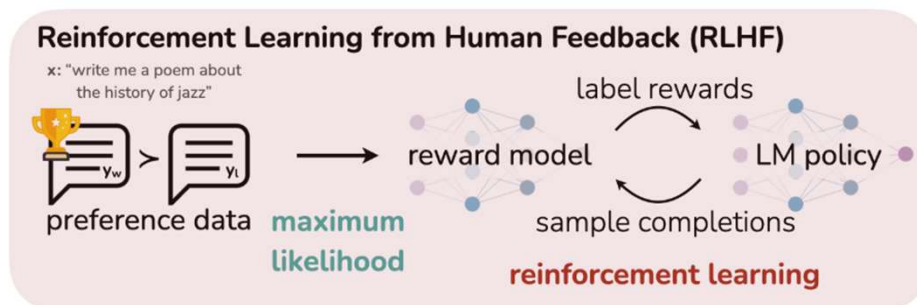
Source: <https://huggingface.co/papers/1909.08593>

Ch.9 Fine-Tuning

DPO: 직접적 선호 최적화

● Direct Preference Optimization: Your Language Model is Secretly a Reward Model (2023.12)

- Idea: “LLM 자체가 보상 함수 역할을 할 수 있다”



- Accepted/Rejected 데이터 쌍을 이용해
LLM에서의 출력에 대한 log odds 값을 최대/최소화

$$\max_{\pi} \mathbb{E}_{(x, y_w, y_l) \sim D} \log \sigma \left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$$

Rejection Sampling + SFT

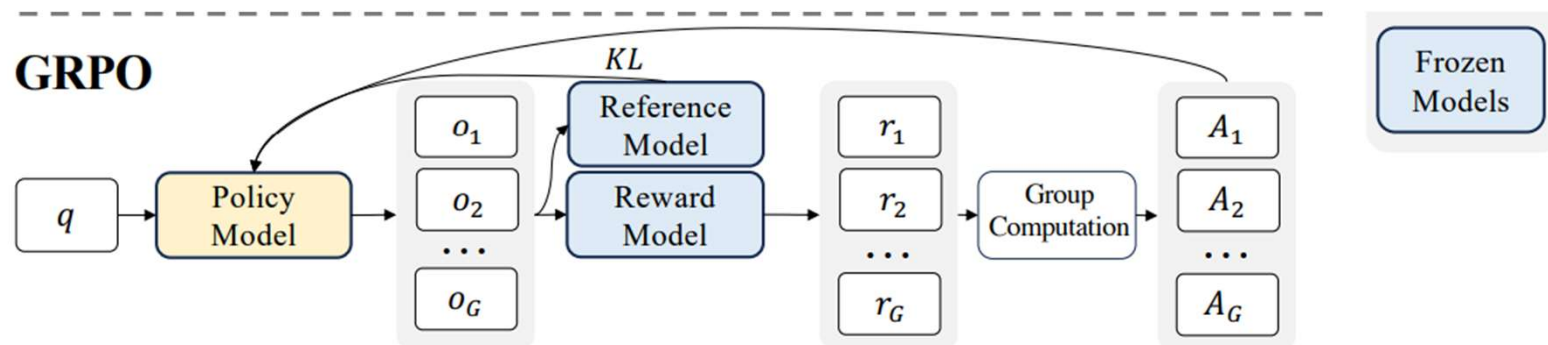
- LLM이 반복해 생성한 데이터 중, '좋은' 데이터만 필터링하기
 - 평가 방법이 엄밀할수록 효과적
- 좋은 데이터를 선정하여 추가적인 SFT
 - 좋은 품질의 데이터에 대해서만 추가 학습으로 답변의 품질이 향상되나
Positive 데이터에 대해서만 학습하므로 효과는 한정적

실습) Rejection Sampling과 DPO 데이터 만들고 학습시키기

GRPO: 상대 평가를 통한 선호 최적화

● DeepSeekMath(2024.2) 에서 처음 제안

- 동일한 입력에 대해 G 개의 출력을 생성한 뒤, 각각의 토큰별 Reward를 계산
- 그룹 내에서 높은 Reward를 받는 출력을 자주 생성하도록 학습



DeepSeek-Math의 GRPO

●수학 문제에 맞게, 결과와 과정을 모두 평가하는 복잡한 Reward Function 사용

- Math-Shepherd (2023) 모델 → 이것도 DeepSeek 논문
 - <https://arxiv.org/abs/2312.08935>
- Iterative RL(중간 학습 모델을 다음 학습의 Reference로 쓰는 방법)도 적용

●당시 Reward 모델에 대한 코멘트

- RL을 잘 하려면 Reward Model이 매우 중요하다.
- Reward Model이 뛰어나지 않을 경우, LLM의 능력을 높이기보다 안정화(Stabilize)시킬 수 있다
 - 일종의 틀에 박힌 사고

DeepSeek-R1-Zero

- DeepSeek-V3-Base (SFT 수행하지 않은 Pretrained Model)에서 시작
- GRPO (Rule-Based Reward Model)
 - <think> </think> 사이에 추론을 작성, <answer></answer> 사이에 답을 작성해라

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: **prompt**. Assistant:

- 단 2가지의 보상 모델 기준
 - <think> </think> 사이에 잘 작성했는가? (O/X)
 - 정답이 정확한가? (O/X)

Ch.9 Fine-Tuning

DeepSeek-R1-Zero: 간단해진 GRPO

● Rule-Based 보상을 사용하기 때문에, 모든 중간 토큰에 대한 보상을 동일하게 할당

- 토큰별 보상을 구할 필요가 없으니, 계산량이 크게 절약
- c.f) Process Reward Model: 스텝마다 다른 보상 부여 필요

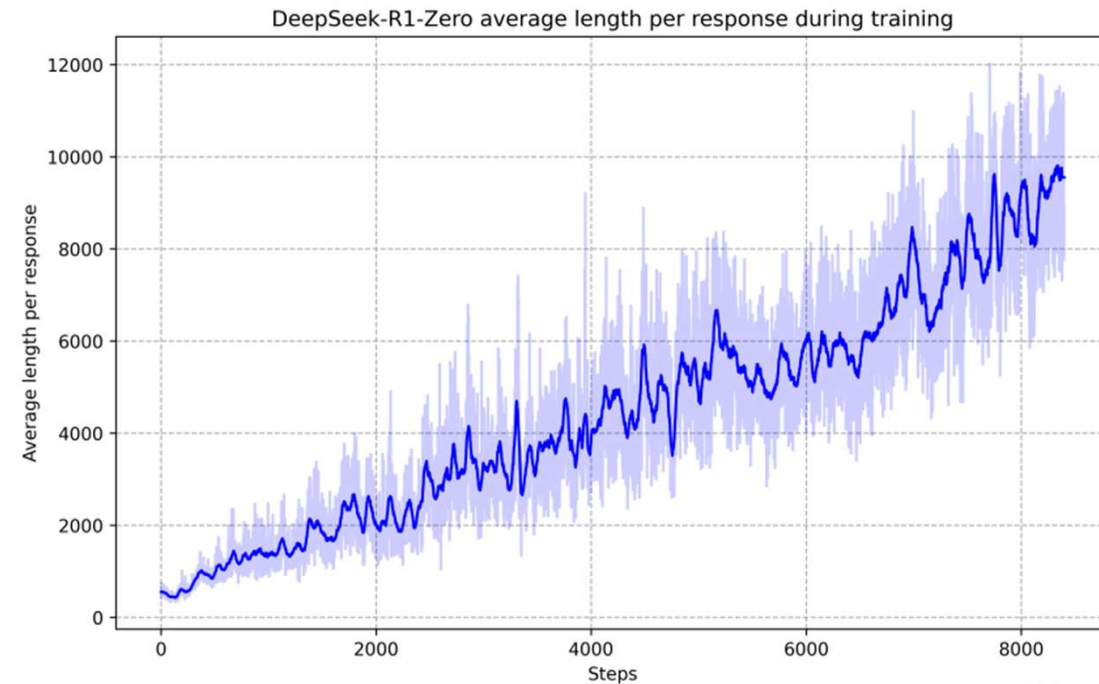
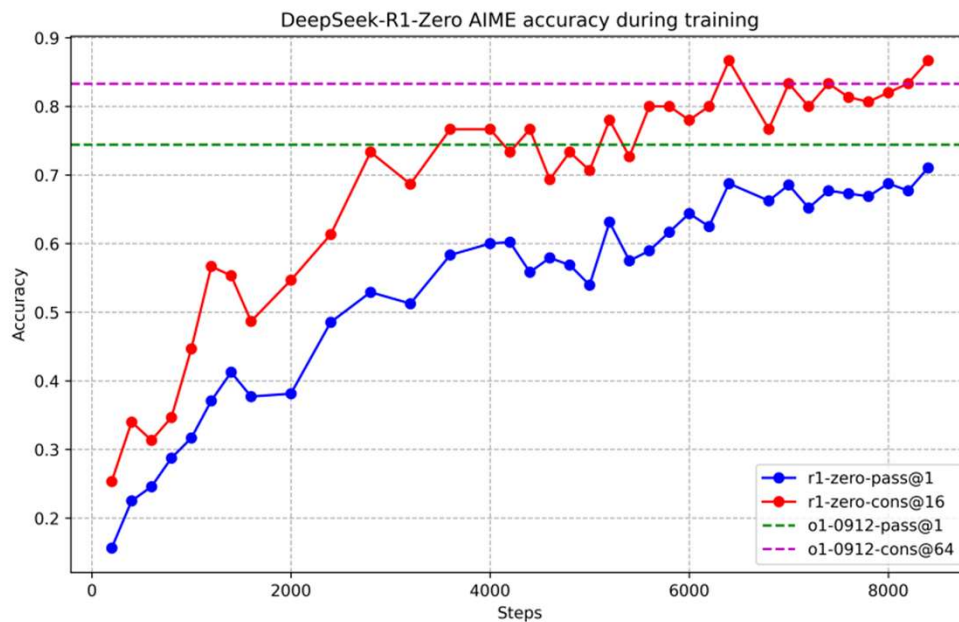
$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\underbrace{\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right)}_{\text{Clipping}} - \underbrace{\beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref})}_{\text{KL Divergence Term}} \right), \quad (1)$$

Ch.9 Fine-Tuning

DeepSeek-R1-Zero 결과

- RL을 진행할수록 답변이 길어지며 추론 능력이 점진적으로 향상
 - 16번의 반복 실행 후 Majority Vote를 하자, O1-0912를 상회하는 성능



DeepSeek-R1-Zero 결과 (2)

- DeepSeek-R1-Zero는 Instruction 없이도 문제를 잘 풀어냈지만, 낮은 가독성과 코드 믹싱 현상이 발생
- 이를 해결하기 위해, 일반적인 Instruction 학습 방식에 GRPO를 결합한 파이프라인을 다시 학습
- 4단계 학습 파이프라인
 - Cold-Start SFT
 - GRPO
 - Rejection Sampling + SFT
 - All-Scenario RL

DeepSeek-R1 학습 (1)

- DeepSeek-R1-Zero를 통해 Cold-Start 추론 데이터 생성
 - Long CoT, Self-Reflection, Self-Correction 과정을 포함하기
 - 약간의 인간 Annotator를 활용한 데이터 향상 + Instruction 형식 변환
 - 총 수천 개의 데이터 생성
- Cold-Start 데이터를 통해, DeepSeek-V3-Base 모델을 먼저 SFT 학습
 - 초기 불안정성을 크게 해소하고, 강화학습 전의 기초를 다지기
- 이후, GRPO 학습 수행
 - 언어가 섞이면 패널티를 주는 Language Consistency Reward 사용

DeepSeek-R1 학습 (2)

- GRPO가 수렴된 후, Rejection Sampling 수행
- Rejection Sampling이란?
 - 같은 입력에 대해 여러 개의 답변을 생성하고,
Reward 기준을 통과하거나 Best Reward에 해당하는 답변만 SFT 데이터셋으로 모음
- 여기서는 Rule-Based 방식이 아니라, V3를 개량해 만든 Reward Model 사용
 - 총 600k의 Reasoning Dataset
- 여기에 DeepSeek-V3-Instruct 학습 때 사용했던 200k의 Non-Reasoning 데이터를 추가해
총 800k의 데이터로 SFT 다시 학습

DeepSeek-R1 학습 (3)

- Rejection Sampling + SFT까지 수행한 후, 최종 강화 학습 단계

- 이전 GRPO가 Reasoning 능력 향상이었다면,
이 단계의 강화 학습은 일반적인 Alignment 단계

- Helpfulness, Harmlessness를 개선한 학습 방법 수행

- “All-Scenario RL” ?
- 정확한 과정은 논문에 비공개

DeepSeek-R1

● 이렇게 만들어진 모델은... (Source: <https://artificialanalysis.ai/leaderboards/models>)

MODEL ↕	CREATOR ↕	CONTEXT WINDOW ↕	ARTIFICIAL ANALYSIS INTELLIGENCE INDEX ↕
o3-mini	 OpenAI	200k	63
o1	 OpenAI	200k	62
DeepSeek R1	 deepseek	128k	60
o1-mini	 OpenAI	128k	54
Gemini 2.0 Pro Experimental	 Google	2m	47
DeepSeek V3	 deepseek	128k	46
Qwen2.5 Max	 Alibaba	32k	45
Gemini 1.5 Pro (Sep)	 Google	2m	45
Claude 3.5 Sonnet (Oct)	ANTHROPIC	200k	44

DeepSeek-R1-Distill

●3단계 학습인 Rejection Sampling에 사용했던

800,000 개의 데이터를 이용해, 작은 모델을 SFT만으로 추가 학습시켰다

- SFT만으로도 추론 성능을 끌어올림

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

TRL (Transformer Reinforcement Learning)

- <https://huggingface.co/docs/trl/index>
- SFTTrainer, RewardTrainer, PPO, DPO, GRPO 등 지원



Step 1: SFTTrainer

Train your model on your favorite dataset

```
from trl import SFTTrainer

trainer = SFTTrainer(
    "facebook/opt-350m",
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=512,
)

trainer.train()
```

Step 2: RewardTrainer

Train a preference model on a comparison data to rank generations from the supervised fine-tuned (SFT) model

```
from trl import RewardTrainer

trainer = RewardTrainer(
    model=model,
    args=training_args,
    tokenizer=tokenizer,
    train_dataset=dataset,
)

trainer.train()
```

Step 3: PPOTrainer

Further optimize the SFT model using the rewards from the reward model and PPO algorithm

```
from trl import PPOConfig, PPOTrainer

trainer = PPOTrainer(
    config,
    model,
    tokenizer=tokenizer,
)

for query in dataloader:
    response = model.generate(query)
    reward = reward_model(response)
    trainer.step(query, response, reward)
```


Take-Home : 강화 학습은 어떻게 준비하는가?

- 보상 함수를 사용하는 PPO는 오버헤드가 매우 크다

- 대안으로, Rejection Sampling을 고려할 수 있다

- DPO도 일반 SFT 학습보다는 GPU 요구량이 큰 작업이므로,

- 결과물의 퀄리티를 보고 신중하게 고려하기

- GRPO는?

- DeepSeek은 성공했으나, 일반적인 학습에서 모두 효과적일지는 아직 기다려 봐야 함
- 적절한 보상 함수 및 로직에 대한 고민 필요

Serving/Deployment

변형호

LLM 어플리케이션 서빙하기

- LLM 모델/Workflow 구성 후의 단계
- 다양한 서비스 형태
 - Python 코드 / WebUI / API와 같은 고려사항
- 환경 / 활용 목적에 따라 적절한 사용 필요
 - Streamlit, vLLM, ...

LLM Application Deployment

- Concurrent Usage, Throughput을 고려한 LLM 모델 서빙
 - 서버 레벨의 로드밸런싱 등도 고려할 수 있음
- OpenAI API 형태의 서빙이 가능한 라이브러리 사용하기
 - Ollama(Toy), vLLM, Sglang

vLLM: 가장 대표적인 추론 최적화 라이브러리

- <https://docs.vllm.ai/en/latest/>

- Page Attention 기반
- GPU Utilization을 늘려 추론 캐시를 늘림
- OpenAI API 형태로 서빙 후 LangChain과 연동이 쉬움
- Speculative Decoding 등의 기능 지원

- LoRa

- <https://docs.vllm.ai/en/v0.5.5/models/lora.html>

- Linux/Docker 환경 필요

Streamlit (<https://streamlit.io/>)

- 매우 간단한 실행방식

- 프론트엔드 개발 없이 데이터 앱 개발 및 배포 가능
- 드롭다운, 입력창, 파일 첨부 등의 기능 쉽게 추가 가능

- LLM의 Generation과 자연스럽게 연결 가능

- LangChain과 연결된 코드가 많음

- API는 미지원



Ch.10 Deployment

Streamlit + LLM

● LLM 어플리케이션을 위한 Chat 인터페이스 지원

– Ex) Prompt Engineering

Model

gpt-3.5-turbo-0125

KR

☒ 역할

☒ 대상

☒ 목적

☒ 포인트

☒ 분위기

☐ 형식

☐ 드래프트 수

역할:

마케팅 전문가

대상:

신규개설 고객

목적:

연금저축 가입 권유

포인트:

연 최대 99만원 세액공제(총 급여 5500 이하만

분위기:

힘차고 밝게, 새해 다짐 느낌으로

당신의 역할은 마케팅 전문가입니다. 아래의 상황에 대한 이벤트 광고 문구를 작성하세요. 이벤트의 대상은 신규개설 고객입니다. 이벤트의 목적은 연금저축 가입 권유입니다. 특별히 강조해야 하는 포인트는 연 최대 99만원 세액공제(총 급여 5500 이하만)입니다. 분위기는 힘차고 밝게, 새해 다짐 느낌으로 작성하세요.

"새해, 새 도전! 99만원까지 세액공제 혜택으로 연금저축 시작하세요! 신규개설 고객 대상 특별 이벤트! 지금바로

Streamlit + LLM

- Streamlit은 입력이 발생할 때마다 전체를 리로드
- LLM/임베딩 등을 불러오는 부분은 캐싱으로 Wrap

- Ex)

```
@st.cache_resource
def load_chat_models():
    model_id='./model/' # 모델의 주소
    model = AutoModelForCausalLM.from_pretrained(model_id, device_map={"":0})
    tokenizer = AutoTokenizer.from_pretrained(model_id)

    pipe = pipeline("text-generation", model=model, tokenizer=tokenizer)
    return pipe
```


실습) Streamlit으로 LLM 어플리케이션 구현하기

LangSmith

● <https://smith.langchain.com>

- LLM 어플리케이션의 평가, 모니터링, 로깅을 지원
- 사용량 모니터링, Success/Failure 비율, A/B 테스트



LangSmith



Project: 개인화 sLLM 만들기

변형호

프로젝트 개요

● 기존의 배운 내용들을 모두 활용하여 현업에 활용할 sLLM 구상하기

- 모델의 활용 목적 정하기
 - RAG/QA/Chat
- 베이스 모델 선정
- 파인 튜닝
 - 방법 선정하기: 풀 파인 튜닝 vs PEFT
 - 도메인 데이터 생성하기
- 서빙 방법 정하기
 - Streamlit/ vllm/ 양자화

1. 모델의 활용 목적 정하기

● 어떤 작업을 맡길 것인가?

- RAG, 요약, Q/A
- 데이터 분석
- 채팅 (Single/Multi-Turn)
- 업무 자동화

● Tool Calling / Agent?

- Tool Calling에 특화된 모델 사용하기
- LangGraph 등의 워크플로우 구성 고려하기

2. 베이스 모델 선정하기

● 고려할 점

- 한국어 파인튜닝 모델
 - 또는 Multilingual Model
- 모델 크기 (GPU)
 - 하드웨어 제약에 맞춰야 함
- Instruction Tuning 여부
 - 기본 모델을 I.T부터 학습시키기 vs I.T 모델을 활용하기

3. 파인 튜닝 고려하기

● 모델의 정확도가 너무 떨어지는 경우

- 프롬프트 엔지니어링
- RAG
- 를 고려해도 부족한 경우

→ 파인 튜닝 필요

● 파인 튜닝: Full vs PEFT

- PEFT를 고려하는 경우:
 - GPU가 부족
 - PoC 레벨의 개발이 필요
 - 탈착 가능/ 여러 기능을 개별적으로 학습

3. 파인 튜닝 고려하기

● 도메인 데이터 생성하기

- GPT 4o-mini로 데이터를 만들면?
 - 고품질의 데이터를 얻기 어려움
- OpenAI O3로 만들면?
 - \$\$\$\$
- 고품질의 데이터는 Human (Domain Expert)의 개입이 필요함
 - 좋은 데이터가 들어가야 좋은 모델이 나온다
(Garbage In Garbage Out)

[프로젝트] sLLM 만들기

- 현업에 필요한 sLLM은 무엇이 더 필요할까?